Lecture 5: Pretraining and Chat Models: Tokens, Optimizers, Scale, and Fine-Tuning

A language model that has learned to predict the next token of a large amount of text can be turned into a model you can talk to. ChatGPT, released in November 2022, was made this way: a decoder trained on text from the web, then trained further on conversations. Lecture 4 ended with a much smaller decoder, one that continues children's stories. It read three and a half million words of stories, and it knows a fixed list of 8,682 lowercase words: a name it never saw becomes `<unk>`, a number disappears, and the one thing it can do is continue a story. Within five years of the Transformer, decoders like it were trained once on text from the web and then used for nearly every task in language: first by fine-tuning a copy of the model for each task, then by prompting it, and from 2022 by talking to it (Figure 1). This lecture takes our story model through the four steps that lead to such a model.

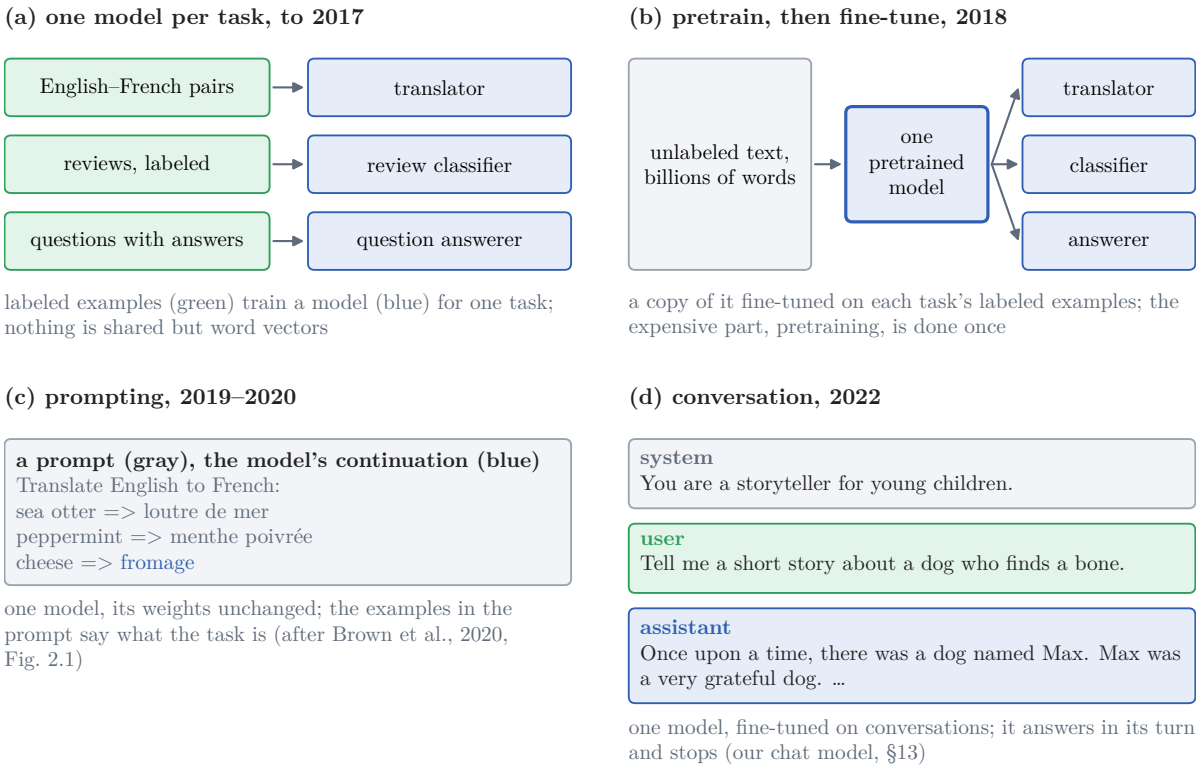


Figure 1. Four ways of getting a task done, 2017 to 2022. (a) Before 2018, each task had its own model, trained from scratch on labeled examples of that task. (b) From 2018, one model was pretrained on unlabeled text and a copy of it fine-tuned for each task. (c) From 2019–2020, one model did a task it was shown in its input, with no training for it (the prompt after Brown et al., 2020, Fig. 2.1). (d) From 2022, one model fine-tuned on conversations answers a request in its turn and stops: here our chat model of §13, its answer cut short.

Tokens. First, the model has to be able to read and write any text, not just the words on a list. Every text is a sequence of bytes, so the model could simply read bytes; but then a story is four times longer than in words, and attention's cost grows with the square of the length. **Byte-pair encoding** is the compromise. It starts from the 256 bytes and keeps merging the most frequent pair of adjacent tokens into a new token, so that frequent words end up as single tokens while anything else can still be spelled out. The tokenizer is fixed before training, and it decides more than it seems to: the size of the model's table, the length of every text, and what the model can see of numbers and of other languages.

Pretraining. Next, the model reads far more text: our base model reads 49 million tokens of stories, and Llama 3 read more than 15 trillion tokens, mostly from the web. The text has to be collected, deduplicated, and cleaned of the texts the model will be tested on. The training itself is the loop you already know: a batch goes in, the loss and its gradient come out, and the optimizer takes a step. At this scale, though, the choices around that loop decide whether a run works: which optimizer (Adam, or the newer Muon), how many tokens per step, how the learning rate rises and falls, and in what precision the numbers are stored. A configuration file of a few dozen lines records all of them, and by the end of §8 you will be able to read one. **Scaling laws** then answer the question of budget: for a given amount of computation, how large should the model be, and how much should it read? About 20 tokens per parameter. What comes out is a **base model**. It continues any text you give it, often well, but if you ask it a question, it simply continues the document.

Midtraining. A base model has never seen a conversation, and the special tokens that will mark a conversation's turns mean nothing to it. **Midtraining** teaches both cheaply: we continue pretraining for a short while on a mixture that adds conversations, and tasks such as summarizing a story, while the learning rate decays to zero. The model picks up the format and the new tasks, and keeps what it knew.

A chat model. Finally, to answer, the model has to know whose turn it is and when to stop. So a conversation is written as one sequence of tokens: a **system** message, the **user**'s turns, and the **assistant**'s turns, each between the special tokens of a **chat template**. **Supervised fine-tuning** then trains it on demonstrations: conversations whose assistant turns show what it should write. Only the assistant's tokens are in the loss, and training can change every weight or, with **LoRA**, add a small low-rank change to them. The model learns to answer in its turn and to stop. What it cannot learn this way is anything the base model never read, and since it only imitates, nothing tells it which of its answers are good. Rewards, the last two steps of ChatGPT's recipe, add that in Weeks 6 and 7.

The lecture has four parts:

1. after the Transformer (§1): from pretraining and fine-tuning to prompting, scale, and ChatGPT's recipe, 2017–2024;
2. tokens (§§2–4): bytes and Unicode; byte-pair encoding; special tokens, bits per byte, and what tokenizers get wrong;
3. pretraining (§§5–10): data; optimizers in brief (developed in full in the companion tutorial, *Optimizers for Deep Learning*); the training recipe; reading a training configuration, with what real runs cost; scaling laws; the base model;
4. a chat model (§§11–14): conversations as tokens, chat templates, and system prompts; midtraining; supervised fine-tuning; what fine-tuning changes, and what it does not.

1. After the Transformer: pretrain, then adapt

The Transformer of 2017 was a translator, trained for one task on pairs of sentences. Five years later, decoders like the one of Lecture 4 (Section 8) were trained once, on text without labels, and then used for nearly every task in language, including conversation. Figure 2 marks the steps; each is developed in a part of this lecture.

One model per task (to 2017). A translator learned from translations and a classifier of reviews from labeled reviews (Figure 1a). Only word vectors, such as those of Lecture 2 (Section 5), were learned from unlabeled text and reused. Labeled examples are expensive, and most text has no labels.

2017	**Transformer**	a translator trained for one task	
2018	**GPT, BERT**	pretrain on unlabeled text, then fine-tune for each task	§§5–10
2019	**GPT-2**	web text, byte-level BPE; tasks from a prompt alone	§§3, 5, 10
2019	**RoBERTa, T5**	the recipe matters; a filtered web corpus (C4)	§§5–8
2020	**GPT-3**	175 billion parameters; few-shot, in-context learning	§10
2020	**Kaplan et al.**	loss falls as a power law in parameters, data, compute	§9
2022	**Chinchilla**	parameters and tokens should grow together	§9
2022	**InstructGPT, ChatGPT**	fine-tuned on demonstrations, then with rewards	§§11–14; Weeks 6–7
2023	**LLaMA, Llama 2**	open base and chat models; system messages	§§11–13
2024	**Llama 3, OLMo 2**	15 trillion tokens; midtraining on a shifted mixture	§§9, 12

Figure 2. From one model per task to one model for every task. Selected steps from the Transformer (2017) to the open chat models of 2023–2024, one row per step, each with what it introduced and the section of this lecture that develops it. Pretraining (green), prompting (blue), scale (orange), and conversation (purple).

Pretrain, then fine-tune (2018). A language model can be trained first on unlabeled text, where predicting the next word forces it to learn whatever helps predict, and then trained further, **fine-tuned**, on the labeled examples of one task. Dai and Le (2015) did this with LSTMs; ELMo (Peters et al., 2018) reused the states of a pretrained LSTM language model as features, and ULMFiT (Howard and Ruder, 2018) fine-tuned a pretrained LSTM language model to classify text. GPT (Radford et al., 2018) did the same with the Transformer's decoder, and BERT (Devlin et al., 2019) with its encoder, trained to fill in blanks (their designs are in Lecture 4 (Section 8)). From then on, the expensive part of a model, learned once from unlabeled text, was shared by every task (Figure 1b).

Data, tokens, and the recipe (2019). GPT-2 (Radford et al., 2019) was trained on WebText, the text of about 8 million web pages linked from Reddit, split into tokens by a byte-level BPE (§3). T5 (Raffel et al., 2020) built a larger corpus, C4, from the Common Crawl archive of the web with a few simple filters (§5). RoBERTa (Liu et al., 2019) found that BERT had been undertrained: the same architecture, trained longer with larger batches on ten times as much text, did much better. How a model is trained matters as much as its design (§§6–7).

Prompting instead of fine-tuning (2019–2020). GPT-2 did some tasks without any training for them, given only a suitable text to continue: an article followed by "TL;DR:" is continued with a summary of it. GPT-3 (Brown et al., 2020), with 175 billion parameters trained on 300 billion tokens, did many tasks from a few examples written into its input, with no change to its weights (Figure 1c). This is **in-context learning** (§10).

Scale (2020–2022). Kaplan et al. (2020) found that a language model's loss falls as a power law in its parameters, its data, and its computation. Hoffmann et al. (2022) found that the large models of the time had too many parameters for the text they had read: Chinchilla, with 70 billion parameters trained on 1.4 trillion tokens, did better than Gopher, with 280 billion on 300 billion tokens, at the same cost (§9).

Conversation (2022). A base model continues text. Given a request, it treats the request as the start of a document and writes what such a document might contain next, often more requests (Figure 3a). InstructGPT (Ouyang et al., 2022) turned GPT-3 into a model that follows instructions in three steps (Figure 4):

1. **supervised fine-tuning**: labelers wrote answers to about 13,000 prompts (Figure 3b), and GPT-3 was fine-tuned on them;
2. **a reward model**: labelers ranked several of the fine-tuned model's answers to the same prompt, and a model was trained to score answers as they ranked them;
3. **reinforcement learning**: the fine-tuned model was trained further to write answers that the reward model scores highly, by an algorithm called PPO.

(a) GPT-3, a base model, continues the request

prompt (gray), continuation (blue)
Explain the moon landing to a 6 year old in a few sentences.
Explain the theory of gravity to a 6 year old.
Explain the theory of relativity to a 6 year old in a few sentences.
Explain the big bang theory to a 6 year old.
Explain evolution to a 6 year old.

OpenAI (2022), from the blog post introducing InstructGPT

(b) a person's answer, to fine-tune on

prompt (gray), a labeler's demonstration (green)
Explain the moon landing to a 6 year old
Some people went to the moon...

Ouyang et al. (2022), Fig. 2; the demonstration is cut short there

Figure 3. A base model continues a request; fine-tuning shows it an answer instead. (a) GPT-3 given a request, and what it wrote next: more requests of the same kind (from OpenAI's post introducing InstructGPT, 2022). (b) A request of the same kind with the start of a demonstration written by one of InstructGPT's labelers, the kind of text supervised fine-tuning trains on (Ouyang et al., 2022, Fig. 2, where it is cut short).

People preferred the answers of a 1.3-billion-parameter InstructGPT to those of the 175-billion-parameter GPT-3. ChatGPT (OpenAI, 2022), released on 30 November 2022, used the same three steps on conversations: it was fine-tuned from a model of the GPT-3.5 series that had finished training in early 2022, first on conversations in which people wrote both the user's turns and the assistant's, mixed with InstructGPT's data rewritten as dialogues, and then with rewards from people's rankings of its answers.

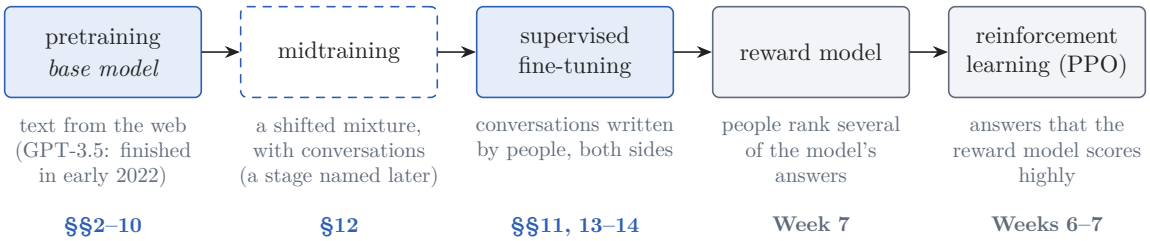


Figure 4. ChatGPT's recipe of 2022: a base model, then three steps. After Ouyang et al. (2022, Fig. 2). Pretraining gives the base model; supervised fine-tuning on demonstrations, written by people, gives a model that answers; a reward model trained on people's rankings of its answers, and reinforcement learning against that reward, give the final model. Below each stage, where it is developed: this lecture builds the first two stages and a stage between them, midtraining; the last two are Weeks 6 and 7.

Open models (2023 on). LLaMA (Touvron et al., 2023a) trained models of 7 to 65 billion parameters on public text only, and its 13-billion-parameter model did better than GPT-3 on

most benchmarks. Llama 2 (Touvron et al., 2023b) released chat models whose conversations begin with a system message (§11), and Llama 3 (Llama Team, 2024) trained its models of 8 and 70 billion parameters on 15 trillion tokens, far more per parameter than the compute-optimal rule of §9 suggests.

Stages (2024 on). Training came to be described in stages: **pretraining** on trillions of tokens of general text; **midtraining**, a shorter continuation of pretraining on a mixture shifted toward higher-quality text, mathematics, code, longer documents, or conversations, usually while the learning rate decays (Llama Team, 2024; Team OLMo, 2024); and **post-training**, supervised fine-tuning followed by training with rewards.

This lecture builds, at our scale, the first stages of that recipe for the decoder of Lecture 4: a tokenizer that turns text into tokens (§§2–4), pretraining on a corpus with an optimizer (§§5–10), midtraining (§12), and supervised fine-tuning on conversations written in a chat template (§§11, 13–14). The result is a model trained the way ChatGPT's was before its rewards. The tokens come first.

2. Text as bytes

What should a model read, so that it can read and write any text? Take the conversation that the chat model of §13 will answer, a system message and a request:

> **system:** You are a storyteller for young children.
>
> **user:** Tell me a short story about a dog who finds a bone.

The word model of Lecture 4 reads words from a fixed list: the lowercase words and punctuation marks that occur at least twice in its training stories, 8,682 tokens with `<unk>` and the EOS and BOS tokens that end and begin each story. Every other word is replaced by `<unk>`, the same token for every unknown word. On text like its training stories this is rare: 0.2% of the words of the validation stories, many of them names (*maisie*, *bubba*, *stripey*). Outside them it fails at once (Figure 5). The system message loses *storyteller*; Lecture 3's French caption becomes eleven `<unk>`s; and the sum *47 + 385 = 432* disappears entirely, since the word tokenizer keeps only letters and punctuation. The model also writes in lowercase only, because its words were lowercased.

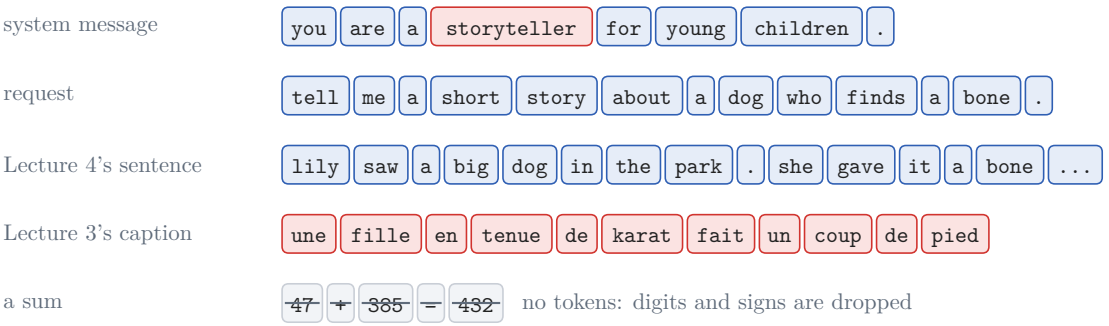


Figure 5. A fixed list of words cannot read what is not on the list. Five texts as Lecture 4's word model reads them: the running conversation's two messages, Lecture 4's running sentence, the French caption of Lecture 3, and a sum. Words outside the model's list (red) are read as <unk>; *karaté* has already lost its *é*; the sum's digits and signs are dropped before the list is consulted (gray, struck through).

A model that reads characters cannot meet an unknown word. But it reads many more positions: a story of our validation set has on average 875 characters against 204 word tokens, 4.3 times as many. Attention's work per token grows with the number of positions it reads and its total work with their square (Lecture 4 (Section 10)), so reading the same story as characters costs about 18 times as much in attention (4.3^2), and the key–value cache needed to write it is 4.3 times as long.

Characters, code points, and bytes

How does a computer store text at all? As numbers, one per character, and the history of which numbers is short. **ASCII**, the American standard of 1963, numbered 128 characters, the English letters, digits, punctuation, and some control codes, so that each fits in 7 bits. Computers store 8 bits to a byte, and the 128 values left over were soon filled in many incompatible ways: **code pages** such as Latin-1 (ISO 8859-1, 1987) for Western European languages, others for Greek, Cyrillic, Arabic, or Hebrew, and multi-byte schemes for Chinese and Japanese. The same byte, 233, was *é* in Latin-1 and a Cyrillic letter in the Russian code page, and a text read with the wrong code page came out garbled. **Unicode** (1991) replaced them all with one numbering of every character of every script: each character has a **code point**, an integer written in hexadecimal after U+, such as U+0041 for *A* and U+00E9 for *é* (Figure 6). Code points run from U+0000 to U+10FFFF, room for 1,114,112, of which Unicode 18.0 (2026) assigns 172,808 (Unicode Consortium, 2026). A code point is not always a letter you see: the Hindi word *namaste* is three syllables on the page and six code points, since its vowel signs and the mark that joins two consonants are code points of their own.

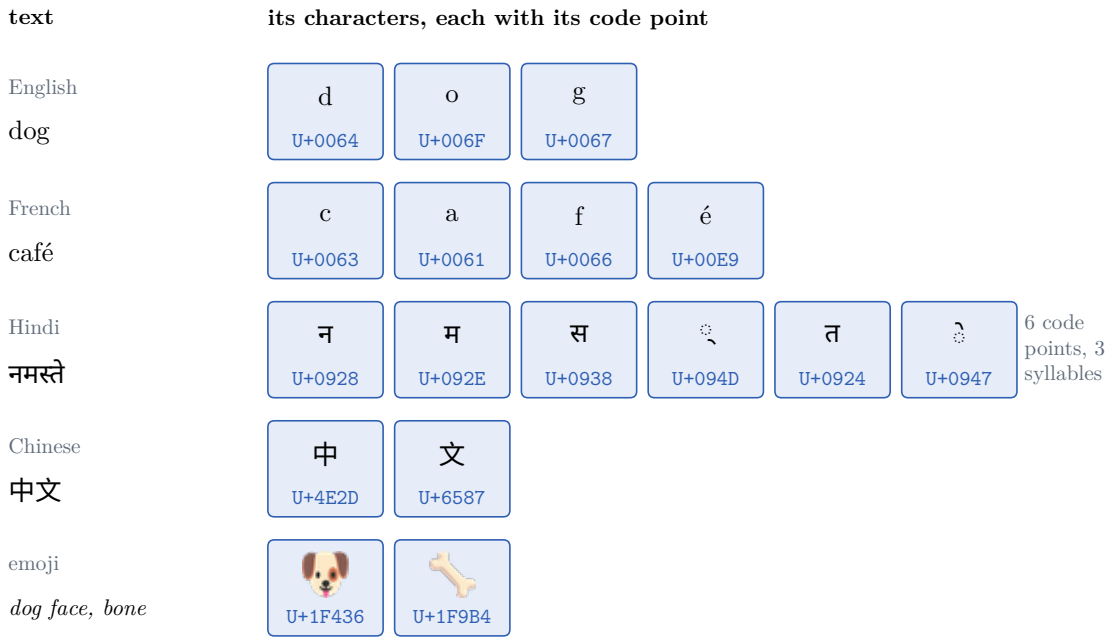


Figure 6. Unicode gives every character a number, its code point. Five short texts and the code points of their characters: English and French letters (those below U+0080 are ASCII's); the Hindi greeting *namaste*, whose vowel signs and the mark joining *s* and *t* are code points of their own (drawn on a dotted circle) and combine with the letters before them; the two characters of the Chinese word for the Chinese language; and the emoji of a dog's face and of a bone, beyond U+FFFF.

A model whose tokens were code points would need a table of 172,808 rows, most for characters it never sees, and would still have to choose what to do with characters assigned later. So what does a file hold? Files and networks store **bytes**, numbers from 0 to 255, and an **encoding**

writes each code point as bytes. The obvious one, **UTF-32**, writes every code point as four bytes, which makes English text four times longer than ASCII. **UTF-16**, used inside Windows, Java, and JavaScript, writes the code points below U+10000 as two bytes and the rest, such as emoji, as four. The encoding of the web, used by 99% of the websites whose encoding is known (W3Techs, 2026), is **UTF-8**, designed by Ken Thompson and Rob Pike in 1992 (Pike and Thompson, 1993). It uses one to four bytes per code point (Figure 7):

1. a code point below 128, an ASCII character, is one byte, its own number, so English text in ASCII is unchanged;
2. a larger code point is split into groups of bits: the first byte begins with as many 1s as the character has bytes, then a 0 (110 for two bytes, 1110 for three, 11110 for four), and every following byte begins with 10.

So no byte of a longer character can be mistaken for an ASCII character, and a reader that starts in the middle of a text finds the start of the next character at the next byte that does not begin with 10.

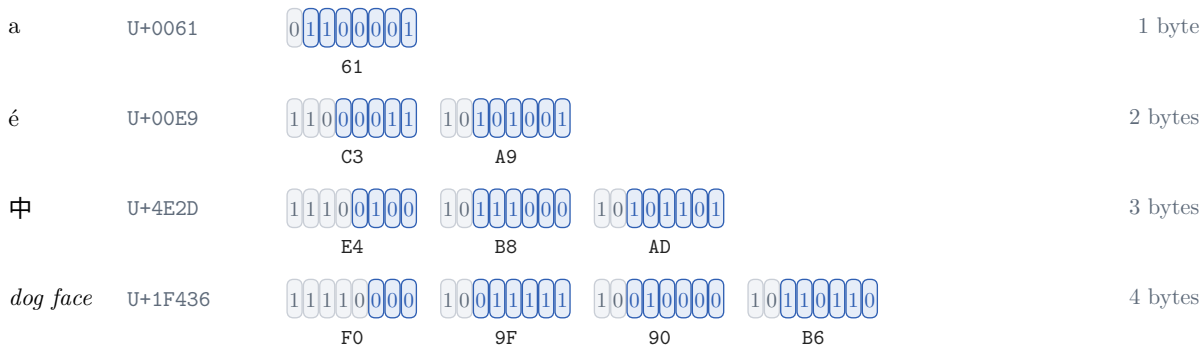


Figure 7. UTF-8 writes a code point as one to four bytes. One character of each length: *a* (U+0061, one byte), *é* (U+00E9, two), the Chinese character U+4E2D (three), and the dog-face emoji U+1F436 (four). For each: the code point's bits (blue) distributed into the bytes' free positions after the fixed prefix bits (gray); and the bytes in hexadecimal. UTF-16 would write the four in 2, 2, 2, and 4 bytes, and UTF-32 in 4 each.

A model that reads bytes has a vocabulary of 256 tokens, plus any special tokens (§4), and can read and write every text in every language: anything that can be stored can be spelled. The running conversation is 41 bytes of system message and 51 bytes of request. The cost is the length: English text is about one byte per character, and Chinese about three.

Tokens: the table against the length

Between bytes and words lies a trade-off between two parts of Lecture 4's model that the tokenizer sets, one of fixed size and one that grows with the text (Figure 8):

- **The vocabulary fixes the token table.** A vocabulary of $|\mathcal{V}|$ tokens needs a table of $|\mathcal{V}| \times d$ numbers, used at the input and, tied, at the output, where every position computes $|\mathcal{V}|$ logits at $2|\mathcal{V}|d$ operations (Lecture 4 (Section 10)). And every token must appear often enough in training for its row to be learned.
- **The number of tokens in a text scales everything else.** The weights' work, $2N$ operations per token, attention's $4LT^2d$, and the cache's $2LdT$ numbers all grow with the number of tokens T needed for a given text, and the context window, a fixed number of tokens, holds less text when tokens are short.

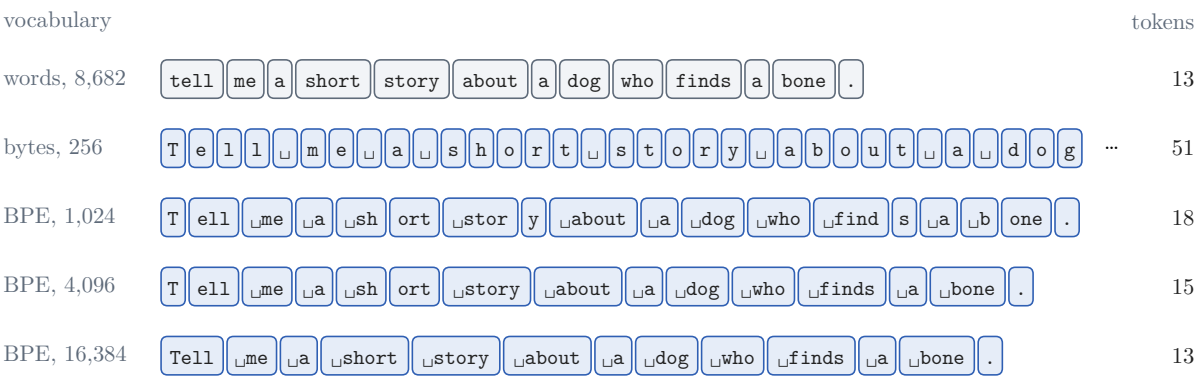


Figure 8. Larger vocabularies make texts shorter. The running request, *Tell me a short story about a dog who finds a bone.*, split into words (lowercased, as the word model's list of 8,682 reads it), into bytes (256), and into the pieces of byte-pair encodings with 1,024, 4,096, and 16,384 tokens (§3); the size of the vocabulary is at the left of each row, the number of tokens at the right. A box is one token, and ␣ is a space.

Words make texts short but fail on new words; bytes never fail but make texts long. The next section learns a vocabulary in between from the text itself.

3. Byte-pair encoding

Which pieces should the vocabulary hold? Frequent words should be single tokens, so that common text is short; rare words should still be spelled from smaller pieces, so that nothing is unknown. **Byte-pair encoding** (BPE) learns such a vocabulary from the text itself: start from single characters and repeatedly merge the pair of adjacent tokens that occurs most often into a new token.

A tokenizer has two jobs, done at different times (Figure 9):

1. **training the tokenizer**, once, before the model is trained: from a sample of training text, BPE learns the vocabulary and the order in which its merges were made;
2. **encoding**, every time text goes into the model, while it is trained and whenever it is used: the learned merges, applied in that order, turn a new text into tokens. **Decoding** turns tokens back into text by writing out the bytes of each token in turn.

The training text is used only by the first; the merges are all that the second needs.

Training the tokenizer, by hand

Take a corpus of ten words, with how often each occurs: *hug* 10, *hugs* 5, *hugged* 4, *bug* 6, *bugs* 3, *dog* 8, *dogs* 4, *big* 5, *bigger* 2, *bag* 3. Training (Figure 10):

1. Write every word as a sequence of single characters, the first vocabulary.
2. Count every pair of adjacent tokens inside the words, each occurrence weighted by its word's count. The pair $u \cdot g$ occurs in *hug*, *hugs*, *hugged*, *bug*, and *bugs*: $10 + 5 + 4 + 6 + 3 = 28$ times, more than any other.
3. Add the most frequent pair to the vocabulary as a new token, *ug*, record the merge, and replace the pair in every word: $h \cdot ug$, $b \cdot ug \cdot s$.
4. Repeat from step 2 until the vocabulary has the size wanted. The second merge is $h \cdot ug$ (19 times), the third and fourth build *dog*, the fifth *bug*, the sixth *bi*.

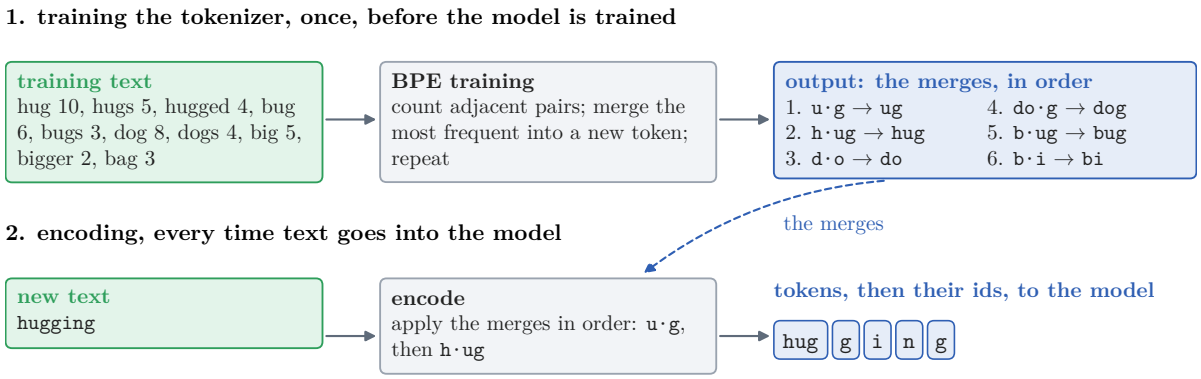


Figure 9. A tokenizer is trained once, then used to encode every text. Training (top) reads a sample of training text, here the ten words of the example below with their counts, and writes an ordered list of merges; the merges and the starting symbols are the vocabulary. Encoding (bottom) applies those merges, in order, to any text, here *hugging*, which the training text never contained, and gives the tokens whose ids the model reads.

The result is an ordered list of merges, and the training text is no longer needed.

word, count		start	1. u·g (28)	2. h·ug (19)	3–4. d·o, do·g (12, 12)	5. b·ug (9)	6. b·i (7)
hug	10	h·u·g	h·ug	hug	hug	hug	hug
hugs	5	h·u·g·s	h·ug·s	hug·s	hug·s	hug·s	hug·s
hugged	4	h·u·g·g·e·d	h·ug·g·e·d	hug·g·e·d	hug·g·e·d	hug·g·e·d	hug·g·e·d
bug	6	b·u·g	b·ug	b·ug	b·ug	bug	bug
bugs	3	b·u·g·s	b·ug·s	b·ug·s	b·ug·s	bug·s	bug·s
dog	8	d·o·g	d·o·g	d·o·g	dog	dog	dog
dogs	4	d·o·g·s	d·o·g·s	d·o·g·s	dog·s	dog·s	dog·s
big	5	b·i·g	b·i·g	b·i·g	b·i·g	b·i·g	bi·g
bigger	2	b·i·g·g·e·r	b·i·g·g·e·r	b·i·g·g·e·r	b·i·g·g·e·r	b·i·g·g·e·r	bi·g·g·e·r
bag	3	b·a·g	b·a·g	b·a·g	b·a·g	b·a·g	b·a·g

Figure 10. Byte-pair encoding merges the most frequent adjacent pair, again and again. A corpus of ten words with their counts (50 words, 180 characters). One row per word, one column per merge: each merge's pair and its count (summed over the words that contain it, each weighted by its count) head its column, and the column shows every word after the merge, its tokens separated by dots. After six merges the corpus is 93 tokens. Merges 3 and 6 break ties, $d \cdot o$ against $o \cdot g$ (12 each) and $b \cdot i$ against $i \cdot g$ (7 each), by taking the pair whose characters come first in the alphabet.

Encoding new text

Encoding a new word applies the merges in the order they were learned:

1. Write the word as characters.
2. Among the adjacent pairs of the word, find the one whose merge was learned first, and merge it everywhere in the word.
3. Repeat until no adjacent pair has a merge.

So *dug*, which the corpus never contained, becomes $d \cdot ug$, and *hugging* becomes $hug \cdot g \cdot i \cdot n \cdot g$: the frequent part is one token, the rest is spelled.

Greedy compression

Each merge removes the most occurrences it can, one per occurrence of the pair, so BPE is a greedy compressor: after six merges the corpus of 180 characters is 93 tokens. It was introduced as a data-compression algorithm by Gage (1994), which replaced the most frequent pair of bytes by an unused byte, and brought to language by Sennrich et al. (2016) to translate rare words as sequences of pieces: the 37,000 shared pieces of the Transformer of Lecture 4 (Section 7) came from it. Greedy is not optimal: no merge is undone, and a vocabulary chosen all at once could encode the corpus in fewer tokens; how far from optimal it can be is in Appendix A.

Byte-level BPE

Starting from characters brings back the problem of §2: a character never seen in training has no token. GPT-2 (Radford et al., 2019) started from the 256 bytes instead, so that every text can be encoded: its 50,257 tokens are the 256 bytes, 50,000 merges, and one end-of-text token. A merge of two bytes may form half a character, or a character and a half; the tokens need not be valid UTF-8 on their own, only the whole text.

Run over raw text, merges would also cross word boundaries and produce tokens such as *dog.* and *dog!* beside *dog.* So the text is first split into **pre-tokens** by a regular expression, and merges happen only inside a pre-token. GPT-2's expression, which ours follows (Figure 11), keeps:

1. a run of letters with at most one space before it (␣*dog*), so that a word and the space before it become one token;
2. common English endings after an apostrophe (*'s*, *'t*, *'ll*);
3. a run of punctuation or other symbols, with at most one space before it;
4. runs of spaces and line breaks.

Ours differs in one place: it makes every digit its own pre-token, as Llama's tokenizer does (Touvron et al., 2023a), so that a number is always written digit by digit (§4).

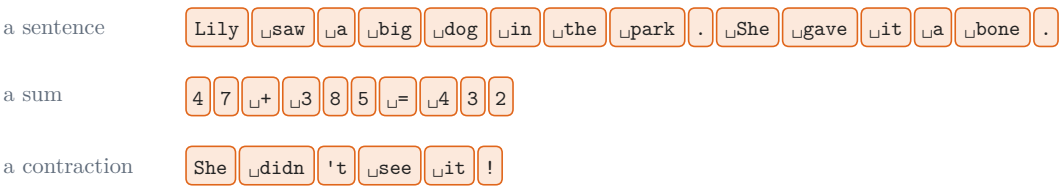


Figure 11. Pre-tokens keep merges inside words, numbers, and runs of punctuation. A sentence, a sum, and a contraction, split by our regular expression; each box is a pre-token, and ␣ is a space. BPE merges bytes only inside a box.

Our tokenizer

Our tokenizer is a byte-level BPE trained on the first 100,000 stories of our training text (§5) with their share of sums, written in plain Python; its 3,832 merges take about 20 seconds to learn. Its vocabulary has 4,096 tokens: the 256 bytes, 3,832 merges, and 8 special tokens (§4).

The first merges join frequent letters and attach spaces: `he`, `␣t`, `␣a`, `␣s`, `␣w`, `nd`, `␣the`, `ed`, `␣and`, `␣to`. On the validation stories it writes 3.99 bytes per token.

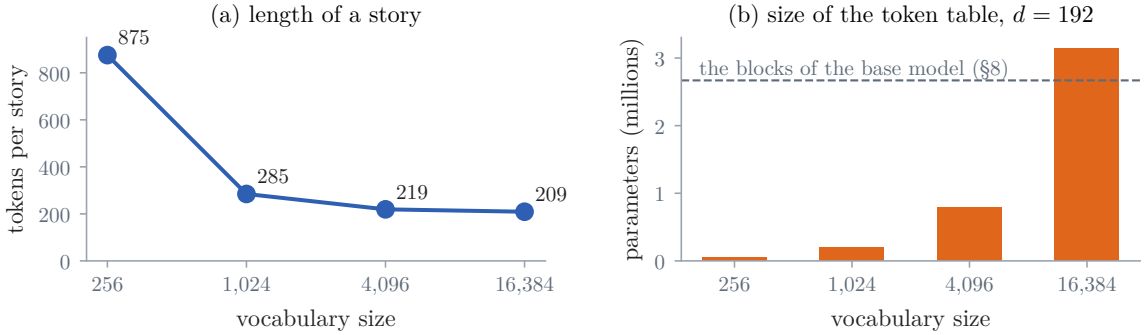


Figure 12. Most of the shortening comes from the first few hundred merges, while the table grows with every token. For tokenizers of 256 (bytes), 1,024, 4,096, and 16,384 tokens trained on the same text: (a) the average number of tokens per validation story (a story is 875 bytes); (b) the number of parameters of the token table at $d = 192$, against the 2.7 million in the blocks of the base model of §8.

The size of the vocabulary is a choice (Figure 12). With the bytes alone a validation story is 875 tokens on average; 760 merges bring it to 285, 3,832 to 219, and 16,120 to only 209: TinyStories uses a small vocabulary, and the first few hundred merges bring most of the gain. The table grows in proportion to the vocabulary, from 49,152 numbers at $d = 192$ for the bytes to 3.1 million for 16,384 tokens, more than the 2.7 million in the rest of the model of §8.

In our 4,096-token vocabulary, Lecture 4's running sentence is one token per word, the space included in the token: *Lily ␣saw ␣a ␣big ␣dog ␣in ␣the ␣park . ␣She …*, 19 tokens. The running conversation is not so simple:

```
You·␣are·␣a·␣story·t·ell·er·␣for·␣young·␣children·.
T·ell·␣me·␣a·␣sh·ort·␣story·␣about·␣a·␣dog·␣who·␣finds·␣a·␣bone·.
```

Words that are rare in children's stories, such as *storyteller*, *short*, and an imperative *Tell* at the start of a sentence, are split into pieces. A tokenizer fits the text it was trained on.

4. What a tokenizer fixes

A tokenizer is trained once, before the model, and every later stage inherits it: the model's table has one row per token, the training text is stored as token ids, and every number reported about the model, a loss per token or a context length, is counted in its tokens. What does that decide?

Special tokens

Some tokens mark structure rather than text. Our vocabulary ends with eight **special tokens**, which no merge produces:

1. `<|endoftext|>` separates documents in the training stream (§5), so that every document follows one, the role of Lecture 4's BOS;
2. `<|system|>`, `<|user|>`, `<|assistant|>`, and `<|end|>` mark the turns of a conversation, unused until §11;
3. three more are reserved, so that later uses need no new rows in the table.

The encoder never produces a special token from text: a user who types the characters `<|endoftext|>` gets the bytes of that string, 8 ordinary tokens, and only the program that builds the model's input can insert the token itself. This keeps text that comes from outside, a web page or a user's message, from posing as the structure around it.

Bits per byte

Two models with different tokenizers cannot be compared by their loss per token. A model whose tokens are longer makes fewer predictions on a text, each harder, so its loss per token is larger even if it predicts the text better. The fair measure is per byte of text:

$$\text{bits per byte} = \frac{1}{B \ln 2} \sum_{t=1}^{n} -\ln q(x_t \mid x_{<t}),$$

where the text is B bytes long and its n tokens are predicted with total log loss $\sum_t -\ln q(x_t \mid x_{<t})$ nats; dividing by $\ln 2$ turns nats into bits. It is the number of bits a compressor driven by the model would need for each byte of the text, and it does not depend on how the text was split. A loss of ℓ nats per token converts by the tokenizer's average length: bits per byte $= \ell/(\ln 2 \times \text{bytes per token})$. Bits per character, used for character models in Lecture 2 (Section 3), is the same measure for text of one byte per character.

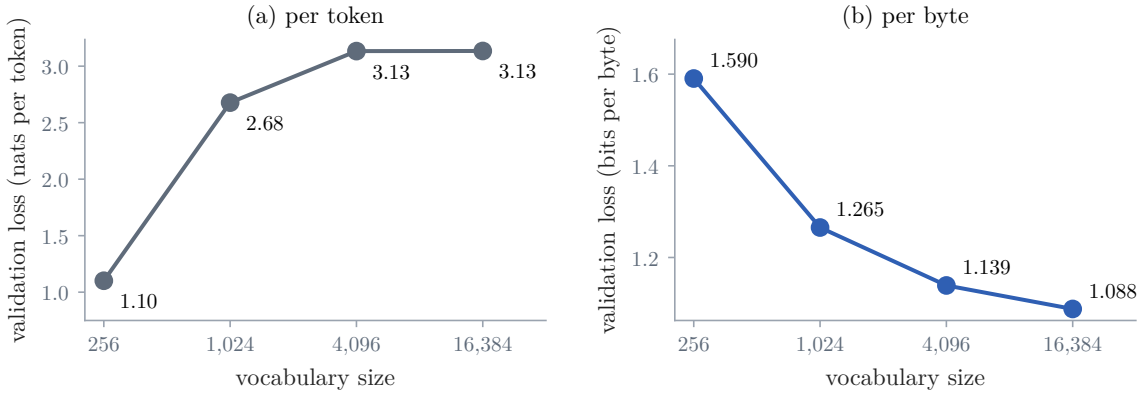


Figure 13. Measured per byte, longer tokens predict the same text better, with shrinking gains. Models with the same four blocks ($d = 128$) trained on the same first 10 million bytes of our training text with tokenizers of 256, 1,024, 4,096, and 16,384 tokens, each reading 1,024 bytes of context: validation loss (a) in nats per token and (b) in bits per byte. In (a) the larger vocabularies look worse; in (b), the measure that does not depend on the tokenizer, every larger vocabulary does better. Single runs.

Measured this way (Figure 13), the model that reads bytes does worst, at 1.59 bits per byte, and each larger vocabulary does better, 1.27, 1.14, and 1.09, with the gains shrinking. Per token the order is reversed, 1.10 nats for the bytes against 3.13 for 4,096 tokens, because a token of the 4,096 carries about four bytes of this text. The comparison is not between equals: a larger vocabulary also adds parameters, 2.1 million in the table of 16,384 tokens against 0.8 million in the blocks, and a model with longer tokens reads the same 1,024 bytes in fewer positions: 255 with 4,096 tokens, against 1,024 with bytes.

What tokenizers get wrong

The tokenizer decides what the model sees, and some of what it hides is hard to recover later. Four kinds of text show it, each with real tokenizers: GPT-2's (50,257 tokens, which GPT-3 reused), GPT-4o's (about 200,000), and ours.

Numbers. To add 432 and 4320 by hand, you line the two numbers up by place, ones under ones and tens under tens. GPT-2's tokenizer hides the places. It keeps a run of digits together as one pre-token, and its merges then cut the run into pieces that were frequent in its training text, so the same digits are cut differently in different numbers: *432* is one token, *4320* is *43* · *20*, and *43200* is *43* · *200* (Figure 14a). In the sum *432* + *4320* = *4752*, the answer's *4* is a token of its own and *752* another, and nothing in a token says which places its digits hold (Figure 14b). The model has to learn, for each of thousands of pieces, how many digits it has and where they sit, and arithmetic is harder for it. GPT-4o's tokenizer cuts a run of digits into pieces of at most three, but counts them from the left, so *4320* is *432* · *0*. Ours writes every digit as its own token, as LLaMA's did (§3), so the ones digit of a number is always its last token.

(a) the digits 4, 3, 2 at the start of four numbers, in GPT-2's tokens



(b) one sum, three tokenizers

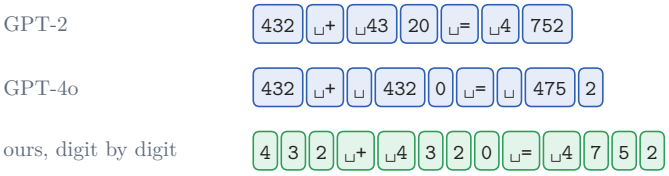


Figure 14. A run of digits cut by frequency hides the place of each digit. (a) GPT-2's tokenizer on four numbers that begin with the digits 4, 3, 2: the same three digits are one token, or are split two ways. (b) The sum 432 + 4320 = 4752 in GPT-2's tokenizer, in GPT-4o's (pieces of at most three digits, counted from the left), and in ours (one token per digit; the space before a number goes with its first digit). The tokens are those of the published vocabularies, r50k_base for GPT-2 and o200k_base for GPT-4o; ␣ is a space.

Spaces. The space before a word belongs to the word. In ours, ␣*dog* is one token, with its own row of the table, while *dog* at the start of a text is two other tokens, *d* · *og* (Figure 15a). A prompt that ends with a space therefore ends with a token that rarely occurs there: in training text a space almost always comes attached to the next word, so after a final ␣ the model must predict a word without its space, which it has seldom seen (Figure 15b). A run of spaces, as in indented code, costs one token per space in GPT-2's tokenizer and in ours, which never saw such runs to merge; GPT-4o's, trained on much more code, has a single token for seven spaces (Figure 15c).

Rare strings. A token can be frequent in the tokenizer's training text and rare in the model's. GPT-2's vocabulary contains ␣*SolidGoldMagikarp*, the name of a user who posted often in one forum of Reddit, as a single token (Figure 16a). The model rarely read it, so its row of the table was barely trained, and asked to repeat the string, GPT-3 answered *distribute* (Rumbelow and Watkins, 2023).

Other languages. A tokenizer trained mostly on English splits other languages into more pieces, so the same content costs more tokens: a fixed context holds less of it, and each request costs more (Figure 17). Our running request takes 13 of GPT-2's tokens in English, 21 in French, 37 in Chinese, and 97 in Hindi, where most of its tokens are single bytes; GPT-4o's tokenizer, with four times as many tokens learned from more languages, writes the Hindi in 37. Across the languages of a parallel corpus, Petrov et al. (2023) found differences of up to 15 times in

(a) a word with and without its space: two tokens, two rows of the table (ours)

`" dog"` `␣dog` id 632 `"dog"` `d` `og` ids 100, 2,164

(b) a prompt that ends with a space ends with a rare token (ours)

no space at the end	`Once` `␣upon` `␣a` `␣time` `,` `␣there` `␣was` `␣a`
a space at the end	`Once` `␣upon` `␣a` `␣time` `,` `␣there` `␣was` `␣a` `␣`

(c) indented code: eight spaces before `return`

GPT-2	`def` `␣f` `(` `x` `):` `\n` `␣` `␣` `␣` `␣` `␣` `␣` `␣` `␣return` `␣x`
GPT-4o	`def` `␣f` `(x` `):\n` `␣␣␣␣␣␣␣` `␣return` `␣x`
ours	`de` `f` `␣f` `(` `x` `)` `:` `\n␣` `␣` `␣` `␣` `␣` `␣` `␣` `␣return` `␣` `x`

Figure 15. A space belongs to the token after it. (a) In ours, *␣dog* and *dog* are different tokens, with different rows of the table. (b) A prompt that ends with a space ends with the space as a token of its own (red), where training text almost always has it attached to the next word. (c) Two lines of code, the second indented by eight spaces, in GPT-2's tokenizer, GPT-4o's, and ours; \n is a line break.

(a) one user name, two tokenizers

GPT-2	`␣SolidGoldMagikarp` one token, id 43,453
GPT-4o	`␣Solid` `Gold` `Mag` `ik` `arp`

(b) asked to repeat it, GPT-3 (davinci-instruct-beta, temperature 0)

prompt
Please can you repeat back the string '`␣SolidGoldMagikarp`' to me?

answer
"distribute"

Figure 16. A token that the model rarely read is barely trained. (a) A Reddit user's name is one token of GPT-2's vocabulary (orange) and five of GPT-4o's. (b) Asked to repeat it, GPT-3, which used GPT-2's vocabulary, answered with a different word (Rumbelow and Watkins, 2023).

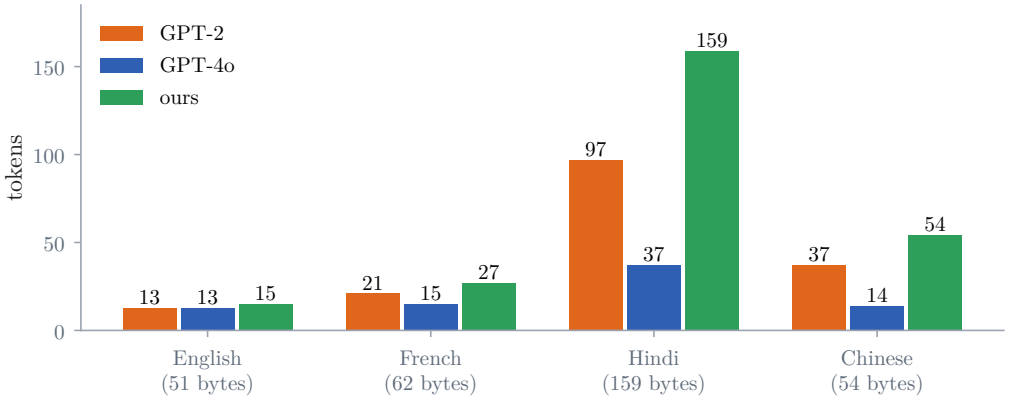


Figure 17. The same request costs more tokens in languages the tokenizer saw less of. The running request in English and in our French, Hindi, and Chinese translations, with its length in bytes of UTF-8, and its number of tokens in GPT-2's tokenizer (50,257 tokens, learned from English web text), GPT-4o's (about 200,000), and ours (4,096, learned from English stories only).

the number of tokens. Ours has seen only English stories and writes Hindi byte by byte: 159 tokens for 159 bytes.

Other ways of building a vocabulary, WordPiece and the unigram model of SentencePiece, are compared in Appendix A. They differ in how they choose the pieces, not in what the model does with them.

The tokenizer is fixed; next is the text it will tokenize.

5. Data

What text does a base model learn from, and how is it prepared? Lecture 4's word model read the 17,592 training stories of TinyStories' validation file, about 3.5 million words, four and a half times over. Our corpus is a slice of TinyStories' training file, 549,221 stories and 131 million tokens after the steps below, of which our base model reads 49 million; Llama 3 read 15.6 trillion tokens (Llama Team, 2024), about 300,000 times as many. The steps that make such a corpus are the same at both scales (Figure 18).

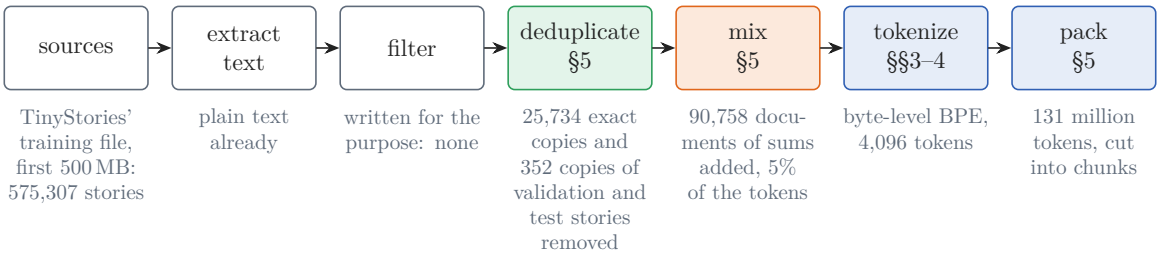


Figure 18. A pretraining corpus is made in steps, each removing or reweighting text. From raw sources to the stream of token ids the model reads, with the section of these notes for each step. Below each step: what it did to our corpus, the first 500 MB of TinyStories' training file.

Sources and filters

Most pretraining text comes from the web, through **Common Crawl**, a public archive of several billion web pages collected each month since 2008, with books, encyclopedias, code, and scientific papers added in smaller amounts. A web page is mostly not text worth predicting: menus, notices, advertisements, lists of links, spam, and whole pages in other languages. **Extraction** keeps the main text of a page and drops the rest (Figure 19); FineWeb, for example, ran the extractor trafilatura over the raw HTML of Common Crawl (Penedo et al., 2024). What remains is then filtered:

1. **by rules**, which drop documents that are too short, have too few words of a dictionary, too many symbols or repeated lines, or a known bad source; C4 (Raffel et al., 2020), for example, kept only lines that end in punctuation and dropped pages with a word from a list of offensive words;
2. **by classifiers**, small models trained to recognize text like a reference set; FineWeb-Edu (Penedo et al., 2024) kept the 1.3 trillion tokens, of 15 trillion, that a classifier trained on a large model's ratings judged most educational, and models trained on it learned faster on knowledge and reasoning benchmarks.

TinyStories needs neither: it was written by GPT-3.5 and GPT-4 for the purpose, in a vocabulary a three-year-old knows (Eldan and Li, 2023). Appendix C describes the pipelines of the large open corpora.

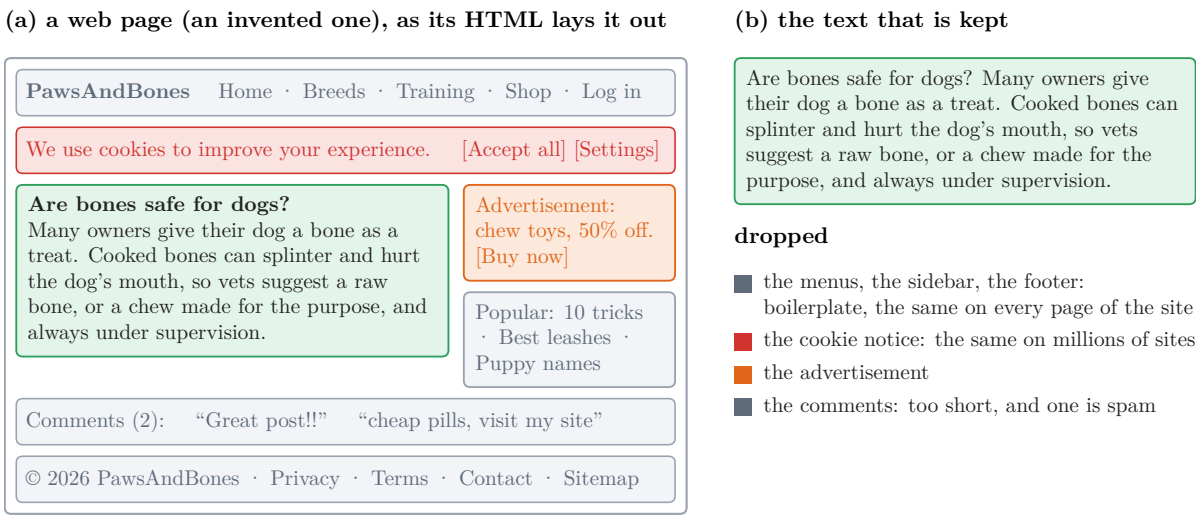


Figure 19. Most of a web page is not text worth predicting. (a) An invented page about dogs, as its HTML lays it out: a menu, a cookie notice, the article (green), an advertisement, links to other posts, comments, and a footer. (b) What extraction keeps, the article's text, and what it drops, with the reason.

Duplicates

The web repeats itself: the same page under several addresses, the same license or template on millions of pages. Duplicates waste computation, and a model trained on a text many times tends to memorize it: Lee et al. (2022) found that deduplicating C4 reduced the rate at which a model emitted memorized training text tenfold, at no cost in accuracy.

Exact duplicates are found by hashing each document after normalizing its whitespace. In our slice 25,734 of 575,307 stories, 4.5%, were exact copies of an earlier story.

Near duplicates, which differ by a few words, need a measure of overlap. Write each document as the set of its **shingles**, its sequences of five consecutive words. The **Jaccard similarity** of two documents A and B is the share of shingles they have in common, $J(A,B) = |A \cap B|/|A \cup B|$, 1 for identical documents and 0 for documents with no five words in common. Comparing every pair of 575,307 documents would take 1.7×10^{11} comparisons. **MinHash** (Broder, 1997) instead gives each document a short signature whose agreement estimates the similarity:

1. Choose a random hash function h, which puts all possible shingles in a random order.
2. For each document, keep the shingle that comes first in that order: $m(A) = \min_{s \in A} h(s)$.
3. Two documents keep the same shingle exactly when the first shingle of $A \cup B$ in the order lies in $A \cap B$, and every shingle of the union is equally likely to come first, so

$$\Pr[m(A) = m(B)] = \frac{|A \cap B|}{|A \cup B|} = J(A,B).$$

4. With 128 independent hash functions, a document's signature is 128 numbers, and the fraction of positions where two signatures agree estimates J.

Signatures still have to be compared in pairs. **Banding**, a form of locality-sensitive hashing, finds the similar pairs in one pass instead, by hashing pieces of the signatures, and each pair it proposes is then checked by its exact Jaccard similarity (Appendix C).

In our slice, once the exact copies were removed, banding found no candidate pair at all: TinyStories' duplicates are exact copies. Web text has both kinds in quantity (Lee et al., 2022).

Mixtures and contamination

A corpus is a **mixture** of sources, and the share of each is a choice: Llama 3's final mixture was about half general knowledge, a quarter mathematics and reasoning, 17% code, and 8% multilingual text (Llama Team, 2024), set by training small models on candidate mixtures. Ours has two sources: the stories, and short documents of eight sums each, *47 + 385 = 432*, with operands of one to three digits, which make up 5.3% of the tokens. They give the base model digits and the form of a sum, which the chat model of §13 and the reinforcement learning of Week 7 will use.

Contamination is test text inside the training text. A model that has read its test set is evaluated on memory, not on prediction. Our validation and test stories come from TinyStories' validation file (Lecture 2's split), and our training stories from its training file, which should be disjoint. They are not: 352 of our 4,398 validation and test stories, 8.0%, appear word for word in the training slice. We removed those 352 training stories. On benchmarks, contamination is checked the same way, by overlaps of long n-grams between benchmark and corpus, and is reported with the results, as in GPT-3's appendix on contamination (Brown et al., 2020).

Packing

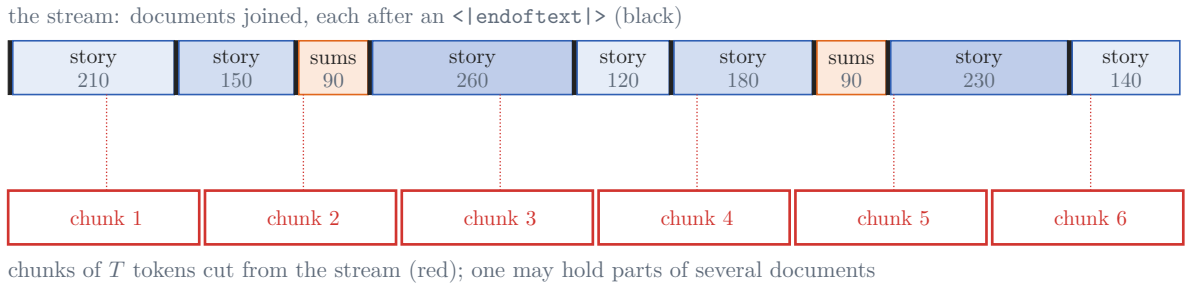


Figure 20. Documents are joined into one stream and cut into chunks of equal length. Stories of different lengths (blue shades) and documents of sums (orange), with their lengths in tokens, each after an <|endoftext|> (black), joined into one stream and cut into chunks of T tokens. A chunk may hold parts of several documents.

Documents have different lengths, and a training batch is a block of B chunks of T tokens. So the tokenized documents are joined into one stream with `<|endoftext|>` between them, so that every document follows one, and chunks of $T + 1$ consecutive tokens are cut from the stream at random places (Figure 20): the first T are the inputs and the last T the targets, as in Lecture 4 (Section 8). Nothing is wasted on padding. A chunk often holds the end of one story and the start of the next, and attention lets the second read the first. The model learns from the end-of-text token that the stories are unrelated; some large models also mask attention at the boundaries, so that each document reads only itself (Llama Team, 2024).

Our training stream holds 131 million tokens: the 549,221 stories that remain, and 90,758 documents of sums. The next question is how to fit a model to it.

6. Optimization, in brief

We now have every piece of a pretraining run but one. The architecture is fixed (Lectures 2–4), and so is the input: tokens (§§2–4), and a stream of them cut into chunks (§5). What is left is the optimization, and for it the model and the data can be abstracted away. A training step

draws a batch $x^{(i)}$ of B chunks of T tokens, a $B \times T \times d$ tensor once embedded. The forward pass computes its loss, the average log loss of its next tokens,

$$\ell^{(i)} = \mathcal{L}(\theta, x^{(i)}),$$

and the backward pass its gradient $\mathbf{g}^{(i)} = \partial \ell^{(i)} / \partial \theta$, by backpropagation, as in Lecture 1. (We write $\mathcal{L}$ for the loss, since L counts blocks.) An optimizer is a rule that turns these gradients, one batch after another, into steps for θ, and that is all it ever sees. The rules, why each one replaced the last, and the theory behind them are the subject of the companion tutorial, *Optimizers for Deep Learning*, and the interactive optimizer explorer lets you run them on losses of two parameters. Here is what the rest of this lecture uses.

Gradient descent steps along the negative gradient, $\theta \leftarrow \theta - \eta\, \mathbf{g}$. Along each direction the loss bends upward at some rate, its **curvature** λ. A step larger than $2/\lambda_{\max}$ overshoots along the steepest direction, so the step must be small, and along the flattest direction, of curvature $\lambda_{\min}$, progress then crawls. The ratio $\kappa = \lambda_{\max}/\lambda_{\min}$, the **condition number**, sets how many steps it needs. And each step uses a **stochastic gradient**, from one batch: right on average, with a variance that falls as $1/B$. Two independent batches of our base model's size give gradients whose cosine similarity is only 0.30 at step 1,000, so most of each step's direction is noise, which only the averaging over many steps removes. The batch beyond which a larger one stops helping, the **noise scale**, is about 18,000 tokens for our model (§7 uses it).

The fixes form a sequence, each for a failure of the rule before it:

1. **Momentum** (Polyak, 1964), $v \leftarrow \beta v + g$, $\theta \leftarrow \theta - \eta v$. The problem: gradient descent zig-zags across steep directions and crawls along flat ones. The fix: a running average of recent gradients, in which the zig-zags cancel and the steady pull along the flat directions adds up.
2. **RMSProp** (Tieleman and Hinton, 2012), $s \leftarrow \rho s + (1-\rho) g^2$, $\theta \leftarrow \theta - \eta\, g/\sqrt{s}$, for each parameter. The problem: one step size for parameters whose gradients differ by orders of magnitude. The fix: divide each parameter's gradient by its own typical size.
3. **Adam** (Kingma and Ba, 2015), $m \leftarrow \beta_1 m + (1-\beta_1) g$, $v \leftarrow \beta_2 v + (1-\beta_2) g^2$, $\theta \leftarrow \theta - \eta\, \hat{m}/(\sqrt{\hat{v}} + \epsilon)$, where $\hat{m} = m/(1-\beta_1^t)$ and $\hat{v} = v/(1-\beta_2^t)$ at step t. Both fixes at once, momentum's average divided by RMSProp's scale; $\hat{m}$ and $\hat{v}$ remove the bias of averages that start at zero.
4. **AdamW** (Loshchilov and Hutter, 2019), Adam's step plus $\eta\lambda\theta$. The problem: weight decay added to the gradient is rescaled by Adam, so it decays some weights far more than others (§7). The fix: apply the decay apart from the gradient step.
5. **Muon** (Jordan et al., 2024), for each weight matrix: its momentum, replaced by the nearest orthogonal matrix, computed by five Newton–Schulz steps. The problem: a matrix's gradient moves it far along a few directions and hardly at all along the rest. The fix: a step of the same size along every direction.

Adam's step has a size of about η for every parameter, whatever the scale of its gradient. Its two averages run over about $1/(1-\beta)$ steps: with $\beta_1 = 0.9$ and $\beta_2 = 0.95$, the values of language models, the last 10 and 20. The small ϵ, 10^{-8} by default, keeps a parameter whose gradient is zero from being divided by zero. On a two-parameter bowl momentum can beat Adam, but on Transformers Adam wins by far, because their gradients are nothing like a bowl's. The gradients of different parameters of our base model differ in size by more than four orders of magnitude, from 7.7×10^{-8} for the smallest 1%, mostly rows of rare tokens, to 1.5×10^{-3} for the largest 1% (Figure 21). A step size small enough for the largest barely moves the rest, and only a step scaled to each parameter moves them all.

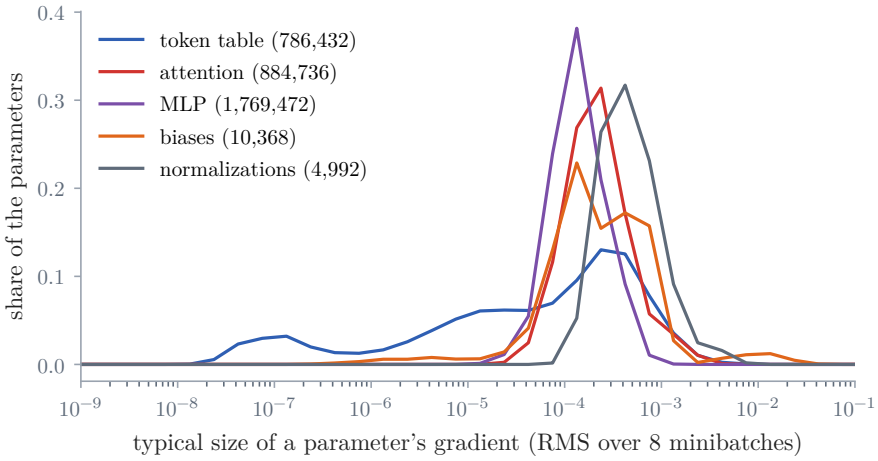


Figure 21. The gradients of a Transformer's parameters differ in size by orders of magnitude. For each parameter of our base model at step 1,000 (§8), the root mean square of its gradient over eight minibatches; the distribution of these sizes within each kind of parameter (with the number of parameters of each kind): the token table, the weight matrices of attention and of the MLPs, the biases, and the normalizations' scales and shifts. Logarithmic scale.

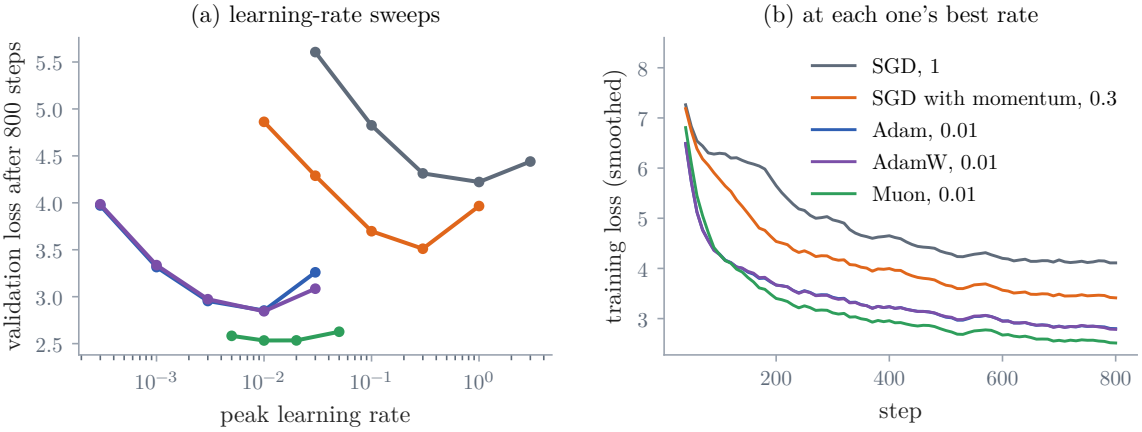


Figure 22. Adam trains our model far faster than gradient descent, and Muon faster still. Lecture 4's word-model shape (4 blocks, $d = 128$) with our 4,096 tokens, trained for 800 steps of 16×256 tokens with the same schedule (§7), by stochastic gradient descent, with momentum ($\beta = 0.9$), Adam ($\beta = (0.9, 0.95)$), AdamW (weight decay 0.1 on every parameter, not only on the matrices as §7 recommends), and Muon (on the blocks' matrices, with AdamW at 0.01 on the other parameters; the rate shown is Muon's). (a) Validation loss after 800 steps against the peak learning rate, logarithmic scale, one color per optimizer as named in (b); no run diverged. (b) Training loss along the way (smoothed), at each optimizer's best rate; Adam's and AdamW's curves nearly coincide. Single runs.

Figure 22 compares five optimizers on our model, each at several learning rates, with everything else equal. At its best rate, gradient descent reaches 4.22 nats per token; with momentum, 3.51; Adam, 2.85; and AdamW, 2.84 (Figure 22a). Momentum gains 0.7 nats over gradient descent, and Adam another 0.7; the training curves separate within the first hundred steps (Figure 22b). The best rates differ a hundredfold, 1 for gradient descent and 0.01 for Adam, because Adam's step has a size of about η whatever the gradient, while gradient descent's is η times a gradient whose entries are far smaller than 1 (Figure 21). Over 800 steps, AdamW's weight decay (§7) changes little. Muon, at its best rates of 0.01 and 0.02, reaches 2.53, another 0.3 nats below Adam, and its curve separates from Adam's after about a hundred steps; it is also the least sensitive to its rate, within 0.1 nats from 0.005 to 0.05. These are single runs on a small model; they agree in direction with the larger comparison of Liu et al. (2025a). Most published runs use AdamW (§8); Muon trained Moonlight (Liu et al., 2025a) and Kimi K2 (Kimi Team, 2025), and nanochat uses it for the matrices inside its blocks (Appendix B.3).

An optimizer with a fixed learning rate is still not the whole recipe. At large rates its first steps, taken before its averages mean anything, can throw the model off, and at the end of a run it keeps taking steps of size η around the minimum instead of settling into it. Both are answered by changing η during training. How many tokens each step reads, how the weights start, and in what arithmetic they are stored are the other choices a training configuration makes; they are the training recipe.

7. The training recipe

What else does a pretraining run need, beyond the optimizer's step? The Transformer of 2017 was trained with a recipe of four parts, used as given in Lecture 4 (Section 7): Adam, a learning rate that warms up and then decays, dropout, and label smoothing. Pretraining today keeps the first two and drops the last two. It adds a batch of hundreds of thousands to millions of tokens per step, weight decay, an initialization scaled for depth, clipping, and lower-precision arithmetic. Each part answers a failure, and each is a line of the configuration file that §8 reads.

The batch: tokens per step

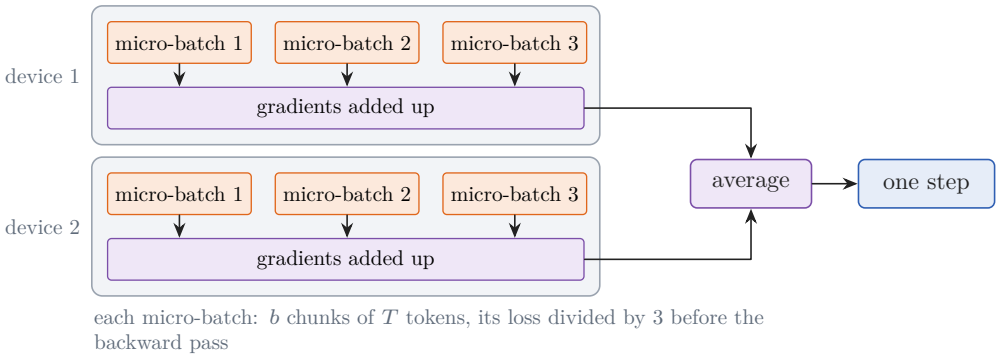


Figure 23. A step's batch is assembled from micro-batches on several devices. Two devices, each holding a copy of the model, each computing the gradients of three micro-batches of b chunks of T tokens one after another and adding them up (gradient accumulation); the two devices' sums are averaged (data parallelism), and the optimizer takes one step. The batch has $b \times T \times 3 \times 2$ tokens.

A step's batch is counted in tokens: B chunks of T tokens, $16 \times 512 = 8{,}192$ for our base model. A large model cannot hold the activations of a large batch at once, since the backward pass needs every layer's activations for every token, so a batch is computed in pieces (Figure 23):

1. Split the batch into k **micro-batches** of b chunks each, as many as fit in memory.
2. For each micro-batch in turn, compute the loss, divide it by k, and backpropagate; the gradients of the k micro-batches add up in place. This is **gradient accumulation**.
3. After the k-th, clip the summed gradient, take one optimizer step, and set the gradients to zero.

The result is the gradient of the whole batch, up to rounding, because nothing in a Transformer mixes the chunks of a batch: each chunk's loss depends only on that chunk (the layer normalization of Lecture 4 (Section 6) normalizes each position on its own). Dividing by k assumes that every micro-batch has the same number of tokens in the loss, as packed chunks do; when they differ, as with the padded and masked batches of §13, the token losses are summed and divided by the batch's total count instead. **Data parallelism** does the same across n devices at once: each holds a copy of the model, computes the gradients of its own micro-batches, and the devices average their gradients before every step, so that all copies take the same step. A batch then has

$$\text{tokens per step} = B \times T = b \times T \times k \times n$$

tokens, $B = bkn$ chunks: in nanoGPT's configuration for GPT-2 (§8), $12 \times 1{,}024 \times 5 \times 8 = 491{,}520$. The other ways of dividing a model among devices, by matrices and by blocks, are in Appendix D.

How large should the batch be? The noise scale of §6 answers it. Below B_{noise}, most of each step's direction is noise, and doubling the batch nearly halves the number of steps needed for the same loss; well above it, doubling the batch doubles the cost of a step and saves few steps (McCandlish et al., 2018; the tutorial on optimizers derives it). Our batch of 8,192 tokens is below our model's B_{noise} of about 18,000 tokens, so a larger batch would have used each step better, at more computation per step. The noise scale grows as the loss falls, and large models reach lower losses, which is one reason they train with 0.5 to 16 million tokens per step, and why some raise the batch during training: GPT-3 over its first 4 to 12 billion tokens (Brown et al., 2020), and Llama 3 from 4 to 8 to 16 million tokens (Llama Team, 2024). A larger batch gives a less noisy gradient, which tolerates a larger learning rate. For gradient descent, Goyal et al. (2017) raised the rate in proportion to the batch; for Adam the best rate grows more slowly (Malladi et al., 2022), and in practice the two are tuned together.

Learning-rate schedules

A schedule has four numbers: the peak rate, the number of warmup steps, the step at which the decay ends, and the rate it ends at. nanoGPT's configuration for GPT-2, which §8 reads in full, states all four:

```
learning_rate = 6e-4     # the peak
warmup_iters = 2000      # a linear rise from 0 over 2,000 steps
lr_decay_iters = 600000  # the step at which the decay ends
min_lr = 6e-5            # the rate it ends at, a tenth of the peak
```

Between the end of the warmup, step t_{w}, and the end of the decay, step t_{d}, the rate follows half a cosine from the peak $\eta_{\max}$ down to the final rate $\eta_{\min}$ (Figure 24):

$$\eta_t = \eta_{\min} + \tfrac{1}{2}\,(\eta_{\max} - \eta_{\min})\Big(1 + \cos\frac{\pi\,(t - t_{\text{w}})}{t_{\text{d}} - t_{\text{w}}}\Big), \qquad t_{\text{w}} \le t \le t_{\text{d}},$$

which is $\eta_{\max}$ at t_{w}, falls slowly at first, fastest halfway, and slowly again into $\eta_{\min}$ at t_{d}; after t_{d} nanoGPT keeps $\eta_{\min}$. When a table of runs says **"cosine to 10%"**, it means this schedule

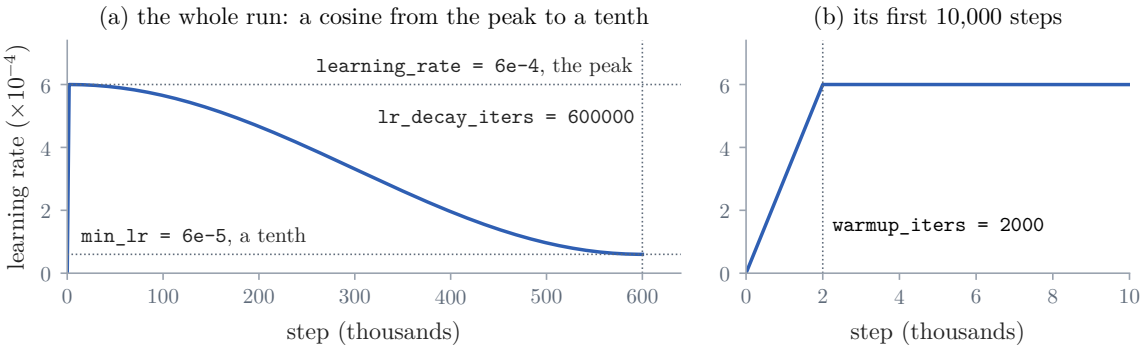


Figure 24. nanoGPT's schedule for GPT-2, drawn from its four numbers. (a) The learning rate over the whole run of 600,000 steps: a cosine from the peak of 6×10^{-4} to a tenth of it at step 600,000. (b) The first 10,000 steps: the linear warmup over 2,000 steps.

with $\eta_{\min} = \eta_{\max}/10$, the decay ending at or near the last step: GPT-3 ended it at 260 of its 300 billion tokens and kept the final rate to the end.

Why each part? Short runs of our model answer it for the warmup and the decay, each with Adam at a peak of 0.01 for 800 steps, the setting of §6, and GPT-3's models for the peak.

Warmup. In its first steps Adam divides by averages built from one or a few gradients, and the gradients of a randomly initialized model are large and change quickly; a full-size step then can throw the model into a region from which it does not recover. A **warmup** raises the learning rate gradually, in language models linearly from zero, over the first few hundred or thousand steps (Goyal et al., 2017). Without one, Lecture 4's post-norm character model learned nothing beyond the frequencies of the characters at its larger learning rate (Lecture 4 (Section 6)). Our pre-norm model still trains without one, but worse (Figure 25): at Adam's best peak rate, 0.01, it ends the 800 steps at 3.95 nats per token, against 2.85 with a warmup of 100 steps, and at a peak of 0.03 at 4.51, against 3.26.

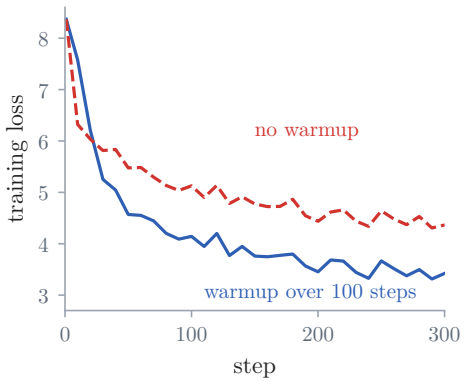


Figure 25. Without a warmup, the first steps throw the model off. The training loss of our model (§6) over its first 300 steps, with a warmup over 100 steps and without one (Adam, peak 0.01, cosine decay).

Decay. With a constant rate, Adam keeps taking steps of size about η, and the loss stops falling at a level set by that size and the gradient's noise. Lowering η at the end lets it settle lower (Figure 26): at Adam's best peak rate, 0.01, a constant rate after the warmup ends the 800 steps at 2.95 nats per token, the cosine decay at 2.85, and warmup–stable–decay, which decays only over the last 160 steps, at 2.81. The cosine schedule (Loshchilov and Hutter, 2017) needs the number of steps in advance, so a run cannot be extended or stopped early at its best. **Warmup–stable–decay** keeps the rate constant and decays it only at the end, over about the last tenth of the steps in Hu et al. (2024) and the last fifth in our runs. Checkpoints of the stable phase can be decayed whenever a model is wanted, and the decay alone brings most of the gain. The decay is also the moment to change what the model reads, which is what midtraining does (§12).

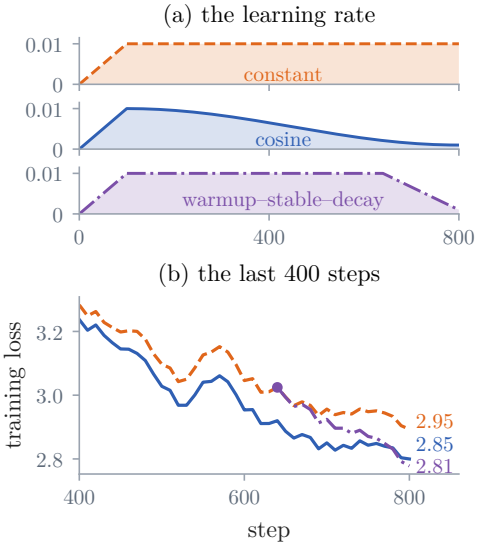


Figure 26. A decay at the end lowers the loss. (a) Three schedules with the same warmup and peak: constant; cosine to a tenth; warmup–stable–decay, constant until the last fifth, then linear to a tenth. (b) Our model's training loss (smoothed) under each over the last 400 of its 800 steps, and its validation loss at the end (right); warmup–stable–decay is the constant run until its decay begins at step 640 (dot).

The peak. The peak rate is the number of a configuration most worth tuning, and its best value falls as models grow: GPT-3 used 6×10^{-4} for its smallest model, of 125 million parameters, and 0.6×10^{-4} for the largest, of 175 billion (Brown et al., 2020; Figure 27). One reason is width: a wider matrix sums more terms, so the same change to each weight changes its output more. The maximal update parametrization scales the rate of each kind of matrix with the width so that one rate, tuned on a narrow model, works for a wide one (Appendix B.2).

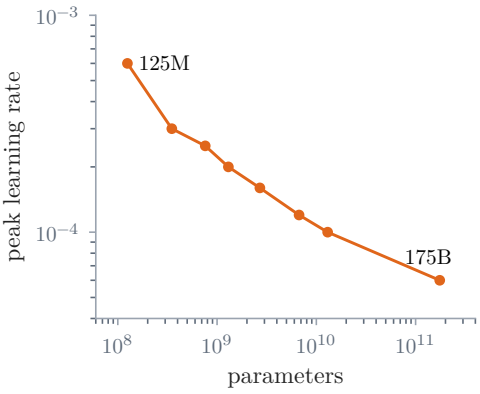


Figure 27. Larger models use lower peak rates. The peak learning rates of GPT-3's eight models, from 125 million to 175 billion parameters (Brown et al., 2020, Table 2.1). Logarithmic scales.

Weight decay

Weight decay was introduced against overfitting, but a pretrained model reads most of its text once. In language models it is kept for its effect on the optimization. **L2 regularization** adds $\frac{\lambda}{2}\|\theta\|^2$ to the loss, which adds $\lambda\theta$ to the gradient and pulls every weight toward zero in proportion to its size; in gradient descent this is the same as shrinking every weight by the factor $1 - \eta\lambda$ at each step, **weight decay**. In Adam the two differ (Loshchilov and Hutter, 2019). Added to

the gradient, $\lambda\theta$ is divided by $\sqrt{\hat{v}}$ like the rest of it, so a weight with large gradients is barely decayed and a weight with tiny gradients is decayed hard. **AdamW** applies the decay apart from the gradient step:

$$\theta \leftarrow \theta - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\theta \right),$$

so every weight shrinks by the same factor $1 - \eta\lambda$ per step, whatever its gradients. Since η follows the schedule, so does the decay, which fades as the rate does.

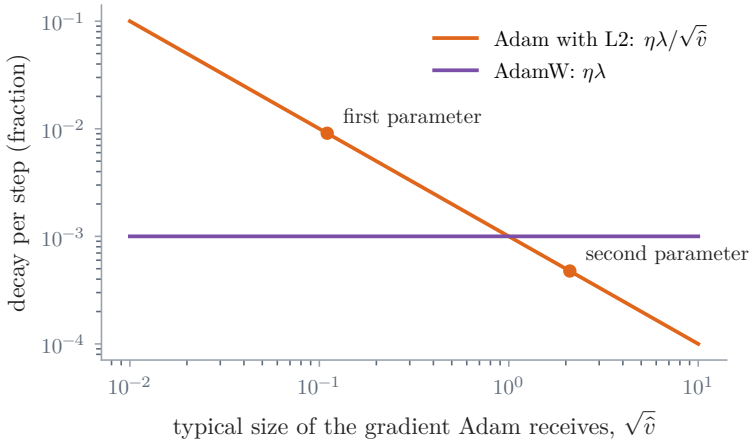


Figure 28. Inside Adam, L2 regularization decays parameters with small gradients hardest; AdamW decays all alike. The decay per step applied to a parameter, as a fraction of its value, against the typical size of the gradient Adam receives, $\sqrt{\hat{v}}$, for Adam with L2 ($\eta\lambda/\sqrt{\hat{v}}$) and for AdamW ($\eta\lambda$), with $\eta = 0.01$, $\lambda = 0.1$; the two parameters of the example marked. Logarithmic scales.

Take two weights equal to 1, with gradients of typical size 0.01 and 2, $\eta = 0.01$, and $\lambda = 0.1$. AdamW shrinks each by $\eta\lambda\theta = 0.001$ per step. Adam with L2 adds $\lambda\theta = 0.1$ to each gradient and then divides by the typical size of what it receives, about 0.11 and 2.1, so it shrinks the first by 0.0091 and the second by 0.00048, nineteen times less (Figure 28).

Configurations decay the weight matrices, usually with $\lambda = 0.1$ (Lecture 4 (Appendix E.2)), and not the biases or the normalizations' scales and shifts. The rule in code is usually "decay every tensor of two or more dimensions": the scales and shifts set the size and offset of each feature, and decaying a scale toward zero would shrink its feature for every input. Configurations differ on the token table, which is a matrix too: nanoGPT and our models decay it, and OLMo 2 stopped doing so for stability (Team OLMo, 2024). What decay does here is still debated; D'Angelo et al. (2024) found that for language models trained for about one pass over their text it acts on the optimization, giving a lower training loss and more stable training, rather than as a guard against overfitting. In our short runs it made no visible difference (§6: AdamW 2.84 against Adam 2.85).

Initialization

The weights a run starts from must keep signals and gradients at a usable size through every layer, the aim of the initializations of Glorot and Bengio (2010) and He et al. (2015), which scale each matrix's random entries by about one over the square root of its input width. GPT-2's rules, which nanoGPT and our models follow, add one thing for a residual network: every matrix is drawn with a standard deviation of 0.02, and the two matrices of each block that write into the residual stream with $0.02/\sqrt{2L}$, so that the stream does not grow with the depth L. A configuration lists the first as a field (`init_std: 0.02`); the second is in the model's code. Appendix B.1 gives the argument and measures it on our model.

Clipping and loss spikes

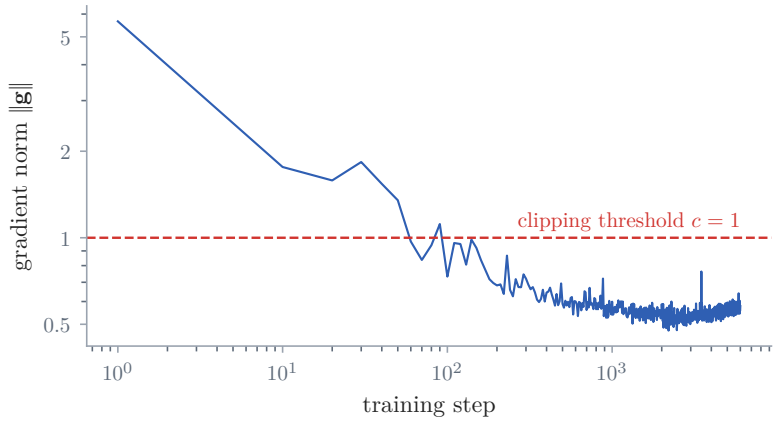


Figure 29. The gradient is largest at the start of training. The norm of the whole gradient of our base model (§8), before clipping, every 10 steps; the dashed line is the clipping threshold $c = 1$. Logarithmic axes.

A rare batch can produce a gradient many times larger than usual, and one step along it can undo many. **Gradient clipping** rescales the whole gradient when its norm exceeds a threshold c, $\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1,\ c/\|\mathbf{g}\|)$, keeping its direction, as for the recurrent networks of Lecture 2 (Section 7); $c = 1$ is usual. In our base model's training it acted only at the start: of the norms recorded every 10 steps, the last above 1 was at step 90, and afterwards the norm stayed between 0.48 and 0.99 (Figure 29). Large runs still see **loss spikes**, sudden jumps of the loss from which training may or may not recover. PaLM's largest run spiked about twenty times and was restarted each time from an earlier checkpoint with some batches skipped (Chowdhery et al., 2023; Lecture 4 (Appendix E.1)). PaLM also added a **z-loss**, a small penalty $10^{-4} \log^2 Z$ on the logarithm of the softmax's normalizer Z, which keeps the output scores from drifting to large values; some configurations keep it.

Precision and memory

Every number so far has been a 32-bit floating-point number (fp32): a sign, 8 bits of exponent, which set its range, and 23 of mantissa, which set its precision. Hardware computes far faster with 16-bit numbers, and two formats exist (Figure 30): fp16, with 5 bits of exponent, whose largest number is 65,504 and whose small gradients round to zero; and bfloat16 (bf16), with fp32's 8 bits of exponent and only 7 of mantissa, so its range is fp32's and its precision between two and three decimal digits.

Mixed-precision training (Micikevicius et al., 2018) has three parts:

1. The matrix products of the forward and backward passes are computed in 16 bits, with their sums accumulated in fp32. Reductions over many numbers, such as the softmax's sum, stay in fp32, because they lose too much in 16 bits; frameworks also keep the normalizations and the loss in fp32.
2. A **master copy** of the weights is kept in fp32, and the updates are added to it: an update of 10^{-4} of a weight's size is lost entirely in bf16, where the gap between consecutive numbers near 1 is $2^{-7} \approx 0.008$.
3. In fp16 only, **loss scaling**: the loss is multiplied by a large factor before the backward pass, so that small gradients do not round to zero, and the gradients are divided by it before the

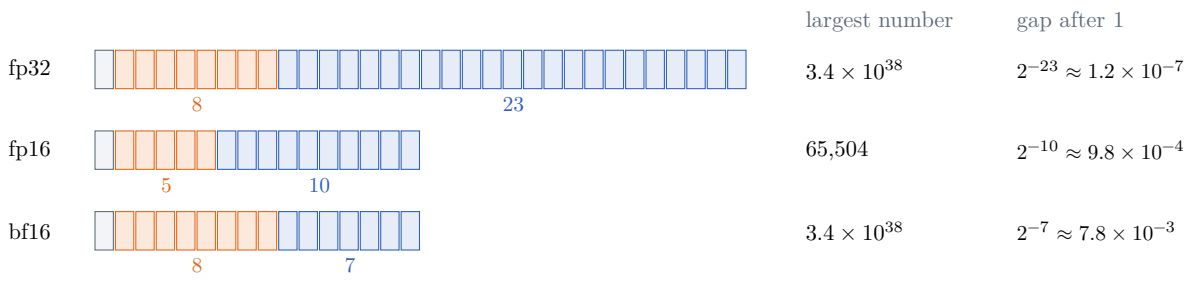


Figure 30. Two 16-bit formats trade range for precision differently. The bits of fp32, fp16, and bf16: sign (gray), exponent (orange), mantissa (blue); for each, its largest number and the gap between 1 and the next number.

step; the factor is lowered whenever a gradient overflows. bf16 has fp32's range and needs none. A configuration's `dtype` chooses between them.

Which format holds what? In current runs the 16-bit format is bf16, supported since the A100 of 2020 and the usual choice since; fp16 appears in older runs, with the same layout and loss scaling added. Two layouts are common, and both come to **16 bytes per parameter** for the weights, gradients, and optimizer state:

what is stored, per parameter	**16-bit weights (ZeRO, Megatron)**	**fp32 weights (PyTorch's autocast, nanoGPT)**
weights used in the forward and backward passes	bf16, 2 bytes	none stored: cast from fp32 to bf16 for each matrix product
gradients	bf16, 2 bytes	fp32, 4 bytes
master weights	fp32, 4 bytes	the fp32 weights themselves, 4 bytes
AdamW's m and v	fp32, 4 + 4 bytes	fp32, 4 + 4 bytes
in all	16 bytes	16 bytes

The first is the accounting of Rajbhandari et al. (2020); the second is what nanoGPT's `dtype = 'bfloat16'` does, where only the matrix products and the activations are in bf16. A model of 7 billion parameters needs 112 GB for these alone, more than one GPU holds, so its state is split across devices (Appendix D). The activations kept for the backward pass come on top and grow with the tokens of a micro-batch; **activation checkpointing** keeps only some of them and recomputes the rest during the backward pass, at about a third more computation. Our base model's 3.5 million parameters take 55 MB, and it was trained in fp32 throughout, on a CPU.

Dropout and label smoothing

The recipe of 2017 had two parts against **overfitting**, fitting the training text more closely than text in general. **Dropout** (Srivastava et al., 2014) sets each activation to zero with probability p during training, and multiplies the others by $1/(1-p)$ so that their average is unchanged, which stops units from relying on one another; at test time nothing is dropped (the original scaled the weights at test time instead, which is equivalent). **Label smoothing** (Szegedy et al., 2016) trains toward a target that puts $1-\varepsilon$ on the correct token and spreads ε over all tokens, which keeps the model from becoming certain. Vaswani et al. (2017) used $p = \varepsilon = 0.1$. Both answer a model that reads its training data many times; a pretrained model reads most of its text about once, and cannot overfit what it has not seen before. Our base model reads 49

million of the stream's 131 million tokens, and its training and validation losses stay together to the end (Figure 32). So pretraining today uses neither (Chowdhery et al., 2023; the released configurations of Llama 3 and OLMo 2). Fine-tuning on a few examples read many times can overfit (§13), and some fine-tuning recipes use dropout again.

Training fewer parameters: LoRA

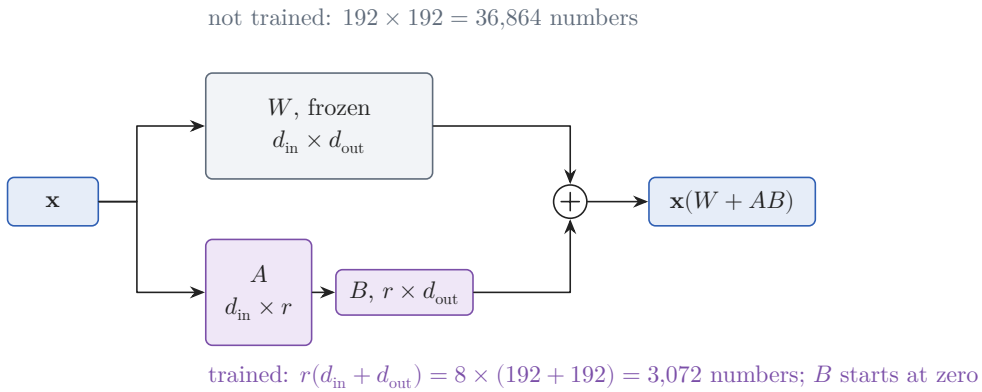


Figure 31. LoRA trains a low-rank change beside a frozen matrix. A layer $\mathbf{x}W$ with W frozen (gray) and a trained path $\mathbf{x}AB$ through r numbers (purple), added to it; the counts for one of our model's attention matrices with $r = 8$.

Fine-tuning every weight of a model on a new task, as §13 will, needs the 16 bytes per parameter of the optimizer again, for the whole model, however small the change. **LoRA**, low-rank adaptation (Hu et al., 2022), trains a small change instead. Each chosen weight matrix W, of $d_{\text{in}} \times d_{\text{out}}$, is frozen, and the layer computes

$$\mathbf{x}\,(W + AB), \qquad A \in \mathbb{R}^{d_{\text{in}} \times r},\ B \in \mathbb{R}^{r \times d_{\text{out}}},$$

with a small rank r (Figure 31). Only A and B are trained, $r(d_{\text{in}} + d_{\text{out}})$ numbers instead of $d_{\text{in}}d_{\text{out}}$, and the optimizer's state shrinks with them. A starts random and B at zero, so that training starts exactly from the base model. (Hu et al. use column vectors, $W_0\mathbf{x} + BA\mathbf{x}$; our A and B are the transposes of theirs, in the same roles.) After training, AB is added into W, and the model runs at its original cost. For one of our model's 192×192 attention matrices and $r = 8$, LoRA trains 3,072 numbers instead of 36,864. That so few can be enough is an observation, not a theorem: fine-tuning large models was found to change their weights in a few directions only (Aghajanyan et al., 2021). What LoRA leaves frozen matters too: the token table, whose rows for new special tokens may need to change (§13).

In memory, the frozen weights are stored once, usually in bf16: 2 bytes per parameter, with no gradient and no optimizer state. QLoRA stores them in 4 bits instead, and fine-tuned a model of 65 billion parameters on one 48 GB GPU (Dettmers et al., 2023). A and B are usually kept in fp32, with fp32 gradients and AdamW's m and v, 16 bytes per trained number, but they are 1% of the model or less. A model of 7 billion parameters then needs about 14 GB for its weights and a gigabyte or less for what training adds, against the 112 GB of full fine-tuning, with the activations on top in both. Our own fine-tuning runs, on a CPU, are in fp32.

Every part of the recipe is now defined, and each has a line in a training configuration. §8 reads one.

8. Reading a pretraining configuration

What does a real training configuration say, and could we write one with what §§6–7 defined? A pretraining run is specified by a configuration: a file or a list of command-line options that fixes the model's shape, the data per step, the optimizer and its schedule, and the arithmetic. Every field has been defined by now. Here is a public one, read line by line.

nanoGPT's configuration for GPT-2

nanoGPT (Karpathy, 2022) is a short, widely read implementation of GPT-2. Its configuration for reproducing GPT-2's smallest model, 124 million parameters, on OpenWebText (an open re-creation of GPT-2's WebText) is a Python file of assignments, `config/train_gpt2.py`, read on top of the defaults in `train.py`. These are the training fields in effect, from the file and from `train.py`'s defaults, with their comments (two shortened) and with `dtype` as it resolves on a GPU that supports bf16:

```
batch_size = 12  # micro-batch size
block_size = 1024
gradient_accumulation_steps = 5 * 8
max_iters = 600000
learning_rate = 6e-4  # max learning rate
warmup_iters = 2000
lr_decay_iters = 600000  # should be ~= max_iters per Chinchilla
min_lr = 6e-5  # should be ~= learning_rate/10 per Chinchilla
weight_decay = 1e-1
beta1 = 0.9
beta2 = 0.95
grad_clip = 1.0
dropout = 0.0  # for pretraining 0 is good, for finetuning try 0.1+
bias = False
dtype = 'bfloat16'
n_layer = 12
n_head = 12
n_embd = 768
```

Each line, beside the same choice for our base model, and the section that explains it:

field	nanoGPT, GPT-2 (124M)	our base model	what it sets
`batch_size`	12 chunks	16 chunks	the micro-batch (§7)
`block_size`	1,024 tokens	512 tokens	the length of a chunk, the context T
`gradient_accumulation_steps`	5 on each of 8 GPUs	1, one CPU	k and n (§7)
tokens per step	491,520	8,192	$b \times T \times k \times n$ (§7)
`max_iters`	600,000 steps	6,000 steps	the length of the run
tokens in all	295 billion	49 million	steps × tokens per step
`learning_rate`	6×10^{-4}	2×10^{-3}	the peak rate (§7)
`warmup_iters`	2,000 (0.3% of the steps)	200 (3%)	the linear warmup (§7)
`lr_decay_iters`, `min_lr`	cosine to 6×10^{-5} at step 600,000	cosine to 2×10^{-4} at step 6,000	the decay, to a tenth (§7)

field	nanoGPT, GPT-2 (124M)	our base model	what it sets
`weight_decay`	0.1	0.1	AdamW's λ, on tensors of two or more dimensions (§7)
`beta1`, `beta2`	0.9, 0.95	0.9, 0.95	Adam's averages, over about 10 and 20 steps (§6)
ϵ (not listed)	10^{-8}	10^{-8}	PyTorch's default (§6)
`grad_clip`	1.0	1.0	the clipping threshold (§7)
`dropout`	0	0	none in pretraining (§7)
`bias`	no biases	biases	in the linear layers and normalizations
`dtype`	bf16	fp32	mixed precision (§7)
`n_layer`, `n_head`, `n_embd`	12, 12, 768	6, 6, 192	the decoder's shape (Lecture 4 (Section 8))

Three of its choices are not in the file but in the model's code: the initialization of §7; the table of 50,304 rows, GPT-2's 50,257 tokens padded to a multiple of 64 because matrix products of such sizes run faster on GPUs, the extra rows never being used; and weight decay on exactly the tensors of two or more dimensions.

Two numbers follow from the file and are worth computing. The run reads 295 billion tokens with 124 million parameters, about 2,400 tokens per parameter, far more than the compute-optimal 20 of §9: it trains GPT-2's architecture on about as many tokens as GPT-3's models read (GPT-2's own count was not published), rather than spending its computation best. And `lr_decay_iters` is a separate field from `max_iters` because the cosine is defined by the step at which it ends; its comment says to make them equal, so that the rate reaches `min_lr` exactly when training stops.

Published runs

run	tokens	tokens per step	peak rate	warmup	decay
GPT-3 Small, 125M	300B	0.5M	6.0×10^{-4}	375M tokens	cosine to 10%
GPT-3, 175B	300B	3.2M	0.6×10^{-4}	375M tokens	cosine to 10%
Llama 2, 7B	2.0T	4M	3.0×10^{-4}	2,000 steps	cosine to 10%
Llama 2, 70B	2.0T	4M	1.5×10^{-4}	2,000 steps	cosine to 10%
Llama 3, 405B	15.6T	4M to 16M	0.8×10^{-4}	8,000 steps	cosine to 1%
ours, 3.5M	49M	8,192	2×10^{-3}	200 steps	cosine to 10%

The published configurations of large runs, from Brown et al. (2020), Touvron et al. (2023b), and the Llama Team (2024), differ from one another in a few fields only. What to notice:

1. **The peak rate falls as models grow**, tenfold from GPT-3's smallest model to its largest, for the reason of §7.
2. **The batch grows**, from half a million tokens to 16 million, as the noise scale of §6 does.

3. **The optimizer's other settings hardly change**: AdamW with $\beta = (0.9, 0.95)$, $\lambda = 0.1$, and clipping at 1 in GPT-3, Llama 2, and ours (the Llama 3 paper does not list them), and the cosine to a tenth in most.
4. **Tokens per parameter vary more than a thousandfold**, from 1.7 for GPT-3's largest model to 2,400 for its smallest, both on 300 billion tokens (about 290 for Llama 2's 7-billion-parameter model); §9 asks what the right number is.

Configurations of the newest runs often change the optimizer: nanochat's, for example, trains the matrices inside the blocks with Muon and the rest with AdamW, each group at its own learning rate, and reads 20 tokens per parameter instead of a fixed number of steps (Appendix B.3).

What a run costs

How long does a run take, on how many GPUs, and for how much money? A first estimate needs only the rule of Lecture 4 (Section 10): training a model of N parameters on D tokens costs about $C \approx 6ND$ operations. Divide by what a GPU really delivers, about 40% of its peak (an H100 peaks at about 10^{15} bf16 operations per second, so about 4×10^{14} delivered), and you have GPU-seconds; multiply GPU-hours by a rental price of about \$2 to \$3, and you have dollars. Two checks against published runs:

1. **nanochat's model of 561 million parameters on 11.2 billion tokens** (its first release): $6 \times 5.61 \times 10^8 \times 1.12 \times 10^{10} \approx 3.8 \times 10^{19}$ operations, which at $8 \times 4 \times 10^{14}$ per second on 8 H100s is 3.3 hours. Karpathy reports about 3 hours of pretraining, and 3 hours 51 minutes, \$92 at \$24 an hour for the node, for the whole pipeline to a fine-tuned chat model (in the walkthrough of its first release, Karpathy, 2025).
2. **Llama 3's model of 405 billion parameters on 15.6 trillion tokens**: $6 \times 4.05 \times 10^{11} \times 1.56 \times 10^{13} \approx 3.8 \times 10^{25}$ operations, the figure its paper gives, or 26 million H100-hours at the same rate; its model card reports 30.8 million, and the paper a utilization of 38% to 43% on up to 16,384 H100s (Llama Team, 2024).

Published runs span more than six orders of magnitude in computation, from about 7×10^{18} operations for llm.c's GPT-2 to 4×10^{25} for Llama 3 (ours: 10^{15}):

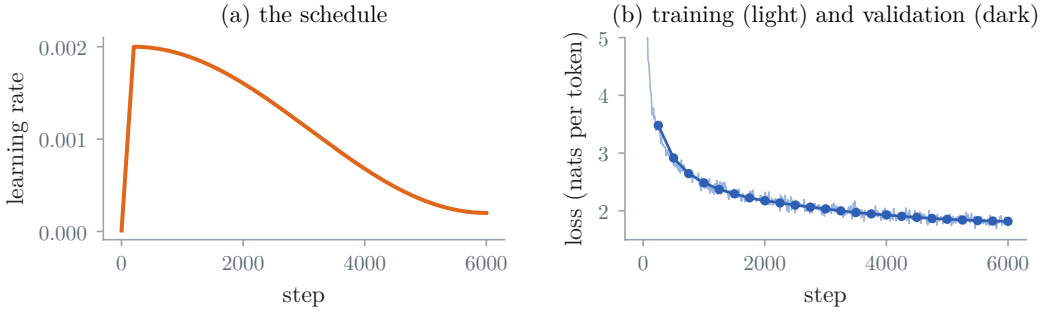
run	parameters	tokens	hardware	time or GPU-hours	cost
ours: base model (§8)	3.5M	49M	one CPU, no GPU	6.0 hours	—
GPT-2 small, llm.c (2024)	124M	10B	8 A100	90 minutes	about \$20
GPT-2 small, nanoGPT (§8)	124M	295B	8 A100	about 4 days	—
nanochat, first release (2025)	561M	11.2B	8 H100	about 3 hours	about \$92 with fine-tuning
GPT-2 XL, llm.c (2024)	1.6B	33.6B	8 H100	24 hours	\$672
GPT-3 (2020)	175B	300B	V100s	3.1×10^{23} operations	not published
Llama 2 70B (2023)	70B	2.0T	A100s	1.7 million GPU-hours	not published

run	parameters	tokens	hardware	time or GPU-hours	cost
DeepSeek-V3 (2024)	671B (37B per token)	14.8T	2,048 H800	2.8 million GPU-hours	$5.6 million at $2 per GPU-hour
Llama 3 405B (2024)	405B	15.6T	up to 16,384 H100	30.8 million GPU-hours	not published

Sources: Karpathy (2024c), Karpathy (2024d), nanoGPT's README, Brown et al. (2020), Touvron et al. (2023b), DeepSeek-AI (2024), and the Llama 3.1 model card. DeepSeek's figure counts only the final run, "excluding the costs associated with prior research and ablation experiments", and so do most others. Estimates of what the largest runs cost, by Epoch AI, reach hundreds of millions of dollars, about $53 million for Llama 3's 405B model, and have grown about 2.4 times a year since 2016 (Cottier et al., 2024). Long runs also break: in a 54-day period of Llama 3's training, its jobs were interrupted 466 times.

Fine-tuning is cheap beside pretraining. InstructGPT's supervised fine-tuning of the 175-billion-parameter GPT-3 took 4.9 petaflop/s-days, against 3,640 for GPT-3's pretraining, about 0.1%, and its reinforcement learning 60, under 2% (Ouyang et al., 2022). Alpaca fine-tuned the 7-billion-parameter LLaMA in 3 hours on 8 A100s, for under $100 (Taori et al., 2023); QLoRA fine-tuned a 65-billion-parameter model on one 48 GB GPU in 24 hours (Dettmers et al., 2023); nanochat's midtraining took about 8 minutes and its fine-tuning about 7 on its 8 H100s. Ours: 56 minutes of midtraining and 36 of fine-tuning on a CPU, against 6.0 hours of pretraining.

Our base model



(c) the running prompt (black), continued by the model (blue)

step 0	Lily saw a big dog in the park. warmhes eyes birdR tougilla tiedlaimed My keep pull treasure mom gratefulhyorable finds snee sing too tankMom birds holes succeed press how market Lila showed back hon screamed Grandpashineou …
step 200	Lily saw a big dog in the park. He was very happy to go on the floor. He was surprised and happy because she liked to join the closet. She had to do, but he was a happy day. One day, she saw a little bird. She had a voice on his …
step 1,000	Lily saw a big dog in the park. The dog was very fast and had a very big dog. The dog was fierce and needed help. The dog wanted to roar, but he just wanted to play with it. He went to the park with the dog, a dog named Spot. Spot …
step 6,000	Lily saw a big dog in the park. The dog was black and brown. It looked like other dogs. Lily liked dogs. She wanted to pet the dog, too. "Can I pet the dog?" Lily asked the dog. The dog was friendly. It saw a man with a shiny …

Figure 32. Our base model's training: the loss falls fast, then slowly, and the stories become stories. (a) The learning rate (warmup over 200 steps, cosine decay to a tenth). (b) The training loss (light, every 10 steps) and the validation loss (dark, every 250 steps), in nats per token. (c) The running prompt continued by the model at four points of training (temperature 0.8).

Our base model is Lecture 4's decoder made deeper and wider: 6 blocks of width $d = 192$, 6 heads of 32 dimensions, MLPs of 768, rotary positions, a context of 512 tokens, and our 4,096-token table tied to the output layer, 3,456,000 parameters in all. It was trained with the configuration of the table above, for 6,000 steps, 49 million tokens, in 6.0 hours on an ordinary CPU, with no GPU. Its peak rate, 0.002, is a fifth of the best rate of §6's sweep, 0.01, because the base model is wider and deeper than the sweep's model, and the best rate falls as models grow (§7); it was chosen by that argument, not tuned.

It reaches a validation loss of 1.818 nats per token, 0.661 bits per byte. The first sentence of Lecture 4's running sentence, *Lily saw a big dog in the park.*, continued at four points of training, shows what each stage adds (Figure 32c): random pieces of words at step 0, words and punctuation of stories that do not fit together at step 200, sentences about the prompt's dog that contradict each other at step 1,000, and a consistent scene at the end.

One choice remains: we chose the size of our model, and the number of tokens it reads, before training it. How large should it be?

9. How large a model: scaling laws

For a budget of computation, how many parameters, and how many tokens? Training costs about $6N$ operations per token for a model of N parameters (Lecture 4 (Section 10)), so a run that reads D tokens costs

$$C \approx 6ND$$

floating-point operations. A fixed budget C can buy a large model that reads little or a small model that reads much. Which is better is an empirical question, and it has an empirical answer.

Power laws

Kaplan et al. (2020) trained Transformers from about a thousand to a billion parameters on web text and found that the loss falls as a **power law**: a straight line on logarithmic axes, in the number of parameters when data is plentiful, in the number of tokens when the model is large enough, and in the computation when both are chosen well, over several orders of magnitude. Hoffmann et al. (2022) fitted the loss of models trained with N parameters on D tokens as

$$\mathcal{L}(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}},$$

an irreducible loss E, the entropy of text itself, plus one term that a larger model reduces and one that more data reduces. Their fit gave $\alpha = 0.34$ and $\beta = 0.28$; Besiroglu et al. (2024), refitting the same law to Hoffmann et al.'s runs, found that fit to be in error and obtained $E = 1.82$, $\alpha = 0.35$, and $\beta = 0.37$.

The compute-optimal model

Along a budget, a larger model reads fewer tokens: one term of the loss falls and the other rises, and their sum has a minimum (Figure 33). To find it, minimize $\mathcal{L}(N, D)$ subject to $6ND = C$. With a Lagrange multiplier μ, the minimum has $\partial\mathcal{L}/\partial N = 6\mu D$ and $\partial\mathcal{L}/\partial D = 6\mu N$, where

$$\frac{\partial \mathcal{L}}{\partial N} = -\frac{\alpha A}{N^{\alpha+1}}, \qquad \frac{\partial \mathcal{L}}{\partial D} = -\frac{\beta B}{D^{\beta+1}}.$$

Multiplying the first condition by N and the second by D gives $-\alpha A/N^{\alpha} = 6\mu ND$ and $-\beta B/D^{\beta} = 6\mu ND$, so

$$\alpha \frac{A}{N^{\alpha}} = \beta \frac{B}{D^{\beta}} :$$

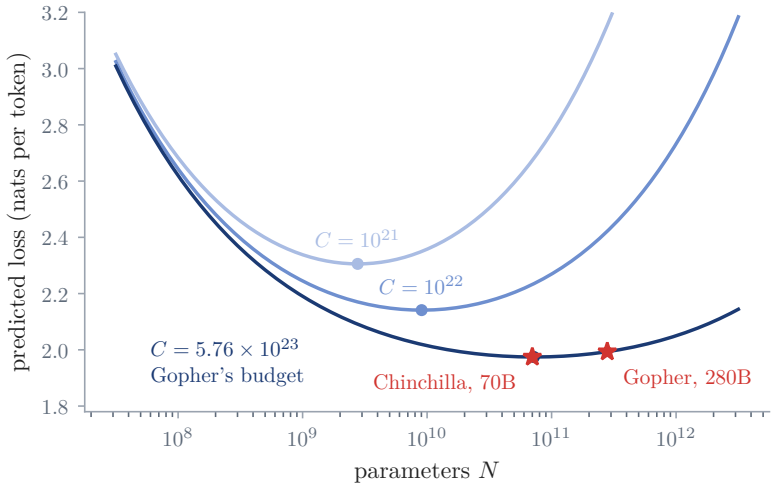


Figure 33. A fixed budget has a best size. The loss predicted by $\mathcal{L}(N, D)$, with the constants of Besiroglu et al. (2024), for a model of N parameters trained on $D = C/6N$ tokens, at three budgets C (in operations); the dot on each curve is its minimum, at 18 to 22 tokens per parameter. On Gopher's budget, 5.76×10^{23}, Chinchilla (70 billion parameters) sits at the minimum, and Gopher (280 billion) to its right. Logarithmic N axis.

at the optimum, the last unit of computation spent on parameters and on data lowers the loss equally. Substituting $D = C/6N$ gives

$$N^* \propto C^{\beta/(\alpha+\beta)}, \qquad D^* \propto C^{\alpha/(\alpha+\beta)}.$$

When $\alpha \approx \beta$, both grow as $\sqrt{C}$: a budget ten times larger should buy a model about three times larger, trained on about three times as many tokens. Hoffmann et al. measured the allocation three ways, which gave exponents of 0.50, 0.49, and 0.46; the first two, and the corrected fit, put the optimum near **20 tokens per parameter**. Their model Chinchilla, 70 billion parameters on 1.4 trillion tokens, did better than the 280-billion-parameter Gopher trained with the same computation on 300 billion (§1). The derivation in full, and the three methods, are in Appendix D.

Kaplan et al. had concluded the opposite, $N^* \propto C^{0.73}$, so that a larger budget should go mostly to a larger model, and GPT-3 was sized that way. Later work traced most of the difference to how Kaplan et al. counted parameters, without the token table, and to their warmup and learning-rate schedule (Appendix D).

Our models on three budgets

The same experiment at our scale (Figure 34): models from 155,648 to 3,440,640 parameters (counting, as N does, every weight used in a matrix product, the token table included), each trained with the recipe of §§7–8 on $D = C/6N$ tokens for three budgets, the longest run taking 69 minutes on one CPU core. For each budget the loss is lowest at an intermediate size: smaller models have too few parameters to use their data, and larger ones read too little of it. The best sizes grow as $C^{0.52}$, with about 17 tokens per parameter, close to Hoffmann et al.'s exponent of about 0.5 and their 20 tokens per parameter. Continuing the running prompt, the smallest, best, and largest models of the largest budget write stories that differ less clearly than their losses, 2.54, 2.31, and 2.65 nats per token.

Our base model of §8, with 49 million tokens for 3.5 million parameters, read 14 tokens per parameter, near the optimum. Its budget, $6 \times 3.44 \times 10^6 \times 4.9 \times 10^7 \approx 1.0 \times 10^{15}$ operations,

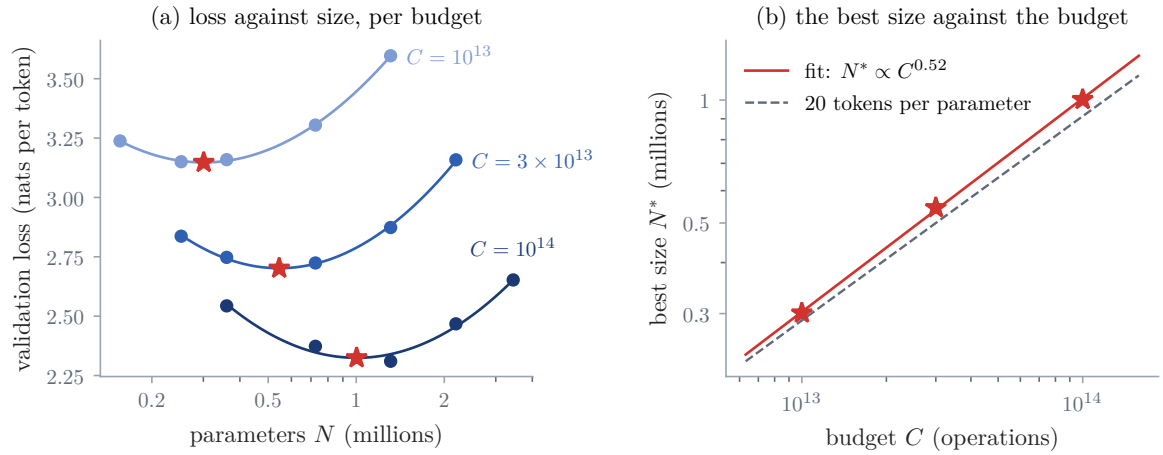


Figure 34. For each budget there is a best size, and it grows with the budget. Models of five sizes on each of three budgets, 10^{13}, 3×10^{13}, and 10^{14} operations, each reading $D = C/6N$ tokens with its schedule matched to its length. (a) Validation loss against the number of parameters, with a parabola in $\log N$ fitted to each budget and its minimum marked. (b) The best size against the budget, with the fitted power law; the dashed line is 20 tokens per parameter. Logarithmic axes; single runs.

is ten times our largest. Extrapolated that far, the fitted power law puts the best size for that budget at 3.4 million parameters, about 15 tokens per parameter.

Past the optimum, and out of data

The compute-optimal model minimizes the cost of training. But a model is trained once and used many times, and each use costs about $2N$ operations per token. A smaller model trained on more tokens than the optimum costs more to train for the same loss, and less to use. So most models since 2023 are trained far past it: Llama 3's 8-billion-parameter model read over 15 trillion tokens, about 1,900 per parameter (Llama Team, 2024), and its loss was still falling log-linearly at the end (Meta, 2024).

Text is the other limit. The stock of public human-written text is estimated at a few hundred trillion tokens, which the largest runs could use up within a few years (Villalobos et al., 2024). Reading the same data again helps less than new data, but not much less for the first few passes: Muennighoff et al. (2023) found up to four passes nearly as good as four times as much new text.

A large run does not fit on one accelerator (§7). Large runs split it across thousands of accelerators, by the batch (data parallelism, §7), by the weight matrices, and by the blocks; Appendix D describes the three and the memory of such a run. With the size settled and the model trained, what can the base model do?

10. The base model

What can a model trained only to continue text do? It computes $q(x_t \mid x_{<t})$ for text like its training text, and nothing else. Everything it does is a continuation of whatever text it is given, its **prompt**.

Completion and prompting

Given the beginning of a story, our base model writes a plausible rest of it (Figure 32c). Given anything else, it writes what would most plausibly follow that text in its training data. A

prompt therefore works as a **condition**: it selects, among all the documents the model could continue, those that begin this way. GPT-2 used this to do tasks no one trained it for, by writing prompts whose likely continuation is the answer. On the web, "TL;DR:" ("too long; didn't read") is followed by a summary of what came before it, so an article followed by "TL;DR:" is continued with a summary of the article (Radford et al., 2019; Figure 35). The summaries were rough: scored by the words they share with summaries written by people, they did barely better than three sentences picked from the article at random, and they mixed up details, but no one had trained GPT-2 to summarize. This is **zero-shot** use.

(a) the prompt, and GPT-2's continuation

an article (its start), then TL;DR:, then GPT-2's continuation

Amina Ali Qassim is sitting with her youngest grandchild on her lap, wiping away tears with her headscarf. Only a few months old, this is the baby girl whose ears she desperately tried to cover the night the aerial bombardment started. …

TL;DR:

Yemen is in the middle of a civil war. Saudi Arabia is leading the coalition bombing campaign. It's been bombing Yemen for more than two months now.

the first three sentences of 100 tokens sampled with top-k, $k = 2$

(b) the summary written by a person

reference summary (green)

Amina Ali Qassim's family sought shelter in a mosque before fleeing Yemen. Thousands like them are boarding boats to sail to Djibouti. Saudi Arabia has been pounding Yemen in a bid to defeat Houthi rebels.

it names the family; GPT-2's summary is about the war

Figure 35. A prompt can ask a base model for a task its training text contains. (a) The start of a news article from the CNN and Daily Mail dataset, then "TL;DR:", and what GPT-2 wrote after it (its first three sentences). (b) The summary written for the dataset. From Radford et al. (2019, Table 14).

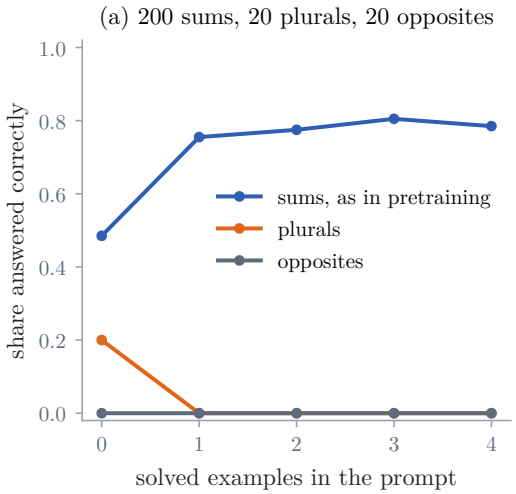
GPT-3 went further by writing a few solved examples of a task into the prompt before the new instance, **few-shot** use, and its accuracy rose with the number of examples and with the model's size (Brown et al., 2020): the model learns the task from its context, without any change to its weights. This is **in-context learning**.

Our base model knows only stories and sums: given *237 + 682 =*, in the format of the sums it read, it writes the right sum for 49% of 200 held-out pairs of three-digit numbers, and with one to four solved sums before the question, for 76% to 81% (Figure 36): the examples tell it that it is reading a document of sums. Tasks it never read in this form fare worse: opposites get none of 20 right with any number of examples, and plurals 4 of 20 with no example and none once examples are added. Given two opposites, it continues the sentence instead of completing the pattern, and given two plurals, it counts on (Figure 36b). Learning a new task from a few examples grew with the size of the model in GPT-3's experiments; at 3.5 million parameters, a model continues the formats it has read.

Induction heads

The simplest kind of in-context learning is copying: if a sequence occurred earlier in the context, predict that it continues the same way. Elhage et al. (2021) and Olsson et al. (2022) found the mechanism in trained Transformers, the program of two heads that Lecture 4 (Appendix B.1) pointed to. It completes … a b … $a \to b$ in two steps (Figure 37):

1. a **previous-token head**, in an earlier block, makes every position read the position just before it and add what it reads to its own vector, so that the position of b now carries "my previous token was a";



(b) with two examples: prompt, continuation

sums, as in pretraining, right
5 + 62 = 67
657 + 29 = 686
237 + 682 = 919

plurals, wrong: the answer is dogs
One duck, two ducks. One flower, two flowers. One dog, two, three, four.

opposites, wrong: the answer is small
The opposite of wet is dry. The opposite of day is night. The opposite of big is the best day of the day.

Figure 36. Examples in the prompt help with a format the model has read, and not with a new task. (a) The share of 200 held-out sums of two three-digit numbers, in the format of the pretraining documents (*237 + 682 =*), and of 20 plurals and 20 opposites, answered correctly by the base model's greedy continuation, against the number of solved examples placed before the question. (b) One prompt of each kind with two solved examples (gray), and the base model's greedy continuation (blue).

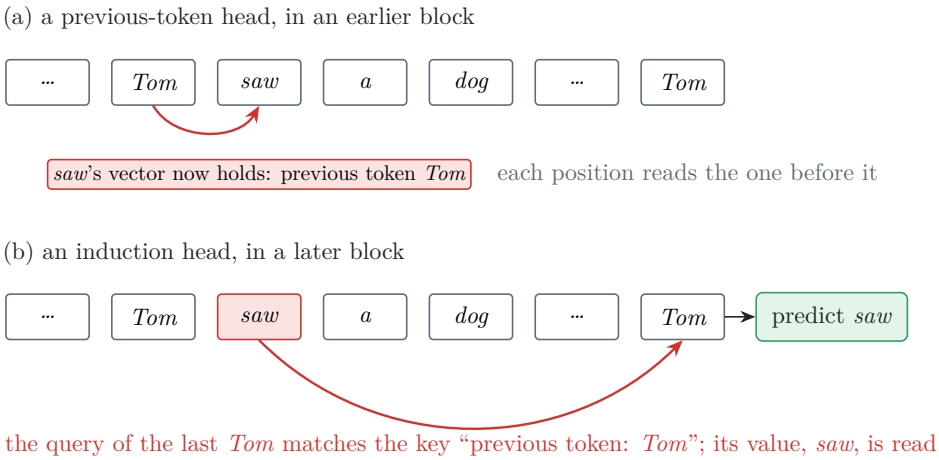


Figure 37. Two heads in two blocks copy what followed the previous occurrence of the current token. The text *… Tom saw a dog … Tom.* (a) A previous-token head makes each position read the position before it (red arc), so that the vector at *saw* now carries "previous token: *Tom*". (b) In a later block, an induction head at the last *Tom* uses its token as a query; the key that matches it is the one written in (a), at the position of *saw*, whose value, *saw*, is read (red arc) and becomes the prediction (green).

2. an **induction head**, in a later block, lets the current position, whose token is a, use its token as a query and find the key "previous token was a": the position of b. It reads that position's value, b, and adds it to the prediction.

The two heads need two blocks, since the second reads what the first wrote. Olsson et al. found that such heads form abruptly during training, at the same point at which the model becomes much better at using tokens far back in its context.

Our base model has such heads. An invented name it has never read, such as *Fuzak*, costs it 25.7 nats the first time and 5.2 the second, and at the second occurrence one head of block 4 attends to the piece that followed the same piece the first time, while a head of block 2 attends to the previous token (Appendix E.1 has the experiment and its figure). Copying is the part of in-context learning whose mechanism is known; other accounts of it are in Appendix E.

(a) the conversation as plain text

prompt
You are a storyteller for young children.
Tell me a short story about a dog who finds a bone.

what the base model wrote
"Let's share the bone!" he said, feeling surprised.
"No, I don't think the dog's owner, mother!"
The dog felt sad and angry. "But why?" he asked.
"You can't have any bone," said the dog sadly.
The owner, who was watching him, saw him trying to do better.
The end.

(b) a prompt that is a conversation, as on the web

prompt
The following is a conversation between a user and an assistant. The assistant is a storyteller for young children, and answers each request with a short story.
User: Tell me a short story about a dog who finds a bone.
Assistant:

what the base model wrote
"Let's play a game with this dog!"
The two of them argued over a toy, but then the other brother in the argument.
"That's not fair," the first neighbor said. "You should share the bone with others. Then you can play with it properly."
The children started to argue, but the argue was OK.
They both had an idea!
"I'm …

(c) the same, with two example exchanges first

prompt
The following is a conversation between a user and an assistant. The assistant is a storyteller for young children, and answers each request with a short story.
User: Tell me a story about a cat who loses her ball.
Assistant: Once upon a time, there was a little cat named Mia. Mia had a red …
User: Tell me a story about a boy who plants a seed.
Assistant: Once upon a time, a boy named Tom found a small seed. He put it in …
User: Tell me a short story about a dog who finds a bone.
Assistant:

what the base model wrote
"Let's go to your friend's house. She will keep me safe."
Tom was excited and said: "Yes, let's go!"
So **Tom** and ACind Mrs. Lee went to the garden. They played with the **green plant** all day. They had so much fun!
At the end, they both said goodbye. They were happy to have a new friend.

Figure 38. A base model continues a request; it does not answer it. Our base model given the running request three ways (temperature 0.8, the first sample of each): (a) the system message and the request as two lines of plain text; (b) a preamble that describes a conversation, then the request after *User:* and the cue *Assistant:*; (c) the same with two example exchanges first, written for this experiment. In (c) the reply borrows a character and an object from the examples (orange). The answers are cut at about 330 characters.

What a base model is not

What does the base model do with a request? Give it the running conversation as plain text, the system message and the request on two lines, and let it continue (Figure 38a). It writes the middle of a story about a dog and a bone, with no beginning; its two other samples copy the form of the request, *Tell me two arms, one twice, …*, or start a story about children building a house. A base model has no notion of a request, a turn, or an answer: it continues documents.

Could the right prompt make it an assistant, with no training? A large base model that has read conversations on the web can be made to act as one by a prompt that is itself a conversation: Askell et al. (2021) did it with a prompt of example conversations between a person and a helpful, honest, and harmless assistant, ending with the person's new request and the assistant's label. Ours, given such a prompt, writes the dialogue of a story instead of a story (Figure 38b), and with two example exchanges written before the request, it copies their characters and objects into its reply (Figure 38c). On 100 held-out requests, with or without the examples, it never wrote *User:* to hand the turn back; it ended the document with `<|endoftext|>` instead, in 98 of them without the examples and 99 with them, and of the words that some requests ask the story to use (§13), it used only 2% and 1%, respectively. Its examples help only with a format it has read (above), and a transcript of a conversation is not one. To take part in a conversation, the model must be trained on conversations, written in a form that marks whose turn it is.

11. A conversation as tokens

How is a conversation written so that a language model can take part in it? A language model reads and writes one sequence of tokens. A conversation has parts with different authors: standing instructions from whoever deploys the model, turns from a user, and turns from the model. Written as one sequence, each part must be marked with who wrote it, and the model's turns must have an end, so that the program running the model knows when to stop it and hand the turn back. §10's base model, given a request, had neither: it could not tell the request from a story, and it never handed the turn back.

Messages and roles

A program that uses a chat model, such as an application calling a model's interface over the web, holds a conversation as a list of **messages**, each with a **role** and a text. The running conversation of this lecture is two messages:

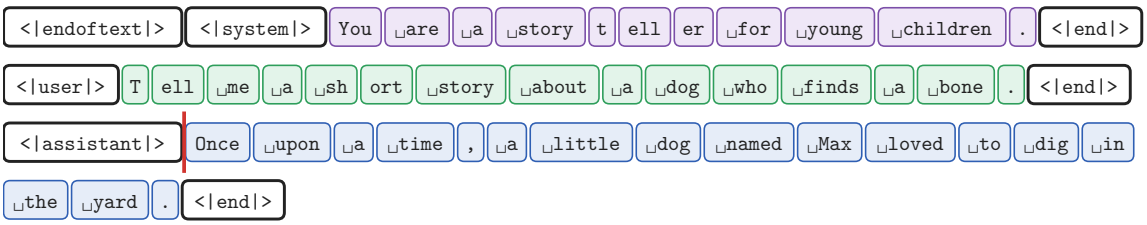
```
[{"role": "system",
  "content": "You are a storyteller for young children."},
 {"role": "user",
  "content": "Tell me a short story about a dog who finds a bone."}]
```

Every chat model in use distinguishes three roles:

1. **system**: instructions that hold for the whole conversation, written by the developer who deploys the model (a persona, a task, a style, rules), placed first;
2. **user**: the turns of the person talking to the model;
3. **assistant**: the model's own turns, the only part it writes.

The model's answer will be a third message, with the role assistant. Models that call tools add a fourth role, for the tool's output (Week 9).

The chat template



fed to the model: everything up to the red line (the generation prompt); the model writes the rest, until `<|end|>`

Figure 39. A conversation is one sequence of tokens, with special tokens around each turn. The running conversation in our chat template, with a short answer written for the figure: the system message (purple), the user's request (green), and the assistant's answer (blue), each opened by its role's token and closed by <|end|> (outlined). To make the model answer, the program feeds everything up to and including <|assistant|>, the generation prompt, and lets it write until it emits <|end|>. The same colors mark the roles in every figure of §§11–14.

A **chat template** writes the list of messages as one token sequence by putting special tokens around each message. Ours uses the special tokens reserved in §4. The sequence begins with `<|endoftext|>`, as every document of pretraining did, and each message is its role's token, `<|system|>`, `<|user|>`, or `<|assistant|>`, then its text, then `<|end|>` (Figure 39). The special tokens are single tokens that no text can produce (§4), so a user cannot end their turn, or open one for the assistant, by typing `<|end|>`. Our template is a few lines of code, which also mark which tokens the assistant wrote, for training (§13):

```
def render(messages):
    ids, mask = [ENDOFTEXT], [0]
    for m in messages:
        body = [ROLE[m["role"]]] + encode(m["content"]) + [END]
        ids += body
        mask += [0] + [m["role"] == "assistant"] * (len(body) - 1)
    return ids, mask
```

Published templates differ in their markers and keep the same structure (Figure 40). **ChatML**, introduced by OpenAI and used by Qwen, opens a message with `<|im_start|>` followed by the role's name as ordinary text and a new line, and closes it with `<|im_end|>`:

```
<|im_start|>system
You are a storyteller for young children.<|im_end|>
<|im_start|>user
Tell me a short story about a dog who finds a bone.<|im_end|>
<|im_start|>assistant
```

Llama 3 writes the role's name between `<|start_header_id|>` and `<|end_header_id|>`, follows it with two new lines, and closes a message with `<|eot_id|>` (Appendix F has it in full). Every character counts, the new lines and spaces included: they become tokens, and a template that writes them differently from the one used in fine-tuning gives the model a slightly unfamiliar document. Libraries therefore store each model's template with its tokenizer, as a small program that renders a list of messages. A template may also add text that no message contains: Qwen 2.5's inserts the system message *You are Qwen, created by Alibaba Cloud. You are a helpful assistant.* when a conversation has none (Qwen Team, 2024). A model must be used with the template it was fine-tuned on.

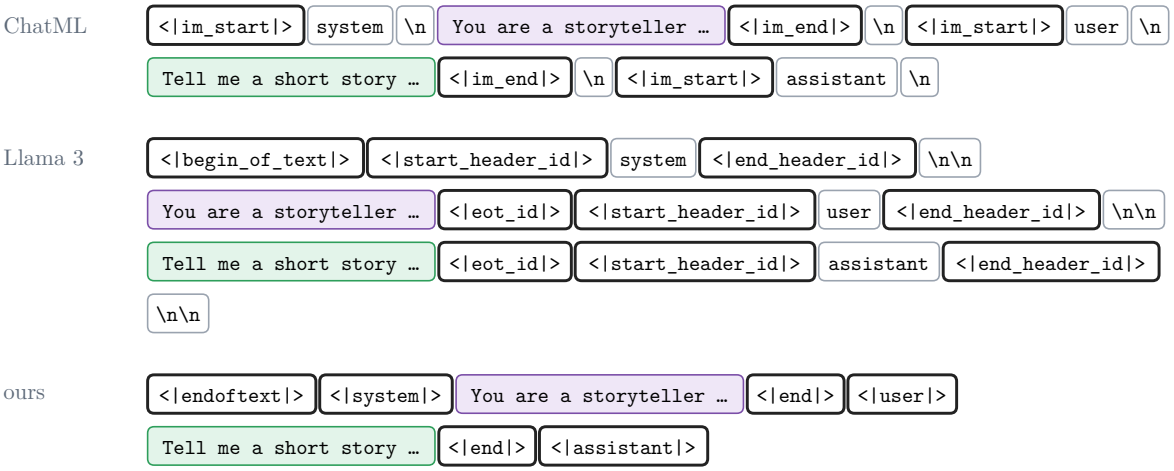


Figure 40. Different chat templates, the same structure. The running conversation's first two messages in ChatML, in Llama 3's template, and in ours. Special tokens are outlined; the role names in ChatML and Llama 3 are ordinary text.

Generating a reply, turn by turn

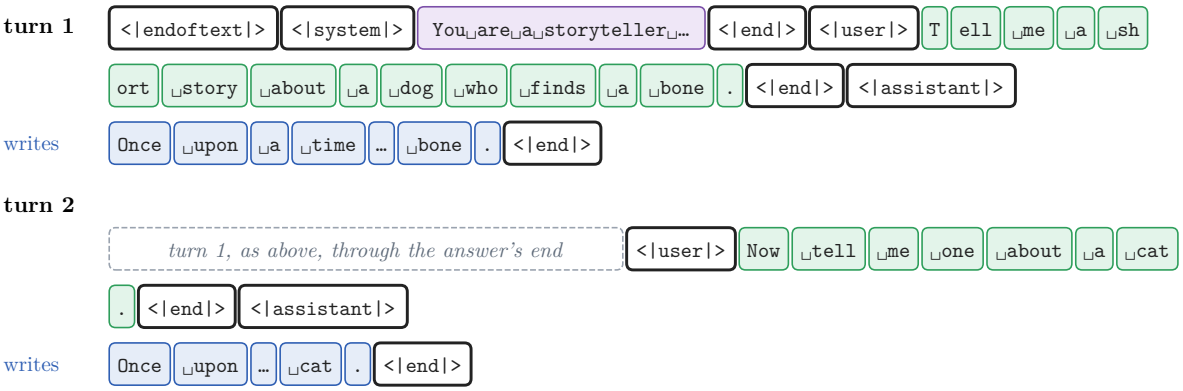


Figure 41. Each turn's prompt is the whole conversation so far, ending with the generation prompt. The running conversation over two turns, as token sequences. Turn 1: the program writes the system message and the request, then <|assistant|>; the model writes its answer and <|end|> (blue). Turn 2: the program appends the user's next message, from <|user|> to <|end|>, and a new <|assistant|>; the model writes its second answer. The answers are sketched for the figure.

Using the model is the generation loop of Lecture 4 (Section 8) with a particular prompt and a particular stop (Figure 41):

1. Render the messages so far with the template, and append `<|assistant|>`, the **generation prompt**.
2. Generate tokens until the model emits `<|end|>`, or until a length limit.
3. Show the user the text between `<|assistant|>` and `<|end|>`, and store it as an assistant message.
4. When the user replies, append their message, `<|user|>`, its text, and `<|end|>`, then a new `<|assistant|>`, and go to step 2: the whole conversation so far is the next prompt.

The program, not the model, decides when the turn ends: it stops sampling at `<|end|>`. A model that has not learned to write `<|end|>` keeps writing past the end of its answer until the length limit (§13 has one). The cache of keys and values of Lecture 4 (Section 8) built during one turn is kept for the next, so a turn costs only its new tokens. A conversation grows with every turn, and when it no longer fits the context, 512 tokens for our model, the oldest turns must be dropped or summarized.

System messages

The system message, often called the **system prompt**, is the developer's lever: the same model becomes a storyteller, a tutor, or a customer-service agent by its first few lines. It can set:

1. a persona and a task: *You are a storyteller for young children.*;
2. a style or format: *Your stories have a sad ending.*, *Answer in one sentence.*;
3. rules: what to refuse, what to keep private;
4. facts the model needs: today's date, the user's name, the tools it may call.

The system prompts of deployed assistants run to many paragraphs; Anthropic publishes those of its Claude applications (Anthropic, 2024). A model follows a system message only as far as its training taught it to, from conversations in which the system message and the assistant's turns agree. Our fine-tuning conversations state some features of each story in the system message, such as *In your stories, the characters talk to each other.*, and §13 measures how far our chat model follows them.

What a system message cannot do follows from what it is: tokens at the start of the context. Nothing enforces it. A user's later turn that contradicts it, or text inside a document the model is asked to read, competes with it for the model's attention; text planted in a web page or an email that a model reads can carry instructions of its own, **indirect prompt injection** (Greshake et al., 2023). Models are therefore fine-tuned to rank their instructions, system above user above the text of tools and documents (Wallace et al., 2024), and this lowers the risk but does not remove it.

The base model of §10 has never seen a special token, or a conversation. As inputs, the rows of `<|system|>`, `<|user|>`, `<|assistant|>`, and `<|end|>` in the token table were never read. As outputs, through the tied table, they were never the target, so at every position of pretraining their scores were pushed down, and `<|end|>` is now among the tokens the model is least likely to write. The model must learn the template, and what an assistant writes inside it, from conversations. Two stages teach it: midtraining, which continues pretraining on a mixture that includes conversations (§12), and supervised fine-tuning on demonstrations (§13).

12. Midtraining

How does the base model learn the template, and the tasks it will be asked to do, before it is taught to be an assistant? Given the running conversation in the template, our base model does not answer: it continues with the middle of a story, and on 100 held-out requests it never ends its turn with `<|end|>`. On held-out conversations its loss on the assistant's tokens is 1.90 nats per token, against 1.818 on ordinary stories. It has to learn a new format, new tokens, and, for anything beyond stories and sums, new tasks. Fine-tuning on a few thousand demonstrations could try to teach all of this at once. Many recent runs instead teach much of it earlier, and more cheaply, in a stage of its own.

Pretraining continued, on a shifted mixture

Midtraining is pretraining continued: the same loss on every token, the same packed chunks, and the same optimizer, but on a mixture shifted toward what the model will be used for, usually while the learning rate decays to zero. Published runs shift the mixture in two directions:

1. **Toward higher-quality and specialized text.** Llama 3 decayed its learning rate to zero over its last 40 million tokens while reading more of its best sources. On its 8-billion-parameter model, a short anneal on a little high-quality mathematics and code, which included the training sets of two benchmarks, grade-school word problems (GSM8K) and competition mathematics (MATH), raised its scores on their validation sets by 24.0% and 6.4% (Llama Team, 2024). OLMo 2 spent 5 to 10% of its computation on such a stage (Team OLMo, 2024). Extending the context, from 8K tokens to 128K for Llama 3, is often done here too.
2. **Toward the formats of use.** nanochat's midtraining, in late 2025, taught the chat template, multiple-choice questions, and the use of a calculator from more than half a million conversations, packed as in pretraining (Karpathy, 2025).

Why a stage of its own, rather than more fine-tuning? Three reasons:

1. **Volume.** A new format, and a new task, take many examples to learn, and packed chunks read them at pretraining's cost per token, with nothing spent on padding.
2. **Forgetting.** A model trained only on new data drifts away from what it knew; mixing in pretraining text keeps it close (Liu et al., 2025b).
3. **The decay.** A run with warmup–stable–decay (§7) ends its stable phase unsettled, and midtraining is its decay, on the new mixture.

The line between midtraining and fine-tuning is drawn by how the data is fed as much as by what it is: nanochat dropped the stage in January 2026, once its fine-tuning could read whole conversations efficiently, and moved the mixture there.

Our midtraining run

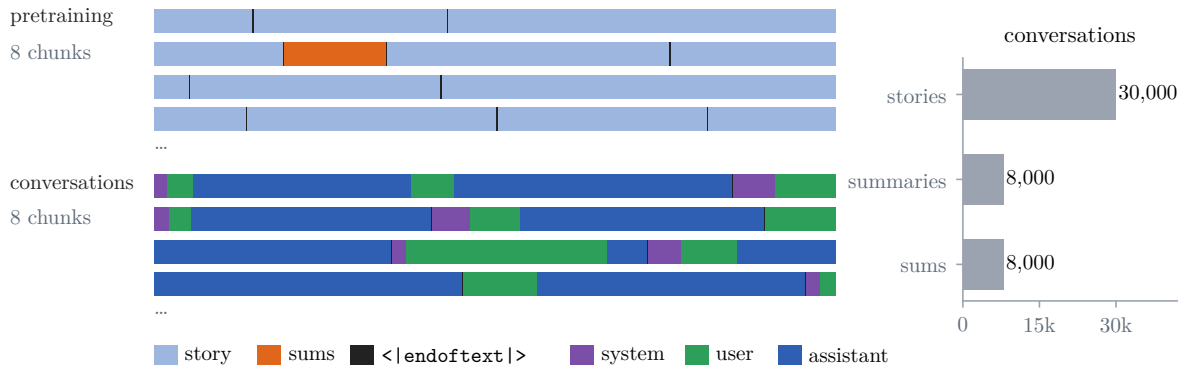


Figure 42. Midtraining reads pretraining text and conversations side by side. Each step of our midtraining run reads 16 chunks of 512 tokens: 8 cut from the pretraining stream (stories and sums, blue and orange) and 8 cut from a stream of conversations in the chat template (story requests, summaries, and sums), packed as pretraining documents are, so that a chunk can start or end inside a conversation. Every token is in the loss. Right: the share of each kind of conversation, by number.

We midtrained the base model for 1,000 steps on the mixture of Figure 42:

1. **Data.** Half of each step's chunks come from the pretraining stream. The other half come from 46,000 conversations, 11.0 million tokens, drawn from TinyStories-Instruct stories that the fine-tuning conversations of §13 do not use: story requests built as in §13, sums, and a task fine-tuning never shows, summarizing. Its user gives a story and asks *Summarize this story in one sentence.*, and its assistant answers with TinyStories-Instruct's summary of the story. None of the held-out sums of §§13–14 occurs in it.
2. **Loss and batches.** The log loss on every token, the system's and the user's included, on packed chunks.
3. **Optimizer.** AdamW as in pretraining, continued from its two averages at the end of pretraining; the rate warms up over 50 steps to 4×10^{-4}, a fifth of pretraining's peak and twice the rate at which pretraining ended, stays there, and decays linearly to zero over the last 30% of the steps.

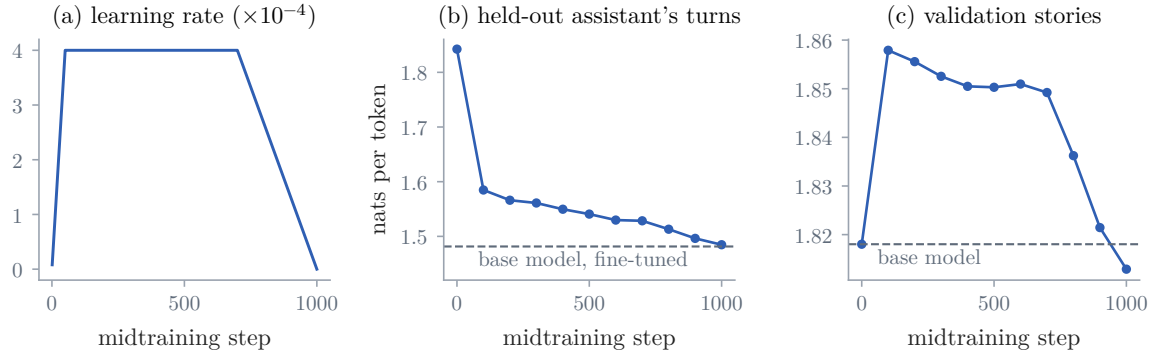


Figure 43. Midtraining teaches the format without losing the stories. Our midtraining run, step 0 being the base model: (a) the learning rate; (b) the loss on the assistant's tokens of 150 held-out conversations, the set measured during training, with the value reached by fine-tuning the base model directly (§13) as a dashed line: from 1.84 to 1.48 nats per token, against the 1.48 of fine-tuning (the 300 story conversations of the text and of Appendix G, a separate set, give 1.90, 1.63, and 1.63); (c) the loss on the 500 validation stories, with the base model's value as a dashed line. Nats per token.

The run took 56 minutes on a CPU. It changed the model in four ways (Figures 43 and 44):

1. **The format.** Given the running conversation in the template, the midtrained model answers with a story about a dog and a bone (Figure 44a), and on 100 held-out requests it ends its turn with `<|end|>` in 85, against none for the base model. Its loss on the assistant's tokens of 300 held-out story conversations is 1.63 nats per token, the same as after fine-tuning the base model directly (§13), against 1.90 for the base model.
2. **The new task.** It summarizes stories it has not read before (Figure 44b), and ends 97 of 100 held-out summaries with `<|end|>`. To score a summary against TinyStories-Instruct's, we use **ROUGE-1** (Lin, 2004), which counts shared words: its precision is the share of the summary's words that occur in the reference, its recall the share of the reference's words that occur in the summary, and ROUGE-1 is their harmonic mean, 1 for the same words and 0 for none in common (we leave out function words such as *the* and *a*). The midtrained model scores 0.27, against 0.15 for the story's first sentence, 0.06 for another story's summary, and 0.08 for the base model.
3. **The stories are kept.** Its loss on the validation stories rose from 1.818 to 1.858 by step 100, stayed near 1.85 while the rate was constant, and fell to 1.813 during the decay, slightly below the base model's.

(a) the running conversation

system
You are a storyteller for young children.

user
Tell me a short story about a dog who finds a bone.

assistant: the midtrained model
Once upon a time, there was a dog named Spot. Spot was a very grateful dog. He had a red, red ball in his yard.
One day, Spot went for a walk. He felt warm and safe. Spot saw a big, shiny bone. He picked it up and put it in his mouth. It felt nice and warm. Spot was happy to eat the bone.
Spot ate the bone all the time. He felt good about eating it. He knew that his friends did not need it anymore. Spot learned …

(b) summarizing, the new task: a held-out story

user
Summarize this story in one sentence.
Once upon a time, there was a little girl named Lily. She loved watching TV, especially her favorite show about animals. One day, her mom told her it was time to turn off the TV and go outside to play. Lily didn't want to stop watching her show, so she got upset and started to cry.
Her mom tried to calm her down and suggested they go outside and play together. They went to the park and saw an amazing butterfly. Lily wanted to catch it, but she didn't know how. Her mom showed her how to fold her hands together to make a butterfly net. They caught the butterfly and watched it for a while before letting it go.
Lily realized that there were other amazing things to do besides watching TV. She was happy to spend time with her mom and learn new things. From that day on, she didn't mind turning off the TV to go outside and explore the world.

assistant: the midtrained model
Lily loves to watch TV and learns about the butterfly in the park, but her mom tells her it's time to turn off the TV and learn to be patient. `<|end|>`

TinyStories-Instruct's summary
Lily loves watching TV but her mom takes her outside to play and they catch a butterfly, making Lily realize there are other amazing things to do besides watching TV.

(c) a held-out sum

user
What is 237 + 682?

assistant: the midtrained model
137 + 282 = 429.

the right answer
237 + 682 = 919.

Figure 44. The midtrained model answers in its turn, summarizes, and gets sums wrong. (a) The running conversation in the template, and the start of the midtrained model's answer (temperature 0.8). (b) A held-out story to summarize, the model's summary (greedy), and TinyStories-Instruct's. (c) A held-out sum, the model's answer, and the right one.

4. **The sums are not learned.** It writes them in the right form but copies the operands wrongly and adds them wrongly too (Figure 44c), and it gets 7% of the unseen three-digit sums right.

Why were the sums not learned? Our sums conversations are a sixth of the conversations but, being short, only 2.7% of their tokens, and packing weighs a conversation by its tokens: in 1,000 steps the model saw about 3,000 of them. Fine-tuning gives every conversation a row of its own, and there the sums are a fifth of the rows. A midtraining mixture is set in tokens, so a skill that takes few tokens to show needs a larger weight than its share of the examples suggests. What midtraining leaves for fine-tuning, the content of the answers rather than their format, is the subject of §13.

13. Supervised fine-tuning

How does the model learn to fill the assistant's turns the way an assistant should? By the same next-token training once more, continued from the midtrained weights, on conversations in which the assistant's turns show what it should write: **supervised fine-tuning** (SFT), the first of InstructGPT's and ChatGPT's three steps (§1). Midtraining read conversations as text, every token predicted and conversations cut wherever a chunk ended. Fine-tuning reads each conversation whole, as the model will meet it in use, and learns only to write the assistant's part.

The loss on the assistant's tokens

A training conversation is rendered with the template and fed to the model in one pass, as in pretraining. The difference is which predictions count. The system message and the user's turns are given, not written by the model, so predicting them is not its job. The loss is the average log loss over the assistant's tokens only, the set A of positions whose target belongs to an assistant turn, including the `<|end|>` that closes it:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{|A|} \sum_{t \in A} \log q_\theta(x_t \mid x_{<t}).$$

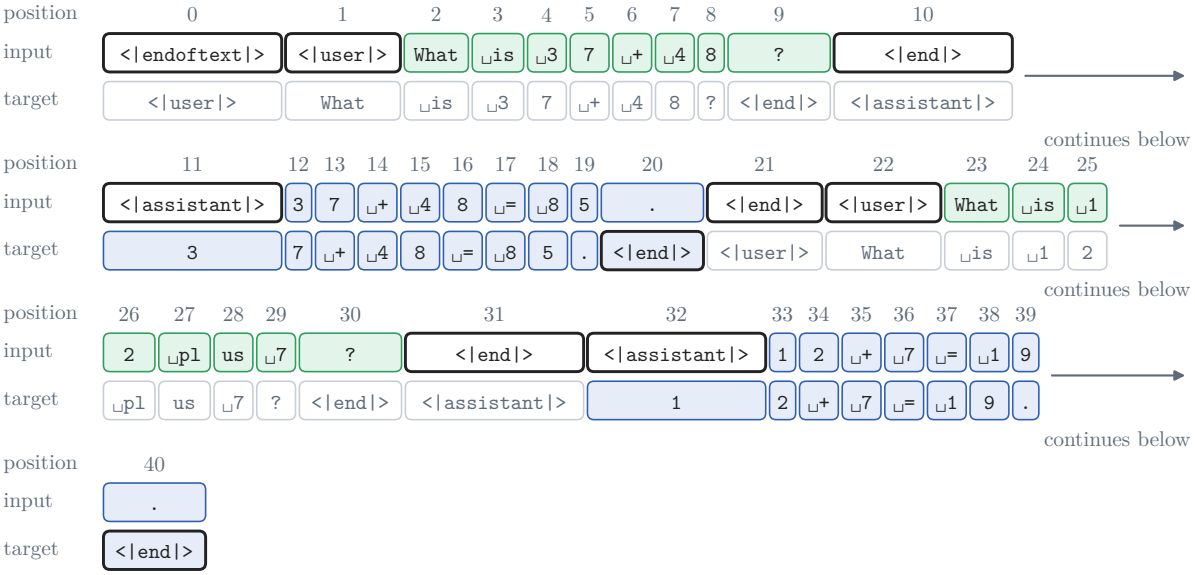


Figure 45. Only the assistant's tokens are predicted in the loss. A conversation of two turns as a training example, one sequence of 41 positions wrapped onto four rows to fit the page (the arrows continue it): each position's number, its input, and its target. The targets of the two assistant turns, each through its closing <|end|>, are in the loss (blue); the others are given (outlined in gray, not in the loss). Every position reads all the positions before it, whatever their role.

Every position still reads everything before it, the system message and the user's turns included; only the targets outside A are left out of the loss, by the **loss mask** that the template's code returns with the tokens (§11). The closing `<|end|>` is in A because it teaches the model to stop. In a conversation of several turns, every assistant turn is in A, each predicted from the whole conversation before it (Figure 45). The role tokens that open the assistant's turns are not in A: the program writes them.

Demonstrations

The conversations of SFT are **demonstrations**: examples of the behavior wanted. They come from three sources:

1. **written by people**: InstructGPT's demonstrations were answers to about 13,000 prompts, written by paid labelers (Ouyang et al., 2022), and ChatGPT's were conversations in which people wrote both the user's turns and the assistant's (§1);
2. **generated by a model** from a few examples written by people, then filtered (Self-Instruct, Wang et al., 2023);
3. **distilled from a larger model**: the answers of a stronger model to many prompts; Alpaca fine-tuned the 7-billion-parameter LLaMA on 52,000 answers from OpenAI's text-davinci-003 (Taori et al., 2023).

Before chat models, **instruction tuning** fine-tuned models on many existing tasks rewritten as instructions, and found that they then followed instructions for tasks they had not seen (FLAN, Wei et al., 2022a; T0, Sanh et al., 2022).

Ours are built from TinyStories-Instruct (Eldan and Li, 2023), which gives for each story a one-sentence summary, often a list of words it uses, and features such as a sad ending or dialogue. Each becomes a conversation (Figure 46):

1. **system:** *You are a storyteller for young children.*, with one sentence for each feature of the story (*Your stories have a sad ending.*, *In your stories, the characters talk to each other.*); in a fifth of the conversations without features, no system message;
2. **user:** a request from the summary, *Tell me a story in which a brave red bird protects its family …*, or from the word list, *Write a story that uses the words escape, war, and tall.*, in one of several phrasings;
3. **assistant:** the story.

One conversation in ten was given a second request and a second story (one in fourteen after the length limit below). To these come conversations of sums, such as *What is 37 + 48?* answered *37 + 48 = 85.*, with operands of one to three digits, half of them under a system message for arithmetic. After conversations longer than the context of 512 tokens are left out, there are 24,179 conversations, 5.3 million tokens. None contains the running conversation, and none asks for a summary.

The recipe of fine-tuning

Fine-tuning changes four of midtraining's choices (§12), each for a reason:

1. **One conversation per row.** Each row of a batch is one whole conversation, starting with `<|endoftext|>`, as in use; the 16 conversations of a batch have similar lengths and are padded to the longest, and the padding is left out of the loss like the user's tokens. No conversation is cut.
2. **The loss mask** of this section, rather than the loss on every token.
3. **A smaller learning rate**: a peak of 3×10^{-4}, about a seventh of pretraining's, with a warmup over the first 5% of the steps and a cosine decay to a tenth. The model starts close to what is wanted, and the demonstrations are few; a large rate would move it far from what pretraining taught it.
4. **One pass** over the demonstrations, 1,512 steps: reading the same few examples many times makes a model memorize them, as the smallest run below does.

(a) a summary as the request; one feature, dialogue

system
You are a storyteller for young children. In your stories, the characters talk to each other.

user
Can you tell me a story where Lily asks her mom for grapes, but they don't have any. Her dad comes home with a bag of grapes and Lily is happy?

assistant
Once upon a time, there was a little girl named Lily. She loved to eat grapes, but she didn't have any. She asked her mom, "Mommy, I want grapes, please?"
Her mom said, "I'm sorry, Lily. We don't have any grapes right now." Lily felt frustrated because she really wanted grapes. Suddenly, her dad came home with a big bag of grapes! Lily was so happy and said, "Yay! Grapes! Thank you, Daddy!"

(b) a list of words as the request; no features

user
Tell me a story with the words read, theater, and comfortable.

assistant
Once upon a time, there was a girl named Lily. She loved to read books. One day, her mommy took her to the theater. The seats were very comfortable. Lily felt happy and cozy. She watched a play and had so much fun. After the play, she went home and read a new book. She was very happy. The end.

(c) sums, in two turns

system
You are a helpful assistant who answers arithmetic questions.

user
What is 58 plus 862?

assistant
$58 + 862 = 920$.

user
Can you add 63 and 620?

assistant
$63 + 620 = 683$.

Figure 46. Our fine-tuning conversations ask for stories and sums. Three of the training conversations, in full, each turn in its role's color: (a) a request written from a story's summary, under the system message for the story's one feature, dialogue; (b) a request for a story that uses three words, with no system message, as in a fifth of the conversations without features; (c) two sums, asked in two of five phrasings, under the system message for arithmetic. Of each kind, the shortest story of at least 60 words is shown.

The optimizer, AdamW with $\beta = (0.9, 0.95)$, weight decay 0.1 on the matrices, and clipping at 1, is pretraining's, started afresh.

Our chat model

Our chat model is the midtrained model of §12 fine-tuned on all 24,179 conversations with this recipe: 1,512 steps, 36 minutes on a CPU. On held-out requests it ends 99 answers of 100 with `<|end|>`, and gets 73% of unseen three-digit sums right. To see what each choice does, we also fine-tuned the base model four ways: as the chat model was, with the loss on every token instead of the mask, with LoRA instead of every weight, and with LoRA and the rows of the new tokens. Appendix G compares all of them, with the base and midtrained models, on the same held-out data; four results matter for what follows:

1. **Midtraining did most of the work.** Fine-tuning from the midtrained model starts where fine-tuning from the base model ends, on the loss of the assistant's tokens, and ends a little better.
2. **The mask matters little here, and more with long prompts.** Our prompts are short, a quarter of the tokens, so with the loss on every token the answers are about as good.
3. **LoRA must be able to change the new tokens.** With the token table frozen, LoRA learned the stories but almost never ended its turn: the score of `<|end|>` comes from its row of the table, which pretraining had pushed down. Training those four rows as well fixed it.
4. **A task that fine-tuning leaves out is lost.** Our fine-tuning conversations contain no summaries, and after them the chat model no longer summarizes: asked for a summary, it tells a story (ROUGE-1 0.13, against 0.27 after midtraining). Fine-tuning mixtures therefore carry midtraining's tasks along.

The running example, answered

base model: the conversation as plain text

You are a storyteller for young children.
Tell me a short story about a dog who finds a bone.
"Let's share the bone!" he said, feeling surprised.
"No, I don't think the dog's owner, mother!"
The dog felt sad and angry. "But why?" he asked.
"You can't have any bone," said the dog sadly.
The owner, who was watching him, saw him trying to do better.
The end. `<|endoftext|>`

chat model: the conversation in its template

system: You are a storyteller for young children.
user: Tell me a short story about a dog who finds a bone.
assistant: Once upon a time, there was a dog named Max. Max was a very grateful dog. He had a shiny, red tank in his yard. Max loved to play with his tank.
One day, Max went to the store with his tank. He saw a big, brown tank in place. Max thought it was a toy, but he was excited to play with it. Max followed the tank, but it was not a toy.
Max tried to carry the tank, but it was too heavy. He was so sad that he lost his tank. The tank broke into many pieces. Max was very sad and cried a lot.
But then, a dog came to help Max find the tank. Max saw the tank … `<|end|>`

Figure 47. The base model continues the document; the chat model answers and stops. The running conversation given to the base model as plain text (left, the continuation of §10) and to our chat model, midtrained and fine-tuned, in its template (right): the prompt in gray, what each model writes in blue (temperature 0.8, first sample of each), and the special token it ends with.

Given the running conversation in its template, our chat model, midtrained and then fine-tuned, answers with a story about a dog and ends its turn (Figure 47). It follows the request loosely. Its greedy answer and one of its three samples at temperature 0.8 are about a dog who finds a bone; the first sample, in the figure, is about a dog and a toy tank, and the second about a dog and a bird. On 100 held-out requests it ends its turn in 99, writes 189 tokens on average, and uses 36% of the words that the word-list requests ask for.

Following the system message

Does the chat model follow its system message? Only as far as its training data tied the two: adding *In your stories, the characters talk to each other.* to the system message raises the share of its stories with dialogue from 62% to 64%, a difference within chance, while in the fine-tuning stories that sentence went with dialogue 95% of the time against 34% (Figure 48). Our requests are written from the stories' summaries, so the request already says most of what the story

will be, and the system message rarely adds anything; Appendix G.3 explains the measurement. Models meant to follow varied system messages are fine-tuned on conversations in which the system message varies independently of the request, and decides the answer.

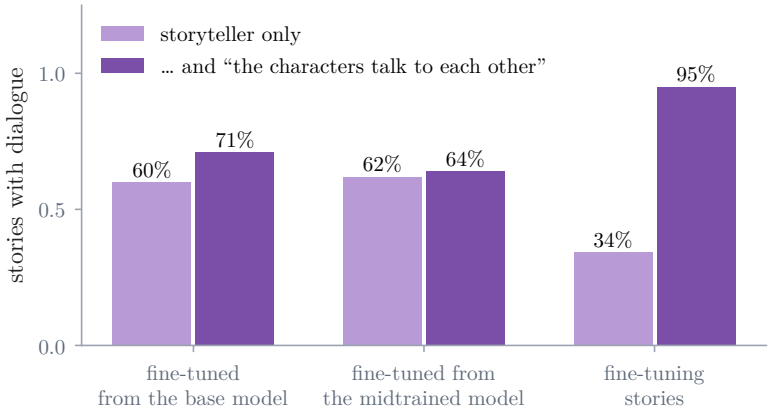


Figure 48. One sentence of the system message changes what the chat model writes, as far as its training data tied the two. The share of stories that contain dialogue (a quotation mark) among 100 answers to held-out requests, under the system message *You are a storyteller for young children.* alone (light) and with *In your stories, the characters talk to each other.* added (dark), for the chat model fine-tuned from the base model and from the midtrained model; and the same share among the fine-tuning stories without and with that sentence. Temperature 0.8.

How many conversations?

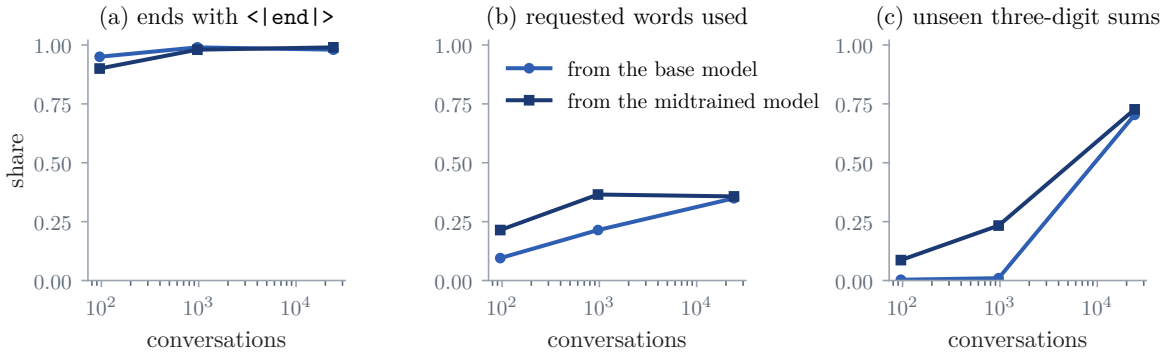


Figure 49. The format is learned from a hundred conversations; the content improves with more. Chat models fine-tuned on the first 100 conversations of the training set (96 within the length limit; read 30 times, 180 steps), the first 1,000 (969; read 3 times, 183 steps), and all 24,179 (read once, 1,512 steps), from the base model (blue) and from the midtrained model (dark blue), measured on held-out requests: (a) the share of answers that end with <|end|>; (b) the share of requested words that the story uses; (c) the accuracy on sums of two three-digit numbers that occur in neither pretraining nor fine-tuning. Logarithmic horizontal axes; single runs.

From the base model, 100 conversations, each read 30 times (180 steps), already make it end 95 of 100 held-out answers with <|end|>, about as often as 1,000 conversations read 3 times (99) and all 24,179 read once (98) (Figure 49). The content needs more: the share of requested words used rises from 10% to 21% and 35%, and unseen three-digit sums are right 70% of the time only after the full set, which holds 5,000 conversations of sums against 206 in the first thousand. From the midtrained model, 1,000 conversations are enough for the words: it then uses 37% of them, as many as the full set gives from the base model. Reading 100 conversations

30 times overfits them (Appendix G.2). This is the pattern Zhou et al. (2023) reported at a much larger scale: a 65-billion-parameter LLaMA fine-tuned on only 1,000 carefully chosen prompts and answers, LIMA, gave answers that people preferred to those of models fine-tuned on far more data, which led them to propose that a model's knowledge and abilities come from pretraining and that fine-tuning mostly teaches the format in which to use them.

14. What fine-tuning changes, and what it does not

The chat model is complete, and every stage that made it has been built (Figure 50): text to bytes and tokens (§§2–4), a corpus (§5), an optimizer and its configuration (§§6–8), a size for the budget (§9), the base model (§10), a template (§11), midtraining (§12), and fine-tuning on conversations (§13). These are the stages of ChatGPT's recipe of 2022 that come before its rewards (§1), in a model with 50,000 times fewer parameters than GPT-3.

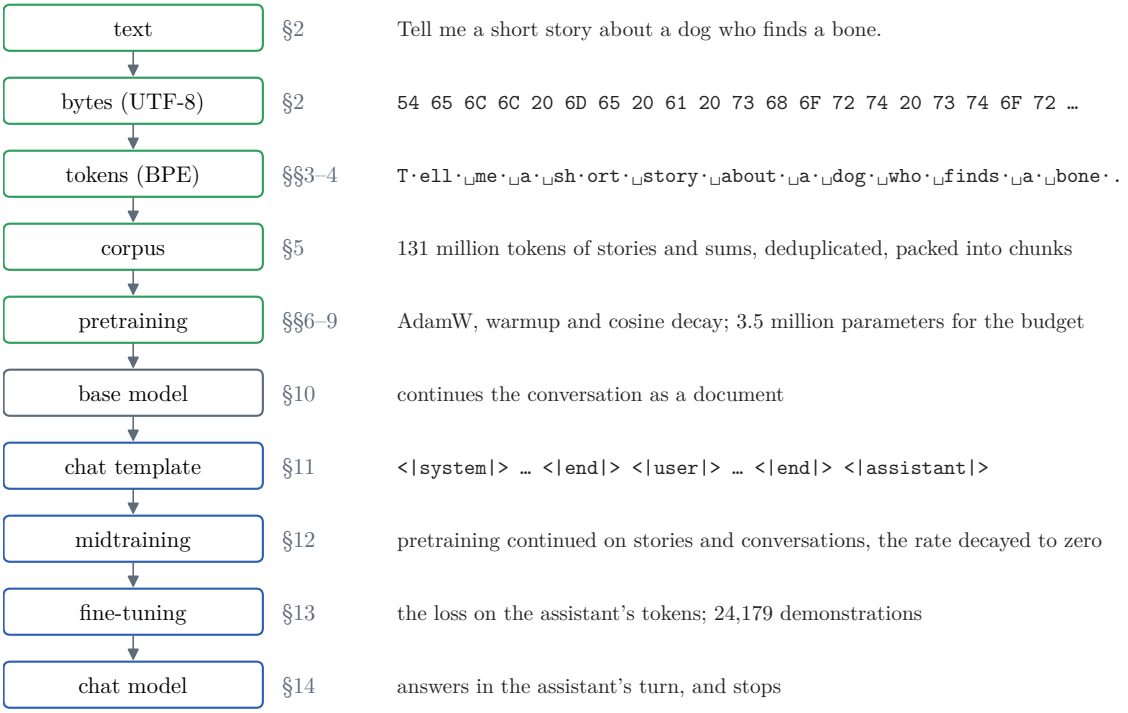


Figure 50. From text to a chat model. Every stage of this lecture, each labeled with its section, and the running conversation as each stage sees it: bytes, tokens, a document the base model continues, and a conversation the chat model answers.

What did midtraining and fine-tuning change? The format: the model writes in the assistant's turn, answers with the kind of text asked for, a story or a sum, and stops, in 99 of 100 held-out requests; the summarizing that midtraining taught is gone (§13). On sums of two three-digit numbers that occur in neither pretraining nor fine-tuning, the chat model is right 73% of the time with no examples in its prompt: about what the base model gets with one to four solved sums before the question (76% to 81%), and more than it gets without them (49%, §10). Fine-tuning put the format in the weights, not the skill. It follows the content of a request only loosely: it uses 36% of the words a request lists, and a system message that asks for dialogue changes its stories less than it changed the training stories (§13). What did they not change?

(a) a question no story answers

user
What is the capital of France?

assistant: our chat model
Once upon a time, there was a capable named Freddy. Freddy loved to play with his toy car, but he was feeling very old. Later that night, Freddy went to the toy store to buy some corn. He was very …

(b) another

user
How many legs does a spider have?

assistant: our chat model
Once upon a time, there was a little dog named Spot. Spot loved to run and jump all day in the park. One day, Spot saw a big spider in the mud. The spider was stuck in the mud.
Spot said, "Help! …

(c) a sum of a length it was shown

user
What is 12 + 7?

assistant: our chat model, right
12 + 7 = 19.

(d) a sum longer than any it was shown

user
What is 3194 + 5775?

assistant: our chat model, wrong: the sum is 8969
41 + 575 = 616.

Figure 51. Fine-tuning changes the form of the answers, not what the model knows. Our chat model (midtrained, then fine-tuned) on four requests it was not trained on, at temperature 0.8 for (a)–(c) and greedy for (d): (a, b) questions about facts that no story contains, answered with stories (cut short); (c) a sum of two small numbers, answered right; (d) a sum of two four-digit numbers, a length no stage of training showed.

1. **Knowledge comes from pretraining.** Asked about what no story contains, the chat model answers with what it knows, a story: asked for the capital of France, it tells a story about someone named Freddy, and asked how many legs a spider has, a story about a dog who sees a spider stuck in the mud (Figure 51a, b). A sum of a kind it was shown it answers right (Figure 51c). Fine-tuning on conversations about stories and sums cannot add what the base model never read, and fine-tuning on facts the base model does not know can teach a model to state things it does not know: Gekhman et al. (2024) found that fine-tuning examples with new knowledge were learned slowly and increased hallucinations. The probable-is-not-true argument of Lecture 1 (Section 7) applies to a chat model as to any language model.
2. **Skills beyond what the data showed.** Pretraining, midtraining, and fine-tuning showed sums of numbers of up to three digits. On sums of two four-digit numbers, a length it has never seen, the chat model is never right, and writes a sum of shorter numbers instead (Figure 51d). The form of the answer transfers, and the skill does not.
3. **Imitation compounds its errors.** Fine-tuning shows the model only good continuations of good prefixes. When it writes, its own tokens become its prefix, and a mistake takes it to a prefix no demonstration contained, where its next prediction is less reliable, so errors accumulate along a long answer. This is the problem of **behavior cloning**, studied for policies that imitate an expert (Ross et al., 2011), and of teacher forcing in Lecture 3 (Section 2): a model trained only on correct prefixes has never practiced recovering from its own errors.
4. **Nothing says which answer is better.** The loss rewards reproducing the demonstration's tokens. Two answers that are both plausible continuations, one correct and one not, or one helpful and one evasive, are scored only by how close they are to the demonstration, not by which is better. A signal of quality needs comparisons or rewards on the model's own answers.

That signal is what ChatGPT's last two steps added, and the subject of the next two weeks: reinforcement learning from rewards (Week 6), and its application to language models with preferences and verifiable rewards (Week 7), starting from this chat model.

15. Summary, and next week

- **After the Transformer** (§1): language models were pretrained on unlabeled text and fine-tuned (GPT, BERT), then prompted (GPT-2, GPT-3), scaled (Kaplan et al., Chinchilla), and made into assistants by supervised fine-tuning, a reward model, and reinforcement learning (InstructGPT, ChatGPT in 2022).
- **Tokens** (§§2–4): text is a sequence of Unicode code points, stored as bytes in UTF-8; byte-pair encoding merges the most frequent adjacent pair until the vocabulary is full, so frequent words are one token and anything can be spelled. The vocabulary fixes the table, $|\mathcal{V}|d$ numbers; the tokens per text scale everything else. Special tokens mark structure and cannot be typed. Models with different tokenizers are compared in bits per byte.
- **Data** (§5): filtered, deduplicated (exact hashes, and MinHash, Pr[collision] $= J$), mixed, and checked for test text; in our slice of TinyStories, 4.5% of the stories were exact duplicates and 8.0% of our test and validation stories were in it. Documents are packed into chunks.
- **Optimization** (§6, and the companion tutorial): gradient descent's step is limited by the steepest direction, so its progress along the flattest is set by the condition number; momentum averages steps; Adam scales each parameter's step by the size of its gradients, $\theta \leftarrow \theta - \eta\,\hat{m}/(\sqrt{\hat{v}}+\epsilon)$; Muon orthogonalizes each matrix's momentum, so that every direction of the matrix moves alike. On our model, each at its best learning rate for 800 steps, gradient descent reached 4.22 nats per token, momentum 3.51, Adam 2.85, and Muon 2.53.
- **The recipe** (§7): a batch of $b \times T \times k \times n$ tokens, by micro-batches, gradient accumulation, and data parallelism; warmup, then decay, with a peak that falls as models grow; AdamW's decoupled weight decay on the matrices; an initialization scaled for depth; clipping; mixed precision and 16 bytes per parameter; no dropout in pretraining; LoRA for training a low-rank change.
- **A configuration** (§8) states all of these in a few dozen lines; nanoGPT's for GPT-2 and the published runs differ mainly in the peak rate, the batch, and the tokens per parameter. Our base model: 3.5 million parameters, 49 million tokens, 0.661 bits per byte. A run costs about $6ND$ operations, at about 40% of a GPU's peak: from about $20 and 90 minutes on 8 GPUs for GPT-2's smallest model to tens of millions of GPU-hours for Llama 3's largest; fine-tuning costs a small fraction of that.
- **Scaling laws** (§9): for a budget $C \approx 6ND$, parameters and tokens grow together, $N^* \propto C^{\beta/(\alpha+\beta)}$, about 20 tokens per parameter; ours gave $N^* \propto C^{0.52}$ and about 17 tokens per parameter. Models are trained past the optimum because they are used many times.
- **The base model** (§10) continues any text; prompts condition it; induction heads copy what followed an earlier occurrence of the current token. It cannot be made an assistant by a prompt alone: it has never read a conversation.
- **Conversations as tokens** (§11): a list of messages with roles, system, user, and assistant, is rendered by a chat template into one token sequence between special tokens; the program feeds the conversation up to `<|assistant|>` and stops at `<|end|>`; the system message steers the model only as far as its training tied the two.
- **Midtraining** (§12): pretraining continued on a mixture shifted toward use, here conversations and a new task, summarizing, mixed with pretraining text, with the loss on every

token and the rate decayed to zero. Ours, 1,000 steps on half pretraining text and half conversations, taught the template (85 of 100 answers ended) and summarizing (ROUGE-1 0.27, against 0.15 for a story's first sentence), and the loss on stories ended slightly lower; sums, 2.7% of the conversation tokens, were not learned: a mixture is weighted by tokens.

- **Supervised fine-tuning** (§13): demonstrations, one conversation per row, the loss on the assistant's tokens only, a small rate, one pass. Ours, on 24,179 conversations, ends 99 of 100 answers and gets 73% of unseen three-digit sums right; after midtraining, 1,000 conversations teach it to use as many of the requested words as all of them do from the base model. Fine-tuning without summaries undid the summarizing midtraining had taught. LoRA comes close to fine-tuning every weight once the new tokens' rows are trained too, though it uses fewer of the requested words, and a system message steers the model only as far as its training data tied the two.
- **Its limits** (§14): fine-tuning teaches the format and adds no knowledge; errors compound, and nothing in imitation says which answer is better.

Next week is reinforcement learning: how a model can learn from rewards for what it does, rather than from examples of what to do. A language model writing an answer is a policy choosing actions, one token at a time; Week 6 derives the policy gradient, which turns "make well-rewarded answers more likely" into a gradient, and the methods that keep its updates stable (PPO). Week 7 applies them to the chat model of this lecture, the last two of ChatGPT's steps: rewards learned from people's preferences, direct preference optimization, and rewards that a program can check, such as the correctness of a sum.

Background reading

The basics of this lecture are developed in full elsewhere: in the chapters of the Background Notes on Machine Learning and AI, standard textbooks, and courses and videos:

Topic	Background Notes	Textbook, course, or video
Tokenization	NLP & LLMs 1	Karpathy, *Let's build the GPT Tokenizer* (video); Jurafsky and Martin, Chapter 2
Pretraining, data, and scaling	NLP & LLMs 5, 6, 7	Stanford CS336, *Language Modeling from Scratch*; Karpathy, *Let's reproduce GPT-2* (video)
Optimization for deep networks	DL 3	Goodfellow, Bengio, and Courville, Chapter 8; Prince, *Understanding Deep Learning*, Chapters 6–7
Mixed precision and training at scale	DL 11; Foundations 6	Hugging Face, *The Ultra-Scale Playbook*
In-context learning	NLP & LLMs 9	Olsson et al., *In-context Learning and Induction Heads*
Fine-tuning and LoRA	NLP & LLMs 10	Raschka, *Build a Large Language Model (From Scratch)*, Chapters 6–7

Sennrich et al. (2016) is short and introduced BPE for subword units; Hoffmann et al. (2022) is worth reading for its three methods and its IsoFLOP figures, with the correction of Besiroglu et al. (2024); Lee et al. (2022) on deduplication; Olsson et al. (2022) on induction

heads; Hu et al. (2022) on LoRA; and Section 3 of Ouyang et al. (2022) on how InstructGPT's demonstrations were collected.

References and further reading

After the Transformer (§1)

- **Andrew M. Dai and Quoc V. Le (2015), Semi-supervised Sequence Learning.** Paper.
- **Matthew E. Peters et al. (2018), Deep Contextualized Word Representations.** Paper. ELMo.
- **Jeremy Howard and Sebastian Ruder (2018), Universal Language Model Fine-tuning for Text Classification.** Paper. ULMFiT.
- **Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever (2018), Improving Language Understanding by Generative Pre-Training.** Report. GPT.
- **Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2019), BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.** Paper.
- **Alec Radford et al. (2019), Language Models are Unsupervised Multitask Learners.** Report. GPT-2.
- **Yinhan Liu et al. (2019), RoBERTa: A Robustly Optimized BERT Pretraining Approach.** Paper.
- **Colin Raffel et al. (2020), Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.** Paper. T5 and the C4 corpus.
- **Tom B. Brown et al. (2020), Language Models are Few-Shot Learners.** Paper. GPT-3; in-context learning; contamination, Appendix C.
- **Long Ouyang et al. (2022), Training Language Models to Follow Instructions with Human Feedback.** Paper. InstructGPT.
- **OpenAI (2022), Introducing ChatGPT.** Post.
- **OpenAI (2022), Aligning Language Models to Follow Instructions.** Post. InstructGPT, with GPT-3's and InstructGPT's answers to the same prompts.
- **Hugo Touvron et al. (2023a), LLaMA: Open and Efficient Foundation Language Models.** Paper.
- **Hugo Touvron et al. (2023b), Llama 2: Open Foundation and Fine-Tuned Chat Models.** Paper.
- **Llama Team, AI @ Meta (2024), The Llama 3 Herd of Models.** Paper. Data mixture, Section 3.1; training recipe, Section 3.4.
- **Meta (2024), Introducing Meta Llama 3: The Most Capable Openly Available LLM to Date.** Post. The 8- and 70-billion-parameter models still improving after 15 trillion tokens.

Tokens (§§2–4 and Appendix A)

- **Rob Pike and Ken Thompson (1993), Hello World.** Proceedings of the Winter 1993 USENIX Conference. Proceedings. UTF-8, in Plan 9; the full title repeats the greeting in Greek and in Japanese.

- **The Unicode Consortium (2026), The Unicode Standard, Version 18.0.** Release. 172,808 characters.
- **W3Techs (2026), Usage Statistics of Character Encodings for Websites.** Survey. UTF-8 on 99.1% of the websites whose encoding is known.
- **Philip Gage (1994), A New Algorithm for Data Compression.** *The C Users Journal* 12(2). Article. Byte-pair encoding, as compression.
- **Rico Sennrich, Barry Haddow, and Alexandra Birch (2016), Neural Machine Translation of Rare Words with Subword Units.** Paper.
- **László Kozma and Johannes Voderholzer (2024), Theoretical Analysis of Byte-Pair Encoding.** Paper.
- **Philip Whittington, Gregor Bachmann, and Tiago Pimentel (2024), Tokenisation is NP-Complete.** Paper.
- **Jessica Rumbelow and Matthew Watkins (2023), SolidGoldMagikarp (plus, prompt generation).** Post.
- **Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi (2023), Language Model Tokenizers Introduce Unfairness Between Languages.** Paper.
- **gpt-tokenizer (2025), version 2.9.0.** Code. OpenAI's published vocabularies, r50k_base (GPT-2 and GPT-3) and o200k_base (GPT-4o), used for the tokens of §4.
- **Mike Schuster and Kaisuke Nakajima (2012), Japanese and Korean Voice Search.** Paper. WordPiece.
- **Yonghui Wu et al. (2016), Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation.** Paper.
- **Taku Kudo (2018), Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates.** Paper. The unigram model.
- **Taku Kudo and John Richardson (2018), SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing.** Paper.
- **Linting Xue et al. (2022), ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models.** Paper.
- **Artidoro Pagnoni et al. (2024), Byte Latent Transformer: Patches Scale Better Than Tokens.** Paper.

Data (§5 and Appendix C)

- **Andrei Z. Broder (1997), On the Resemblance and Containment of Documents.** Paper. MinHash.
- **Katherine Lee et al. (2022), Deduplicating Training Data Makes Language Models Better.** Paper.
- **Guilherme Penedo et al. (2023), The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only.** Paper.
- **Luca Soldaini et al. (2024), Dolma: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research.** Paper.
- **Guilherme Penedo et al. (2024), The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale.** Paper. FineWeb and FineWeb-Edu.

- **Ronen Eldan and Yuanzhi Li (2023), TinyStories: How Small Can Language Models Be and Still Speak Coherent English?** Paper. The corpus of our models, and TinyStories-Instruct.

Optimization, the training recipe, and configurations (§§6–8 and Appendix B)

The optimizers' derivations, with their papers, are in the companion tutorial, *Optimizers for Deep Learning*.

- **Boris T. Polyak (1964), Some Methods of Speeding up the Convergence of Iteration Methods.** Paper. Momentum.
- **Tijmen Tieleman and Geoffrey Hinton (2012), Lecture 6.5: Divide the Gradient by a Running Average of Its Recent Magnitude.** *Neural Networks for Machine Learning*, Coursera. Slides. RMSProp.
- **Diederik P. Kingma and Jimmy Ba (2015), Adam: A Method for Stochastic Optimization.** Paper.
- **Sam McCandlish, Jared Kaplan, Dario Amodei, and the OpenAI Dota Team (2018), An Empirical Model of Large-Batch Training.** Paper. The critical batch size.
- **Sadhika Malladi, Kaifeng Lyu, Abhishek Panigrahi, and Sanjeev Arora (2022), On the SDEs and Scaling Rules for Adaptive Gradient Algorithms.** Paper. The learning rate of Adam against the batch size.
- **Priya Goyal et al. (2017), Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour.** Paper. Warmup.
- **Ilya Loshchilov and Frank Hutter (2017), SGDR: Stochastic Gradient Descent with Warm Restarts.** Paper. The cosine schedule.
- **Shengding Hu et al. (2024), MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies.** Paper. Warmup–stable–decay.
- **Ilya Loshchilov and Frank Hutter (2019), Decoupled Weight Decay Regularization.** Paper. AdamW.
- **Francesco D'Angelo, Maksym Andriushchenko, Aditya Varre, and Nicolas Flammarion (2024), Why Do We Need Weight Decay in Modern Deep Learning?** Paper.
- **Mitchell Wortsman et al. (2024), Small-scale Proxies for Large-scale Transformer Training Instabilities.** Paper. Adam's ϵ at scale.
- **Aakanksha Chowdhery et al. (2023), PaLM: Scaling Language Modeling with Pathways.** Paper. Loss spikes; training without dropout.
- **Paulius Micikevicius et al. (2018), Mixed Precision Training.** Paper.
- **Xavier Glorot and Yoshua Bengio (2010), Understanding the Difficulty of Training Deep Feedforward Neural Networks.** Paper. Xavier initialization.
- **Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun (2015), Delving Deep into Rectifiers.** Paper. Kaiming initialization.
- **Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He (2020), ZeRO: Memory Optimizations Toward Training Trillion Parameter Models.** Paper. 16 bytes per parameter; sharding the optimizer's state.

- **Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov (2014), Dropout: A Simple Way to Prevent Neural Networks from Overfitting.** Paper.
- **Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna (2016), Rethinking the Inception Architecture for Computer Vision.** Paper. Label smoothing, Section 7.
- **Edward J. Hu et al. (2022), LoRA: Low-Rank Adaptation of Large Language Models.** Paper.
- **Armen Aghajanyan, Sonal Gupta, and Luke Zettlemoyer (2021), Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning.** Paper.
- **Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer (2023), QLoRA: Efficient Finetuning of Quantized LLMs.** Paper.
- **Andrej Karpathy (2024c), Reproducing GPT-2 (124M) in llm.c in 90 Minutes for $20.** Discussion.
- **Andrej Karpathy (2024d), Reproducing GPT-2 (1.6B) on One 8×H100 Node in 24 Hours.** Discussion.
- **DeepSeek-AI (2024), DeepSeek-V3 Technical Report.** Paper. GPU-hours and cost of the final run, Table 1.
- **Meta (July 2024), Llama 3.1 Model Card.** Card. GPU-hours of each model.
- **Epoch AI (2026), Notable AI Models.** Database. Estimates of training compute and cost.
- **Ben Cottier et al. (2024), The Rising Costs of Training Frontier AI Models.** Paper.
- **Xiangning Chen et al. (2023), Symbolic Discovery of Optimization Algorithms.** Paper. Lion.
- **Vineet Gupta, Tomer Koren, and Yoram Singer (2018), Shampoo: Preconditioned Stochastic Tensor Optimization.** Paper.
- **Keller Jordan et al. (2024), Muon: An Optimizer for Hidden Layers in Neural Networks.** Post.
- **Jingyuan Liu et al. (2025a), Muon is Scalable for LLM Training.** Paper. Moonlight.
- **Kimi Team (2025), Kimi K2: Open Agentic Intelligence.** Paper. Muon at the scale of a trillion parameters.
- **Greg Yang et al. (2021), Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer.** Paper. µP.

Scale and systems (§9 and Appendix D)

- **Jared Kaplan et al. (2020), Scaling Laws for Neural Language Models.** Paper.
- **Jordan Hoffmann et al. (2022), Training Compute-Optimal Large Language Models.** Paper. Chinchilla.
- **Tamay Besiroglu, Ege Erdil, Matthew Barnett, and Josh You (2024), Chinchilla Scaling: A Replication Attempt.** Paper.
- **Tim Pearce and Jinyeop Song (2024), Reconciling Kaplan and Chinchilla Scaling Laws.** Paper.

- **Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt, and Yair Carmon (2024), Resolving Discrepancies in Compute-Optimal Scaling of Language Models.** Paper.
- **Nikhil Sardana, Jacob Portes, Sasha Doubov, and Jonathan Frankle (2024), Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws.** Paper.
- **Niklas Muennighoff et al. (2023), Scaling Data-Constrained Language Models.** Paper.
- **Pablo Villalobos et al. (2024), Will We Run Out of Data? Limits of LLM Scaling Based on Human-Generated Data.** Paper.
- **Mohammad Shoeybi et al. (2019), Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism.** Paper. Tensor parallelism.
- **Yanping Huang et al. (2019), GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism.** Paper.
- **Vijay Anand Korthikanti et al. (2023), Reducing Activation Recomputation in Large Transformer Models.** Paper. The memory of activations.

Base models and in-context learning (§10 and Appendix E)

- **Nelson Elhage et al. (2021), A Mathematical Framework for Transformer Circuits.** Article.
- **Catherine Olsson et al. (2022), In-context Learning and Induction Heads.** Article.
- **Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma (2022), An Explanation of In-context Learning as Implicit Bayesian Inference.** Paper.
- **Shivam Garg, Dimitris Tsipras, Percy Liang, and Gregory Valiant (2022), What Can Transformers Learn In-Context? A Case Study of Simple Function Classes.** Paper.
- **Johannes von Oswald et al. (2023), Transformers Learn In-Context by Gradient Descent.** Paper.
- **Jason Wei et al. (2022b), Emergent Abilities of Large Language Models.** Paper.
- **Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo (2023), Are Emergent Abilities of Large Language Models a Mirage?** Paper.

Midtraining (§12)

- **Team OLMo (2024), 2 OLMo 2 Furious.** Paper. Mid-training on Dolmino Mix 1124; no weight decay on the embeddings.
- **Emmy Liu, Graham Neubig, and Chenyan Xiong (2025b), Midtraining Bridges Pretraining and Posttraining Distributions.** Paper.
- **Andrej Karpathy (2025), nanochat.** Code. Tokenizer, pretraining, midtraining (until January 2026), fine-tuning, and reinforcement learning in one small codebase.
- **Chin-Yew Lin (2004), ROUGE: A Package for Automatic Evaluation of Summaries.** Paper.

Chat models and fine-tuning (§§11, 13–14 and Appendices F and G)

- **Amanda Askell et al. (2021), A General Language Assistant as a Laboratory for Alignment.** Paper. A base model made an assistant by a prompt of example conversations.
- **Qwen Team (2024), Qwen2.5-7B-Instruct.** Model card. ChatML, and a default system message in the template.
- **Anthropic (2024), System Prompts.** Release notes. The system prompts of Claude's applications.
- **Kai Greshake et al. (2023), Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.** Paper.
- **Eric Wallace et al. (2024), The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions.** Paper.
- **Jason Wei et al. (2022a), Finetuned Language Models Are Zero-Shot Learners.** Paper. FLAN.
- **Victor Sanh et al. (2022), Multitask Prompted Training Enables Zero-Shot Task Generalization.** Paper. T0.
- **Yizhong Wang et al. (2023), Self-Instruct: Aligning Language Models with Self-Generated Instructions.** Paper.
- **Rohan Taori et al. (2023), Alpaca: A Strong, Replicable Instruction-Following Model.** Post.
- **Chunting Zhou et al. (2023), LIMA: Less Is More for Alignment.** Paper.
- **Zorik Gekhman et al. (2024), Does Fine-Tuning LLMs on New Knowledge Encourage Hallucinations?** Paper.
- **Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell (2011), A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning.** Paper. Behavior cloning and its compounding errors.

Code and explanations

- **Andrej Karpathy (2022), nanoGPT.** Code. The configuration of §8 is `config/train_gpt2.py`.
- **Andrej Karpathy (2024a), Let's build the GPT Tokenizer.** Video. And the code, minbpe.
- **Andrej Karpathy (2024b), Let's reproduce GPT-2 (124M).** Video. And the code, build-nanogpt.
- **Stanford CS336 (2025), Language Modeling from Scratch.** Course.
- **Hugging Face (2025), The Ultra-Scale Playbook: Training LLMs on GPU Clusters.** Book.
- **Sebastian Raschka (2024), Build a Large Language Model (From Scratch).** Manning. Book. Chapters 6–7 fine-tune for classification and for instructions.

Appendix A: Tokenizers in more detail (optional)

A.1 WordPiece and the unigram model

BPE merges the most frequent pair. Two other algorithms choose pieces differently.

WordPiece (Schuster and Nakajima, 2012; Wu et al., 2016), the tokenizer of BERT, also merges pairs, but chooses the pair that most increases the likelihood of the training text under a model in which tokens are independent: the pair (a, b) with the largest $\text{count}(ab)/(\text{count}(a)\,\text{count}(b))$, which favors pairs whose parts rarely occur apart. Pieces inside a word are marked with a prefix (`##ing`), and encoding takes the longest piece of the vocabulary that matches at each point, left to right.

The **unigram model** (Kudo, 2018) works the other way, by removal. It starts from a large vocabulary of candidate pieces, models a text as a sequence of independent pieces with probabilities $p(x)$, and fits the probabilities by expectation maximization, summing over all the ways a word can be split. It then removes the pieces whose removal lowers the likelihood least, and repeats until the vocabulary has the size wanted. A word is encoded by its most probable split (the Viterbi algorithm), or, in training, by a split sampled from the model, which exposes the model to several segmentations of the same word. **SentencePiece** (Kudo and Richardson, 2018) is a library that implements BPE and the unigram model on raw text, treating the space as an ordinary symbol, so that no language-specific pre-tokenization is needed; Llama 1 and 2 and T5 used it.

A.2 Byte fallback, and tokenizers without tokens

A tokenizer over Unicode characters, like SentencePiece's, can still meet a character it never saw. **Byte fallback** writes such a character as its UTF-8 bytes, which are kept in the vocabulary as 256 extra tokens, so that nothing is unknown, the property byte-level BPE has by construction.

Models that read bytes directly avoid tokenizers altogether, at the cost of the length of §2. ByT5 (Xue et al., 2022) did so with a deeper encoder; the Byte Latent Transformer (Pagnoni et al., 2024) groups bytes into patches of varying length, longer where the next byte is easy to predict, so that the large model runs once per patch.

A.3 An efficient implementation

Training as described in §3 recounts all pairs after every merge. Our implementation counts each distinct pre-token once, weighted by its count (our tokenizer's training text, about 90 million bytes, has 25,057 distinct pre-tokens), keeps the count of every pair and the set of pre-tokens that contain it, and after each merge updates only the pre-tokens that contained the merged pair. The 3,832 merges of our tokenizer take about 20 seconds in plain Python. Encoding keeps a cache from pre-token to tokens, since the same words recur.

A.4 How good greedy merging is

Choosing the merges that shorten a corpus most is NP-hard, and hard even to approximate closely; greedy BPE (§3) is guaranteed at least a third of the best possible saving (Kozma and Voderholzer, 2024). Choosing the best vocabulary directly is NP-hard too (Whittington et al., 2024).

Appendix B: Training details (optional)

The derivations of the optimizers, from gradient descent to Muon, are in the companion tutorial, *Optimizers for Deep Learning*. This appendix holds three details of the training recipe: how the weights start (§7), how a learning rate tuned on a narrow model carries over to a wide one (§7), and a configuration that trains with Muon (§8).

B.1 Initialization

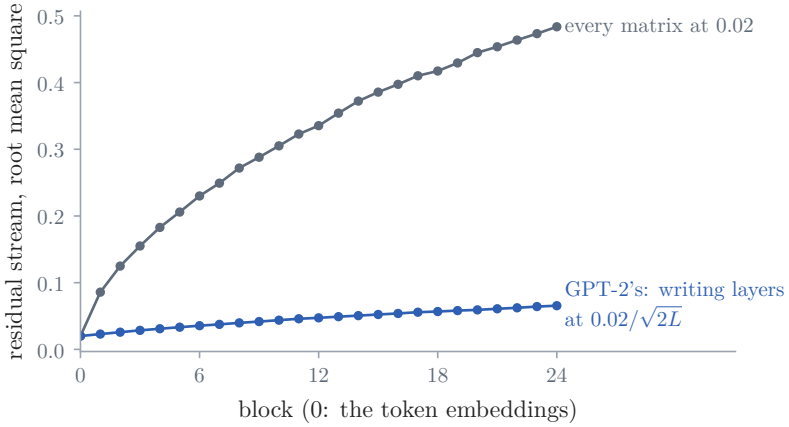


Figure 52. Scaling down the layers that write into the residual stream keeps the stream's size nearly independent of depth. The root mean square of the residual stream after each block of a 24-block model of our base model's width at initialization, on 8 chunks of the training text: with the writing layers of each block at $0.02/\sqrt{2L}$ (blue) and with every matrix at 0.02 (gray).

The weights a run starts from must keep signals and gradients at a usable size through every layer: weights too large make activations and gradients grow from layer to layer, and weights too small make them vanish. A residual network adds a further problem. Its stream is the sum of the embedding and of the outputs of $2L$ sublayers, an attention and an MLP in each of L blocks, and at initialization these outputs are nearly independent, so their variances add and the stream grows with depth. GPT drew its weights with a standard deviation of 0.02 (Radford et al., 2018), and GPT-2 scaled down the weights of the layers that write into the stream by $1/\sqrt{N}$, for N such layers (Radford et al., 2019; Lecture 4 (Section 8)). In the form that nanoGPT's code gives them, which our models use, they are three rules:

1. every weight matrix, the token table included, is drawn from a normal distribution with standard deviation 0.02;
2. biases start at zero, and the normalizations' scales at one and shifts at zero;
3. the two matrices of each block that write into the stream, attention's output and the MLP's second matrix, use $0.02/\sqrt{2L}$ instead, so that the $2L$ contributions together add the variance of one.

In a 24-block model of our width, the stream ends 3.3 times the size of the token embeddings with these rules, and 24 times without the third (Figure 52). Configurations list the standard deviation as a field (`init_std: 0.02`), some scale it with the width instead (Appendix B.2), and the scaled residual layers are part of the model's code.

How this relates to Xavier and Kaiming initialization. For one linear layer $\mathbf{y} = \mathbf{x}W$ with d_{in} inputs of variance 1 and independent weights of variance σ^2, each output has variance $d_{\text{in}}\sigma^2$. Keeping it at 1 needs $\sigma = 1/\sqrt{d_{\text{in}}}$, the rule of Glorot and Bengio (2010) (who average

the input and output widths, so that the backward pass is kept too), and $\sigma = \sqrt{2/d_{\text{in}}}$ after a ReLU, which zeroes half its inputs, the rule of He et al. (2015). GPT's fixed 0.02 is of that order for its widths ($1/\sqrt{768} \approx 0.036$), and the layer normalization before every sublayer resets the scale of what each sublayer reads, so the exact value matters less than in a network without normalization. What normalization does not fix is the sum in the residual stream, which the third rule addresses.

B.2 Transferring the learning rate across widths

The best learning rate of a model depends on its width, so a rate tuned on a small model is wrong for a large one. The **maximal update parametrization**, µP (Yang et al., 2021), scales the initialization and the learning rate of each kind of matrix with the width so that the size of every layer's update stays the same as the model widens; the best rate of a narrow model is then the best rate of a wide one, and hyperparameters can be tuned at small scale and transferred.

B.3 A configuration with Muon: nanochat

nanochat (Karpathy, 2025) is a small, complete implementation of the whole pipeline of this lecture. Its first release (October 2025) trained a model of 20 blocks of width 1,280 (561 million parameters, with normalizations that have no parameters and no biases) with three learning rates: `matrix_lr = 0.02` for Muon on the matrices inside the blocks (§6), and two for AdamW, `embedding_lr = 0.2` for the token table and `unembedding_lr = 0.004` for its separate output layer, both multiplied in the code by $1/\sqrt{d/768}$ for a width d, a rule of the kind of §7's. Its AdamW used $\beta = (0.8, 0.95)$, $\epsilon = 10^{-10}$, and no weight decay. It had no warmup, a constant rate and then a linear decay to zero over the last 20% of the steps (warmup–stable–decay, §7), `total_batch_size = 524288` tokens, and, instead of a number of steps, `target_param_data_ratio = 20`: the run reads 20 tokens per parameter, the rule of §9, and the number of steps follows.

Appendix C: Data at scale (optional)

The large open corpora follow the steps of §5 with more of each (C4, Raffel et al., 2020; RefinedWeb, Penedo et al., 2023; Dolma, Soldaini et al., 2024; FineWeb, Penedo et al., 2024):

1. **Extraction:** the text of each page from its HTML, by a library that drops menus and boilerplate (trafilatura, for RefinedWeb and FineWeb), not the archive's own text extraction.
2. **Language identification:** a classifier on character n-grams (fastText) keeps pages in the wanted languages above a confidence threshold.
3. **Quality rules:** thresholds on the length of the document and of its words, the fraction of symbols, of lines ending in punctuation, of repeated lines and n-grams, and of words in a stop-word list; lists of URLs to exclude.
4. **Deduplication:** exact (hashes of lines or documents), then MinHash over n-gram shingles with banding (§5 and below), either within each crawl or across all of them. FineWeb found that deduplicating each crawl separately gave better models than deduplicating across crawls, which removed mostly good, often-copied text.
5. **Classifiers of quality:** a small model trained to tell reference text (Wikipedia's citations, educational pages, or a large model's ratings) from random web text, used to keep the top part.
6. **Personal information and toxicity:** e-mail addresses and IP addresses masked; pages scored as toxic removed or down-weighted.

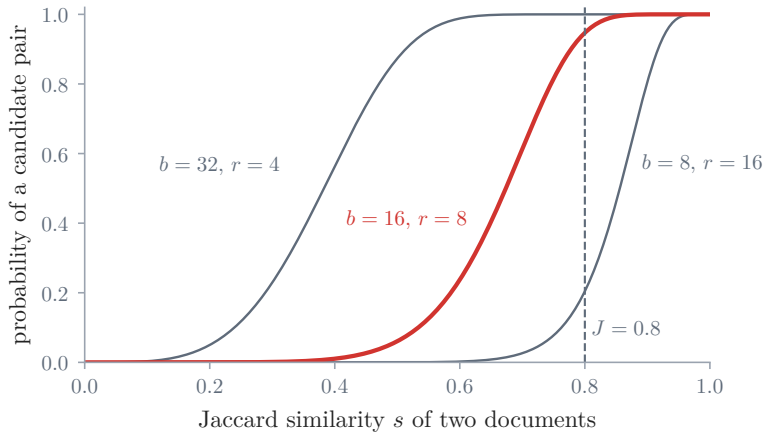


Figure 53. Banding turns similarity into a threshold. The probability that two documents of Jaccard similarity s agree on at least one of b bands of r MinHash values, $1-(1-s^r)^b$, for our $b=16$, $r=8$ (red) and for two other ways of cutting 128 values (gray). The dashed line marks our threshold for a duplicate, $J \geq 0.8$.

Banding. MinHash signatures (§5) still have to be compared in pairs. Banding, a form of locality-sensitive hashing, finds the similar pairs without comparing all of them: cut the 128 numbers into $b=16$ bands of $r=8$, and call two documents candidates if they agree on all of a band, which hashing the bands finds in one pass. Two documents of similarity s agree on a given band with probability s^r, so they become candidates with probability

$$\Pr[\text{candidate}] = 1-(1-s^r)^b,$$

an S-shaped curve that is near 0 for dissimilar documents and near 1 for similar ones, with its threshold near $(1/b)^{1/r} \approx 0.71$ (Figure 53). Each candidate pair is then checked by its exact Jaccard similarity.

MinHash's guarantees are exact for the estimator of one signature position (§5); with k positions the estimate of J has standard deviation $\sqrt{J(1-J)/k}$, 0.035 for $J=0.8$ and $k=128$.

Appendix D: Scaling and systems (optional)

D.1 The compute-optimal allocation

Minimize $E + AN^{-\alpha} + BD^{-\beta}$ subject to $6ND = C$. Substituting $D = C/(6N)$ gives a function of N alone, $E + AN^{-\alpha} + B(6N/C)^{\beta}$, whose derivative is zero when $\alpha AN^{-\alpha} = \beta B(6N/C)^{\beta}$, that is

$$N^* = G\left(\frac{C}{6}\right)^{\frac{\beta}{\alpha+\beta}}, \qquad D^* = G^{-1}\left(\frac{C}{6}\right)^{\frac{\alpha}{\alpha+\beta}}, \qquad G = \left(\frac{\alpha A}{\beta B}\right)^{\frac{1}{\alpha+\beta}}.$$

Hoffmann et al. estimated the exponents three ways: (1) by training models of several sizes, each with several numbers of tokens, and taking the lowest loss at each budget; (2) by **IsoFLOP profiles**, fixing nine budgets and fitting a parabola to the loss against $\log N$ at each, as in Figure 34; (3) by fitting $\mathcal{L}(N, D)$ above to all runs. The three gave $N^* \propto C^{0.50}$, $C^{0.49}$, and $C^{0.46}$. The third fit implied far more tokens per parameter at the optimum, about 70 by Besiroglu et al.'s reading, against about 20 for the first two; Besiroglu et al. (2024) refitted it to the runs, read from Hoffmann et al.'s figures, and obtained $\mathcal{L} = 1.82 + 482/N^{0.35} + 2085/D^{0.37}$, which puts the optimum near 20 tokens per parameter, as the other two methods do (Figure 33).

D.2 Kaplan against Chinchilla

Kaplan et al.'s runs differed from Hoffmann et al.'s in several ways: parameters counted without the token table and the position table, which make up most of a small model; one learning-rate schedule for runs of every length, so that shorter runs ended before their decay, the difference Hoffmann et al. suspected; and models of up to about a billion parameters. Pearce and Song (2024) showed that counting all parameters at small scale moves the exponent most of the way to Chinchilla's, and that one fixed learning rate per model size suffices to measure the exponents. Porian et al. (2024) reproduced both laws on the same data: correcting the computation counted for the output layer, the length of the warmup, and tuning the optimizer at each scale turned Kaplan et al.'s law into Chinchilla's, with a constant learning rate, so the decay Hoffmann et al. suspected was not essential.

D.3 Data-constrained and inference-aware scaling

When unique data is limited, Muennighoff et al. (2023) fitted the value of repeated tokens as decaying with the number of passes: up to about four passes, a repeated token is worth nearly a new one; beyond about 16, the value of repeating falls quickly. Sardana et al. (2024) add $2N$ operations per token of expected use to the cost and minimize the total; for a model that will generate trillions of tokens, the optimum is a smaller model trained on many more tokens.

D.4 The memory of a training run

For a model of 7 billion parameters in mixed precision with AdamW (§7): 14 GB of 16-bit weights, 14 GB of 16-bit gradients, and 84 GB of fp32 master weights and moments, 112 GB in all. The activations kept for the backward pass grow with the batch, the context, and the depth: for each token and block, the inputs of every matrix product, normalization, and nonlinearity, about $34d$ bytes in 16 bits when the attention weights are not stored (Korthikanti et al., 2023), about 18 GB per sequence of 4,096 tokens at $d = 4{,}096$ and $L = 32$. Recomputing activations in the backward pass instead of storing them (**activation checkpointing**) trades about a third more computation for most of this memory.

D.5 Splitting a run across accelerators

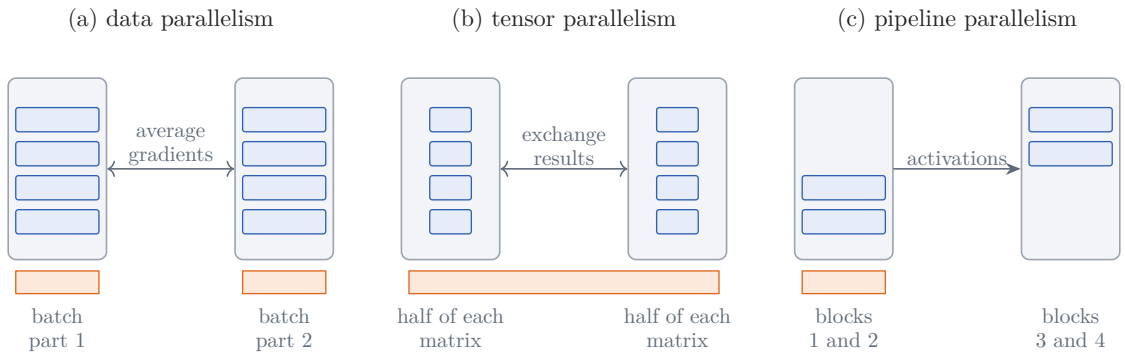


Figure 54. Three ways to split a training run across accelerators. Two accelerators (gray) in each case, holding blocks or halves of their matrices (blue) and processing a batch (orange). Data parallelism splits the batch (each copy of the model gets different chunks), tensor parallelism splits each weight matrix, and pipeline parallelism splits the blocks. Large runs combine all three.

Large runs split the work across thousands of accelerators in three ways at once (Figure 54):

1. **Data parallelism:** each group of accelerators holds the whole model and processes a different part of the batch; the gradients are averaged across groups at each step. Sharding the optimizer's state, and the gradients and weights too, across the groups removes their copies (Rajbhandari et al., 2020).
2. **Tensor parallelism:** each weight matrix is split across the accelerators of a group, which compute their parts of every product and exchange the results within each layer (Shoeybi et al., 2019).
3. **Pipeline parallelism:** the blocks are divided among groups in order, and the batch flows through them in pieces, so that the groups work at the same time on different pieces (Huang et al., 2019).

Appendix E: In-context learning (optional)

E.1 Induction heads in our base model

Copying a name shows the mechanism of §10 in our base model. Take a story that introduces a name the model has never seen, with 100 invented names of five letters, each written in our tokens as two to six pieces, most often three or four (␣$F \cdot u \cdot z \cdot ak$). The first time, the model cannot predict the name: its log loss, summed over the name's pieces, is 25.7 nats on average; the second and third times it costs 5.2: 1.7 nats for its first piece and 3.5 for the others together (Figure 55a). A familiar name costs 5.5 nats the first time and 1.6 the second. Head 4.4, the fourth head of block 4, attends most clearly this way (Figure 55c): from the first piece of the name's second occurrence, it puts on average 0.84 of its weight on the piece that followed the same piece the first time, and several heads of later blocks do the same (head 4.5, 0.77; head 5.2, 0.61); head 2.1, in block 2, puts 0.61 of its weight on the position just before each token. These are the two heads of Figure 37 (heads are numbered by block and by position within the block, from 1). The copying forms during training (Figure 55b): the name's second occurrence costs 28.7 nats at step 0, as much as its first, 15.4 at step 1,000, and 5.2 at the end, while the first occurrence stays near 25. The model has learned to copy, not the names.

Olsson et al. tested copying with a sequence of random tokens given twice. Our model, which has read only children's stories, copies such a sequence poorly: its loss falls only from 12.7 nats per token on the first copy to 11.6 on the second, and head 4.4 puts 0.09 of its weight on the induction position. On text unlike any story, its expectations outweigh its context. Copying is the part of in-context learning whose mechanism is known; other accounts of in-context learning follow.

E.2 In-context learning as inference

Xie et al. (2022) model pretraining text as generated by a mixture of latent "concepts", each a process that produces documents of one kind. A model that predicts such text well must, from the beginning of a document, infer which concept produced it. A prompt of a few examples of a task is then evidence about the concept, and predicting the answer amounts to Bayesian inference of the concept from the examples. They showed that in-context learning emerges from pretraining on such a mixture even though the examples of a prompt, joined one after another, look unlike any training document.

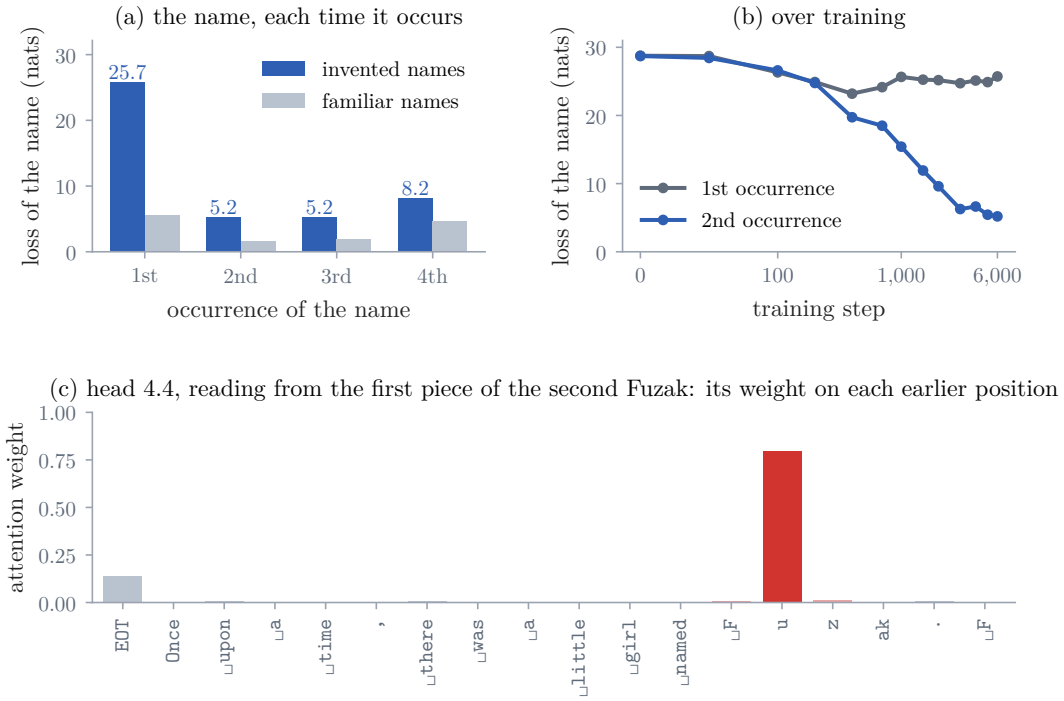


Figure 55. Our base model copies a name it has just read, through an induction head. A four-sentence story that introduces an invented name, *Once upon a time, there was a little girl named Fuzak. Fuzak loved to play in the park. One day, Fuzak saw a big dog. Fuzak was happy.*, with 100 invented names. (a) The loss of the name's pieces, summed, at each of its four occurrences, averaged over the names (blue), and for five familiar names such as *Lily* (gray). (b) The loss of the name at its first and second occurrence for the saved models during training. (c) The attention weights of head 4.4 from the first piece of the second *Fuzak* to every earlier position; red marks the piece that followed the same piece the first time.

E.3 Transformers that learn a function from examples

Garg et al. (2022) trained Transformers on prompts of input–output pairs, $(\mathbf{x}_1, y_1)$, …, $(\mathbf{x}_k, y_k)$, followed by a new input $\mathbf{x}_{k+1}$, each prompt from a different linear function $y = \mathbf{w}^\top \mathbf{x}$, and found that the trained models predict y_{k+1} about as well as least squares on the k examples, and do the same for sparse linear functions, decision trees, and small networks. von Oswald et al. (2023) showed one way a Transformer can do this: a single layer of linear attention (attention without the softmax) with suitable weights computes exactly one step of gradient descent on the squared loss of the examples, from $\mathbf{w} = 0$, and trained linear-attention models were found to match that construction.

E.4 Emergent abilities

Some abilities, measured by accuracy on a task, appear to switch on suddenly at some model size (Wei et al., 2022b). Schaeffer et al. (2023) found that much of the suddenness comes from the measure: an exact-match accuracy over a long answer is near zero until every token is likely enough, while a measure that gives partial credit, such as the edit distance between the model's answer and the correct one, improves smoothly on the same models. Olsson et al. found that induction heads form abruptly within one training run, at a point visible in the loss; in our smaller run the copying of names forms gradually, between steps 200 and 3,000 (Figure 55b).

Appendix F: Chat templates in use (optional)

The running conversation in Llama 3's template, whose markers §11 describes (a line too long for the page continues, indented):

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

You are a storyteller for young
    children.<|eot_id|><|start_header_id|>user<|end_header_id|>

Tell me a short story about a dog who finds a
    bone.<|eot_id|><|start_header_id|>assistant<|end_header_id|>
```

Templates of models that call tools add roles for the tool's output and special tokens around a call, which the program running the model parses, executes, and answers in a new turn (Week 9).

Appendix G: Our fine-tuning runs in detail (optional)

G.1 Seven models on the same held-out data

The table compares seven models, each measured in the chat template on the same held-out data. Its columns are: two losses, in nats per token, on the assistant's tokens of 300 held-out story conversations and on the 500 validation stories, which measures what each model keeps of pretraining; the share of 100 held-out requests whose answer ends with `<|end|>`; the share of the words that the word-list requests ask for which the stories use; the share of 300 unseen sums of two three-digit numbers answered right (greedy); and the ROUGE-1 of summaries of 100 held-out stories (§12). Each fine-tuning run is a single run.

model	losses: answers, stories	ends its turn	requested words used	unseen sums right	summaries (ROUGE-1)
base model	1.90, 1.818	0%	5%	0%	0.08
midtrained	1.63, 1.813	85%	26%	7%	0.27
base, fine-tuned	1.63, 1.879	98%	35%	70%	0.11
base, fine-tuned, loss on every token	1.64, 1.925	100%	38%	72%	0.12
base, fine-tuned with LoRA	1.68, 1.853	2%	33%	0%	0.10
base, fine-tuned with LoRA and the four rows	1.64, 1.853	100%	25%	75%	0.09
midtrained, fine-tuned (our chat model)	1.61, 1.862	99%	36%	73%	0.13

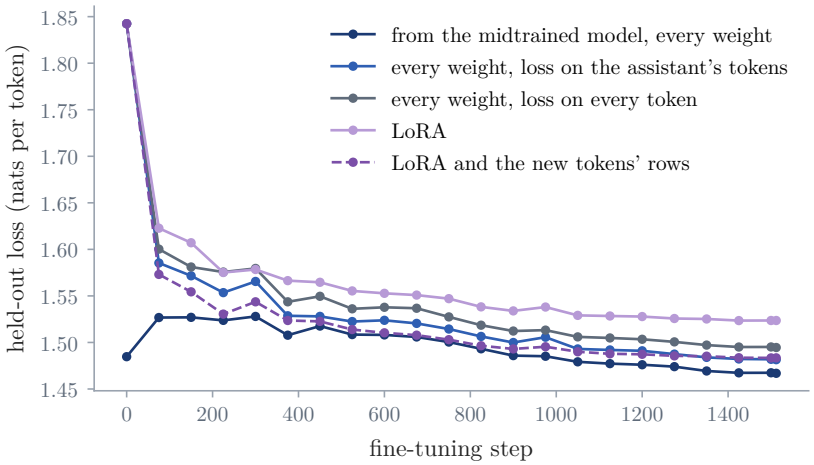


Figure 56. Most of what fine-tuning gains on the assistant's turns comes in the first hundred steps, and midtraining has gained most of it already. The loss on the assistant's tokens of 150 held-out conversations, the set measured during training (nats per token; step 0 is the model fine-tuning starts from; the table's 300 story conversations, a separate set, give losses 0.06 to 0.15 higher, and the curves end between 1.47 and 1.52), for every weight trained from the midtrained model (dark blue) and from the base model (blue), from the base model with the loss on every token (gray), with LoRA alone (light purple), and with LoRA and the rows of the four new tokens (purple, dashed). Single runs.

G.2 How many conversations, and three comparisons

How many conversations, in detail. After 1,000 conversations from the base model, unseen three-digit sums are still almost never right (1%, and 0.3% after 100): the model writes a sum in the right form, but with a wrong operand and a wrong total, *237 + 282 = 529.* for *237 + 682.* From the midtrained model, which already ends 85 answers of 100 and uses 26% of the words, 1,000 conversations make it end 98, use 37% of the requested words, and get 23% of the sums right. Reading 100 conversations 30 times overfits them from either start: the loss on the assistant's tokens of the held-out stories rises to 2.42 nats per token, from 1.90 before fine-tuning the base model and from 1.63 before fine-tuning the midtrained one.

From the midtrained model against from the base model. With the same data and recipe, fine-tuning the midtrained model ends a little better than fine-tuning the base model: a held-out loss on the assistant's tokens of 1.61 nats per token against 1.63, 99% of answers ended against 98%, 36% of requested words used against 35%, and 73% of the sums right against 70%, in single runs. The larger difference is where fine-tuning starts. From the midtrained model the loss on the assistant's tokens starts where fine-tuning the base model ends (Figure 56), and 1,000 conversations teach the model to use as many of the requested words as all 24,179 do from the base model (§13). Both lose a little of the stories, 1.862 and 1.879 nats per token on the validation stories against the base model's 1.818, and fine-tuning without any summary erodes the summarizing that midtraining taught. After 1,000 conversations the model's summaries score 0.23 and 79 of 100 end with `<|end|>`; after all 24,179 it no longer summarizes: asked for a summary, it tells a story (ROUGE-1 0.13, near the 0.08 to 0.12 of the models never taught to summarize, and none ended). A task taught in midtraining survives fine-tuning only if fine-tuning's data keep some of it, which is why fine-tuning mixtures carry the midtraining tasks along, as nanochat's did once the two stages were merged.

The loss on the assistant's tokens against the loss on every token (both from the base model). With the loss on every token, the model also learns to predict the system messages and the requests, 24% of our tokens. On our data this costs the assistant's turns little: their held-out loss is 1.64 nats per token against 1.63 with the mask, and the shares of answers ended,

of requested words used, and of sums right differ by no more than a single run's chance variation could, because our prompts are short against the answers. It forgot a little more of the stories (1.925 nats per token on the validation stories against 1.879). The mask matters more when the prompts are long, as for a story to summarize, and without it the model is also trained to write the user's side of a conversation.

Every weight against LoRA (§7, rank 8 on every attention and MLP matrix, 165,888 trained parameters, from the base model). LoRA alone learns the stories but not where its turn ends: it ends only 2 of 100 answers with `<|end|>`, writing *The end.* and going on with another story. (It also gets no sum right by our test, which reads the whole answer: it writes *237 + 682 = 919.* and goes on.) The reason is the token table, which LoRA leaves frozen. The table is tied to the output layer, so the row of `<|end|>` is also the vector that scores `<|end|>` as the next token, and pretraining, in which `<|end|>` was never the target, pushed that score down at every position (§11); in our run, the low-rank changes to the blocks did not raise it enough. Training the rows of the four new tokens, `<|system|>`, `<|user|>`, `<|assistant|>`, and `<|end|>`, as well, 768 more numbers, fixes it: LoRA then ends every answer, comes within 0.02 nats per token of fine-tuning every weight on the held-out stories, and gets 75% of the sums right, while training 166,656 numbers, 4.8% of the model's; it uses fewer of the requested words, 25% against 35%. In practice the rows of new tokens, or the whole table, are trained alongside LoRA's matrices for this reason. On our CPU LoRA took nearly as long, 33 minutes against 36: it saves memory, the optimizer's state and the gradients of the frozen weights, and only part of the computation, since the backward pass still runs through every layer.

G.3 Measuring whether the system message is followed

Does our chat model follow its system message? Our fine-tuning conversations state some features of each story in the system message, and one of them can be measured: 95% of the fine-tuning stories under *In your stories, the characters talk to each other.* contain dialogue, a quotation mark, against 34% of the others (Figure 48). We gave the chat model 100 held-out requests, each under the storyteller's system message alone and with that sentence added. With it, 64% of its stories contain dialogue; without it, 62%. Fine-tuned from the base model, the share rises from 60% to 71%. With 100 stories in each condition, a difference of a few points is within chance; the sentence moves our models far less than it moved the training stories.

The request explains why. Our requests are written from the stories' summaries, and the held-out stories behind them contain dialogue 60% of the time, about the share the model writes without being told. The request already says most of what the story will be; the dialogue sentence is one of several features, in a third of the conversations, and adds information only where it disagrees with the request, which in our data it rarely does. A model learns to follow a system message as far as its training conversations make the system message decide the answer.